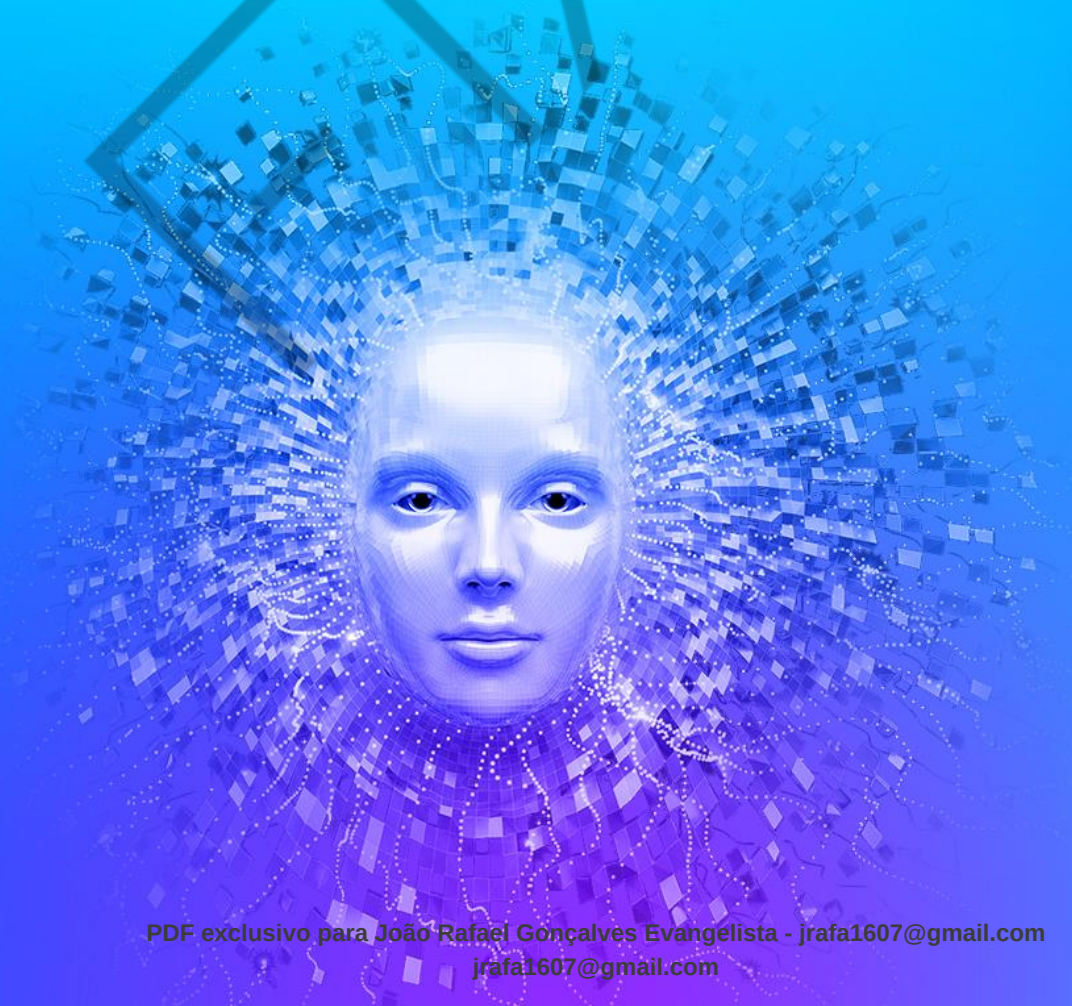


INTELIGÊNCIA ARTIFICIAL E COMPUTACIONAL

INTRODUÇÃO À **INTELIGÊNCIA ARTIFICIAL**

LEANDRO RUBIM E RICARDO SOARES



LISTA DE FIGURAS

Figura 1.1 – Sophia, um dos robôs mais famosos dotados de Inteligência Artificial e a primeira a receber a cidadania de um país (Arábia Saudita).....	4
Figura 1.2 – Camadas de uma Rede Neural Artificial	6
Figura 1.3 – Fractal de um floco de neve	6
Figura 1.4 – Ilustração sobre Internet, utilizando recursos da computação quântica	7
Figura 1.5 – Drone Ehang 184 sendo apresentado em Dubai	9
Figura 1.6 – Perspectiva do Drone Ehang 184 sobrevoando Dubai.....	9

EVANGELISTA

SUMÁRIO

1 INTRODUÇÃO À INTELIGÊNCIA ARTIFICIAL	4
1.1 Colocando os pingos nos “is” da Inteligência Artificial	4
1.2 Computação Natural	5
1.3 Inteligência Computacional	7
1.4 Aplicabilidade	8
REFERÊNCIAS	11

EVANGELISTA

1 INTRODUÇÃO À INTELIGÊNCIA ARTIFICIAL

1.1 Colocando os pingos nos “is” da Inteligência Artificial

Agora, sim! A Inteligência Artificial (IA), de fato, viralizou! Muito se estudou desde 1950 sobre a IA, suas definições e variações, soluções e aplicações. E devido à veloz evolução tecnológica das últimas duas décadas alinhada às necessidades de negócios e apoiada à necessidade humana, a IA se tornou efetivamente implementável nos sistemas.

Permitir que os sistemas aprendam com a zeta quantidade de dados disponíveis e forneçam análises e previsões concretas passou a ser indispensável ao mercado financeiro, no planejamento estratégico das empresas, nos sistemas de logísticas, nas ações de marketing, no tato e sentimento com o cliente, em um mundo mais sustentável, na educação, na criação de cidades inteligentes, na prevenção e cura de doenças, na Robótica e em todo e qualquer segmento da humanidade.



Figura 1.1 – Sophia, um dos robôs mais famosos dotados de Inteligência Artificial e a primeira a receber a cidadania de um país (Arábia Saudita)
Fonte: Wikipédia (2015)

As perguntas que não querem calar são: Aonde iremos chegar? Onde já podemos alcançar? Como implementar? Computador mais inteligente que o homem? Grandes pesquisadores já tinham um questionamento instigante desde os primórdios da IA: e o que será do ser humano quando os robôs forem mais inteligentes? Muitas destas perguntas já têm respostas.

Estamos em um momento ímpar, onde as grandes corporações estão simplificando o que antes dependia de algoritmos matemáticos complexos e linhas de códigos intermináveis, e disponibilizando em Cloud. A mistura de tecnologias emergentes, tais como Robótica, Big Data, IoT, blockchain e chatbots, com a IA permite a criação de agentes inteligentes para aplicações a soluções reais. Como exemplos bem acabados e extremamente funcionais, temos o Amazon Alexa, o IBM Watson, o Google Assistant, o Apple Siri, entre outros.

E ainda está redefinindo como os programas são escritos: humanos colaborando com computadores vão alavancar a programação de sistemas de um modo que nenhum dos dois conseguiria desenvolver sozinho. O que está mudando é o casamento das linguagens de interface com IA, que tende a revolucionar o futuro da programação, abrangendo tudo ao que se refere a tornar os sistemas inteligentes.

Na IA clássica, o grande objetivo é trabalhar com sistemas baseados em conhecimento ou sistemas especialistas. Por meio do mapa completo de dados de uma situação (por exemplo, histórico de entregas de produtos), algoritmos são desenvolvidos para aprender com este mapa novas situações que possam ser similares, permitindo traçar melhores rotas com baixos custos.

Em paralelo à evolução da IA clássica, outros termos surgiram, tais como: Computação Natural (1960), Inteligência Computacional (1994) e Aprendizagem de Máquina (2007).

1.2 Computação Natural

A Computação Natural (CN), formalizada em 2004 por Leandro N. de Castro & Fernando J. Von Zuben, permeia três formas de implementação:

- Computação em que ideias são obtidas através da observação da natureza, que é a inspiração para o desenvolvimento de soluções de problemas complexos. Exemplos de algoritmos nesta linha são: Redes Neurais Artificiais (1943), Computação Evolutiva (1965), Inteligência de Enxame (1988), Sistemas Imunológicos Artificiais (1999), entre outros.

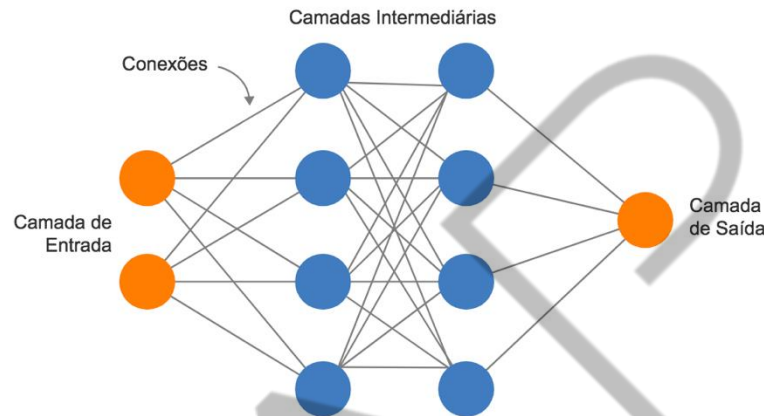


Figura 1.2 – Camadas de uma Rede Neural Artificial
Fonte: Conteúdo ICMC / USP (2017)

- Síntese de fenômenos naturais através da computação que envolve mecanismos de computação para sintetizar comportamentos naturais, padrões e processos biológicos. As principais linhas de atuação são os estudos sobre a vida de organismos artificiais, batizados de Vida Artificial (1998) e a Geometria Fractal (1982).



Figura 1.3 – Fractal de um floco de neve
Fonte: Metamorfose Digital (2014)

- Computação com mecanismos naturais que são novos paradigmas de computação e podem resultar em computadores altamente potentes, chamados computadores naturais, baseados em computação molecular, através de cadeias de DNA (1998), computação quântica (2000), entre outros.



Figura 1.4 – Ilustração sobre Internet, utilizando recursos da computação quântica
Fonte: BBCBrasil.com (2017)

1.3 Inteligência Computacional

O termo Inteligência Computacional (IC) surgiu em 1994 como uma forma de desassociar o que a IA clássica pretendia. Muitos pesquisadores defendiam que o mundo científico ainda estava distante de conhecer por completo a inteligência humana, portanto, era muito cedo para tentar criar algum tipo de inteligência artificial.

Os algoritmos de Redes Neurais Artificiais (RNA), Computação Evolutiva (algoritmos genéticos) e Lógica Fuzzy fizeram parte das técnicas de implementação do IC. O Machine Learning (Aprendizagem de Máquina), atualmente muito associado ao Big Data e ao Analytics, foi defendido por T. Mitchell em 1997 e surgiu dos sistemas baseados em conhecimento da IA clássica. O grande desafio é desenvolver sistemas capazes de aprender:

- Por si mesmos, através de experiências e comportamentos passados (aprendizagem não supervisionada).
- Por meio de entrada de mapas de dados (aprendizagem supervisionada).

- Interagindo com o ambiente (aprendizagem por reforço), por exemplo, dirigindo um carro.

Para a implementação de Machine Learning, diversas técnicas estão envolvidas, do uso de estatística para auxiliar na análise e predição de dados a técnicas de mineração de dados (Data Mining), algoritmos de árvore de decisão, redes Bayesianas e processos de clustering.

O Deep Learning (aprendizado profundo) é uma técnica de Machine Learning eficaz e precisa para a Aprendizagem de Máquina que utiliza grandes quantidades de dados não estruturados e possibilita uma representação hierárquica das camadas de dados.

Algoritmos de RNA são utilizados no Deep Learning justamente por permitir que o aprendizado de padrões ocorra. Quaisquer soluções que envolvam reconhecimento de voz, processamento de imagem, análise de comportamento, entre outras características, são aplicações factíveis de Deep Learning.

1.4 Aplicabilidade

O mais interessante do posicionamento da IA é ser altamente aplicável nas soluções de negócios. Instituições financeiras e de cobrança têm utilizado algoritmos de detecção de fraude que analisam padrões no processamento dos dados trafegados, buscando validar, encontrar erros de informações e minerá-los em grupos segmentados. O que permite que os sistemas sejam capazes de prever “candidatos” a fraudes, tendo por base situações anteriormente detectadas e mapeadas, ou ainda simular fraudes que jamais ocorreram, mas que tenham possibilidade de serem aplicadas.

Técnicas de análise forense, arquitetura de redes Bayesianas e algoritmos de classificação são muito utilizados na predição dos dados para detecção. Grandes montadoras têm apostado em veículos autônomos. Por meio de algoritmos de Deep Learning, é possível aperfeiçoar o reconhecimento de imagens, aumentando a segurança, além de permitir o aprendizado com a experiência realizada.

A Ford anunciou que lançará sua primeira frota de carros autônomos em 2021, e, em 2017, o fabricante chinês de drones Ehang começou a operar seu táxi aéreo sem piloto para trafegar pelos céus de Dubai.



Figura 1.5 – Drone Ehang 184 sendo apresentado em Dubai
Fonte: Google Imagens (2017)



Figura 1.6 – Perspectiva do Drone Ehang 184 sobrevoando Dubai
Fonte: Google Imagens (2017)

Os grandes centros médicos no mundo estão cada vez mais recheados de equipamentos e sistemas inteligentes para leituras de imagens médicas. A técnica de processamento de imagens utiliza métodos de IA, tais como Redes Neurais Artificiais e Lógica Fuzzy. O Google desenvolveu a API Google Vision, baseada em Aprendizagem de Máquina e reconhecimento de padrões, sendo possível, de

maneira muito simples, implementar a leitura de imagens e a exportação em texto do conteúdo da imagem.

Os cada vez mais atuantes chatbots como atendimento inteligente e automatizado é um grande canal de agilidade no atendimento sem a perda da qualidade da informação. O Poupatempo, projeto do Estado de São Paulo que oferece diversos serviços de emissão de documentos, atestados, licenciamento veicular, entre outros, implantou o Poupinha, um atendente virtual (Chatbot). Através do próprio Portal Poupatempo ou da página do Facebook, ele tira dúvidas do usuário sobre informações das condições, prazos, valores e retirada de documentos, atendendo, em média, 5 mil usuários ao dia.

A implementação da mineração de sentimentos e de opiniões tem sido outra grande aposta da utilização de IA. As redes sociais e as interações digitais, via os diversos dispositivos móveis, têm produzido uma grande quantidade de informações descontroladas e desorganizadas, afastando a personificação do ser humano perante a sociedade, produtos e serviços.

Uma reaproximação se faz necessária para um entendimento mais real dos significados intrínsecos dos posts, imagens, vídeos, conversas e likes. Classificar o grau de sentimento que o usuário dá às interações e suas opiniões implícitas ou explícitas é artefato sendo trabalhado pela IA em prol da personificação.

Onde houver Inteligência Humana sendo aplicada, é factível aplicar a Inteligência Artificial para auxiliar o desenvolvimento e a evolução dos negócios e da sociedade. É claro, há sempre a preocupação sobre a substituição da mão de obra humana pela Robótica. E sim, é um risco.

Mas assim como em outros momentos da evolução do planeta, houve a adaptação do ser humano e o surgimento de novas profissões, com a IA não será diferente. É só pensarmos em qual foi o impacto da eletricidade, das máquinas industriais, dos computadores e da Internet quando surgiram e quanto foram importantes para o equilíbrio do desenvolvimento humano.

REFERÊNCIAS

CASTRO, Leandro N. de; Ferrari, Daniel G. **Introdução à mineração de dados: conceitos básicos, algoritmos e aplicação**. São Paulo: Saraiva, 2016.

MITCHELL, Tom M. **The discipline of Machine Learning**. 2006. Disponível em: <<http://www-cgi.cs.cmu.edu/~tom/pubs/MachineLearningTR.pdf>>. Acesso em: 23 out. 2017.

SILVA, Rafael Rodrigues da. **Ford anuncia que irá lançar sua primeira frota de carros autônomos em 2021**. 04 jan. 2019. Disponível em: <<https://canaltech.com.br/carros/ford-anuncia-que-ira-lancar-sua-primeira-frota-de-carros-autonomos-em-2021-130163/>>. Acesso em: 18 jun. 2020.

VINHOLAS, Thiago. **Táxi voador começa a operar em Dubai a partir de julho**. 15 fev. 2017. Disponível em: <<https://www.airway.com.br/taxi-voador-comeca-operar-em-dubai-partir-de-julho/>>. Acesso em: 18 jun. 2020.

WIKIPÉDIA. **Sophia (robô)**. [s.d.]. Disponível em: <[https://pt.wikipedia.org/wiki/Sophia_\(rob%C3%B4\)](https://pt.wikipedia.org/wiki/Sophia_(rob%C3%B4))>. Acesso em: 18 jun. 2020.

INTELIGÊNCIA ARTIFICIAL
E COMPUTACIONAL

RESOLUÇÃO DE PROBLEMAS POR MEIO DE BUSCA

ERICK GALANI MAZIERO



2

LISTA DE FIGURAS

Figura 2.1. Possíveis movimentos das peças de um jogo de xadrez	6
Figura 2.2. Ilustração do sensoriamento de um carro autônomo	7
Figura 2.3. Mapa com localização do CD e LE para o problema de entrega de uma mercadoria	8
Figura 2.4. Exemplo de estrutura de dados em árvore.....	9
Figura 2.5. Árvore binária de busca balanceada	10
Figura 2.6. Ilustração de uma busca em largura	11
Figura 2.7. Ilustração de uma busca em profundidade	12
Figura 2.8. Possíveis caminhos entre o CD e o LE, para o problema do agente de entregas	15
Figura 2.9. Exemplo do estado inicial e final de um quebra-cabeça de oito peças ...	17
Figura 2.10. Configuração do quebra-cabeça, em um estado n.....	18

LISTA DE TABELAS

Tabela 2.1. Comparação entre estratégias de busca.....	13
--	----

EVANGELISTA

LISTA DE ALGORITMOS

Algoritmo 2.1. Algoritmo da busca A^*	16
---	----

EVANGELISTA

SUMÁRIO

2 RESOLUÇÃO DE PROBLEMAS POR MEIO DE BUSCA.....	6
2.1 Problemas e Soluções.....	6
2.1.1 Definição formal de um problema.....	7
2.2 Métodos de busca por solução.....	9
2.3 Busca sem informação	11
2.3.1 Busca em largura.....	11
2.3.2 Busca em profundidade.....	12
2.3.3 Comparação entre estratégias.....	13
2.4 Busca informada (heurística).....	14
2.4.1 Busca gulosa de melhor escolha.....	14
2.4.2 Busca A* (minimizando custos).....	15
2.4.3 Funções heurísticas.....	16
REFERÊNCIAS	19
GLOSSÁRIO	20

2 RESOLUÇÃO DE PROBLEMAS POR MEIO DE BUSCA

2.1 Problemas e Soluções

Há diversas maneiras de se resolver um problema, assim como há diversos tipos de problemas. Geralmente, os problemas podem ser solucionados com um conjunto de ações que levam a um objetivo (**solução**). Imagine um jogador de xadrez (**agente**), ele deve seguir um conjunto de ações que leve à vitória. A cada jogada, ele deve escolher uma peça, que tem uma quantidade finita de possíveis movimentos. A melhor escolha eleva as chances de o jogador ganhar a partida.

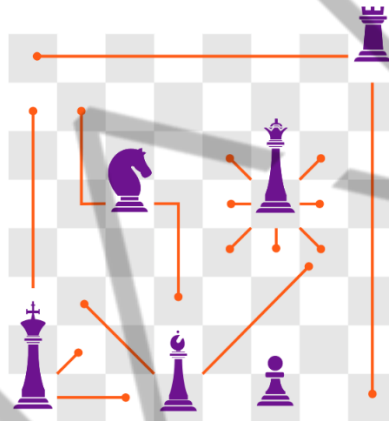


Figura 2.1. Possíveis movimentos das peças de um jogo de xadrez
Fonte: Google (2017)

Considere, como outro exemplo, um carro autônomo (**agente**) que deve se locomover até um destino final. Durante o percurso, seus sensores detectam uma série de objetos, como: ciclistas, pedestres, outros carros, postes, guias e a via. Baseado nesses reconhecimentos, o sistema autônomo (**inteligente**) deve escolher as melhores ações: como movimentar o volante, trocar de marcha, acelerar ou frear, dentre outras ações.

Quando as possibilidades (**ações**) para solução de um problema tornam-se quase inumeráveis, o espaço para se explorar uma solução deve ser considerado com técnicas que guiem a resolução do problema, levando em conta limitações de tempo, processamento e memória.

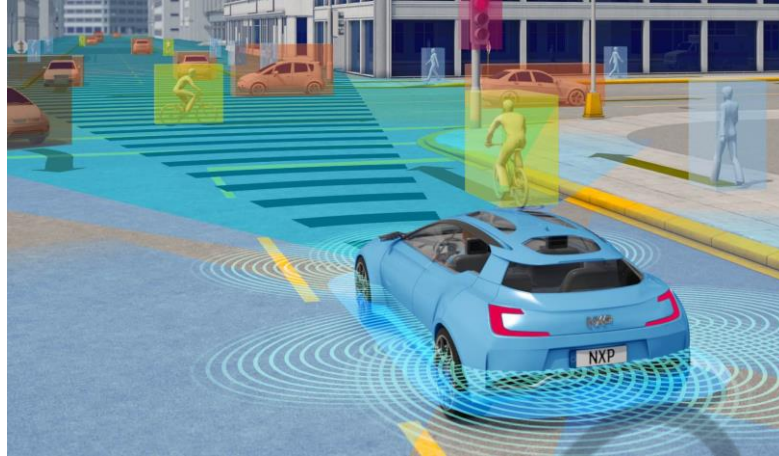


Figura 2.2. Ilustração do sensoriamento de um carro autônomo
Fonte: Google (2017)

Antes de explorar os métodos de busca de solução para um problema, é importante definir formalmente um problema.

2.1.1 Definição formal de um problema

Um problema pode ser definido formalmente com cinco componentes. Durante a definição, considere um agente que realiza entregas de produtos adquiridos pela internet partindo de um centro de distribuição (CD) até chegar ao local de entrega (LE), percorrendo o caminho (C) mais barato possível, de acordo com uma função de custo (FC).

Um problema é formalmente definido em cinco componentes (muito semelhante à definição de uma máquina de estados finitos, ou autômato):

1. Um **estado inicial** (s_0) a partir do qual o agente inicia. Tendo em vista o exemplo proposto, o agente inicia no CD, que é o s_0 .
2. Uma descrição das **ações** (a_n) possíveis ao agente. Dado um estado s , $AÇÕES(s)$ consiste em um conjunto de possíveis ações. Por exemplo, $AÇÕES(s_0)$ consiste nas vias que partem do CD.
3. Um **modelo de transição** entre as ações, que define o próximo estado s_y , dada a escolha de uma ação, estando no estado s_x . É formalizado como $RESULTADO(s_x, a) = s_y$. Esses três componentes, estado inicial, ações e o modelo de transição compõem o **espaço de estados** do problema. O espaço de estados pode ser representado como um **grafo** dirigido, em que

os nós são os estados e as arestas são as ações. Nesse grafo, um **caminho** é uma sequência de estados conectados por ações (arestas).

4. Um teste de **objetivo**, que verifica, a cada estado, se o estado é o objetivo. Para alguns problemas, pode-se ter mais de um **estado objetivo**, que define a solução do problema. No exemplo dado, o estado objetivo é o local de entrega (LE) da mercadoria adquirida.
5. Uma **função de custo do caminho**, que atribui um valor numérico a cada caminho do grafo. Na função de custo do agente entregador a quilometragem, por simplicidade, pode ser considerada como função de custo do caminho, quanto menor o caminho, menor o custo.

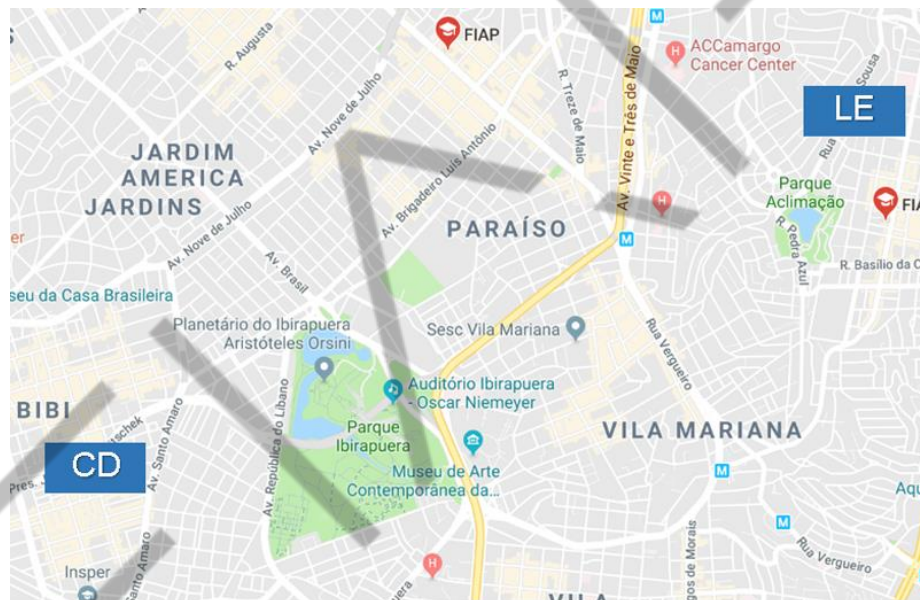


Figura 2.3. Mapa com localização do CD e LE para o problema de entrega de uma mercadoria
Fonte: Google (2017)

A figura ilustra o problema do agente que realiza entregas, partindo do CD, com objetivo de chegar ao LE com o menor custo do caminho. Como pode ser visto, há vários caminhos e, conseqüentemente, diversos custos. A busca pelo melhor caminho pode ser feita de diversas maneiras, como veremos mais à frente, neste capítulo.

A formalização realizada anteriormente é abstrata e possibilita a criação de um modelo simples para a resolução do problema. No mundo real, no entanto, há inúmeras variáveis a serem consideradas na modelagem do problema, aumentando as chances de falha do projeto.

No exemplo do agente entregador, além de considerar o caminho mais curto, a modelagem real deve levar em consideração outras variáveis, tais como:

- Tempo da viagem;
- Horário de trabalho do entregador, que deve se adequar à legislação trabalhista vigente;
- Velocidade das vias;
- Condições reais do trânsito nas vias;
- Número de cruzamentos;
- Dentre muitas outras variáveis.

Pode pensar em mais alguma variável para o problema do entregador? Lembre-se de que o veículo utilizado no transporte das mercadorias tem espaço limitado e, durante um caminho, o agente tem mais de uma entrega a realizar.

2.2 Métodos de busca por solução

Feita a formulação de um problema, deve-se trabalhar para encontrar a sua solução. De acordo com o modelo proposto, uma solução consiste em uma sequência de ações possíveis, partindo do estado inicial a um estado objetivo.

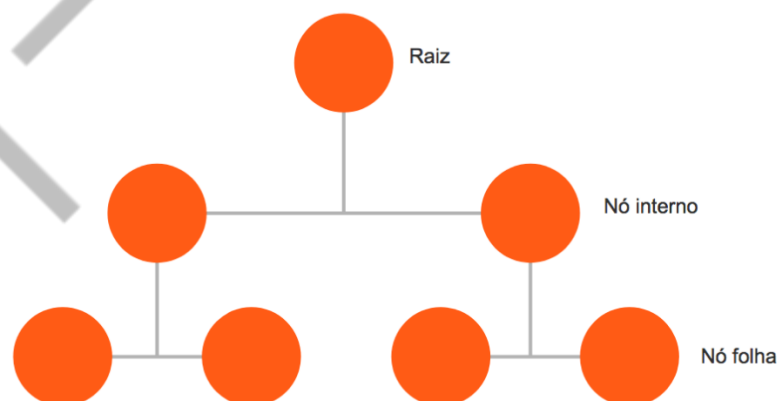


Figura 2.4. Exemplo de estrutura de dados em árvore
Fonte: Google (2017)

Cada possível solução de um problema forma o que chamamos de **árvore de busca**, originada do universo de possíveis soluções. Uma árvore consiste em uma

estrutura de dados não linear composta por nós (informações) organizadas hierarquicamente. Vale lembrar que, na computação, árvore é uma das principais estruturas de dados utilizadas e há diversos algoritmos eficientes de busca nessa estrutura.

Diversos algoritmos de ordenação, muito explorados em disciplinas introdutórias de Ciências da Computação, utilizam estruturas de dados de árvores. Por exemplo, um vetor de números inteiros pode ser armazenado em uma **árvore binária de busca**. Nessa árvore, os nós à esquerda do nó raiz contêm valores menores que o valor presente na raiz. Os nós à direita contêm valores maiores.

Essa restrição de ordenação é recursivamente aplicada a cada nó interno da árvore, até chegar a uma folha da estrutura. Isso garante uma complexidade computacional aceitável, mesmo quando a quantidade de números armazenados é muito grande. Se a árvore estiver balanceada, isto é, a quantidade de nós à direita e à esquerda da raiz é quase a mesma, a complexidade de busca é dada por $\log_2 n$, sendo n a quantidade de números armazenados. A figura exemplifica uma árvore binária de busca balanceada. Por exemplo, se armazenarmos 1 milhão de números, será necessário percorrer em torno de 20 nós da árvore para encontrar um número desejado ($\log_2 1000000 = 19.9315685693$).

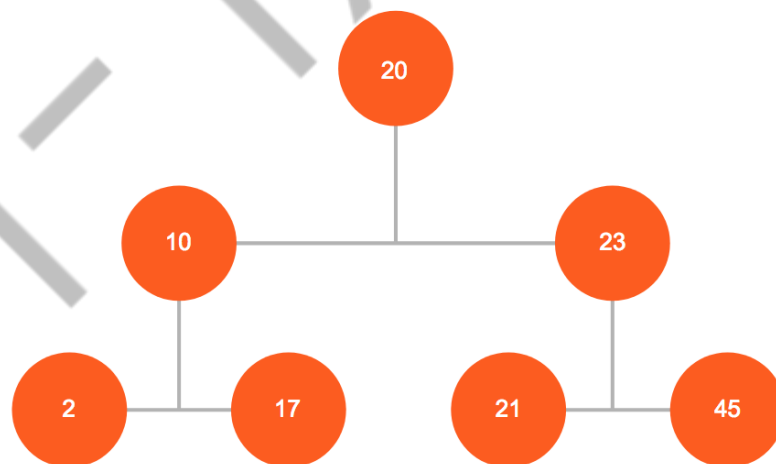


Figura 2.5. Árvore binária de busca balanceada
Fonte: Google (2017)

Dado um problema, basicamente pode-se optar por duas estratégias de busca: **busca sem informação** e **busca informada** com o uso de **heurísticas**. Nas próximas subseções serão apresentadas essas duas estratégias.

2.3 Busca sem informação

A busca sem informação, ou **busca cega**, é realizada sem informação adicional sobre os estados, além da definição do problema. A estratégia é gerar novos estados e verificar se um estado objetivo é alcançado ou não. Difere na ordem em que os estados são explorados: **busca em largura** ou **busca em profundidade**.

2.3.1 Busca em largura

A busca em largura (BFS – *Breadth-First Search*) é uma estratégia em que o nó raiz é verificado antes dos nós filhos do nó raiz. Essa estratégia é utilizada recursivamente para os nós internos da árvore (subárvores). A figura ilustra a busca em largura. Primeiro o nó A é visitado e seus filhos (B e C) são marcados para visita. Posteriormente, o nó B é visitado e seus filhos marcados para visita. A seguir, o nó C, filho de A é visitado e seus filhos marcados para visita. Esse procedimento é continuado até que todos os nós sejam visitados.

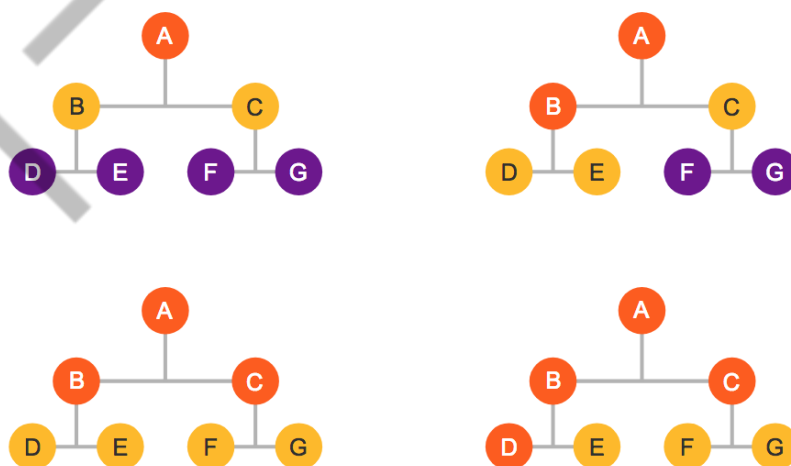


Figura 2.6. Ilustração de uma busca em largura
Fonte: Google (2017)

No exemplo, A é o nó raiz da árvore e seus filhos formam subárvores: B é raiz da subárvore e tem filhos D e E. Já C é raiz da subárvore formada por F e G. Esse é um exemplo de árvore binária pois cada nó tem dois filhos. No entanto, pode-se ter árvores n-ária, isto é, com n (fator de ramificação) filhos cada nó ($n=2$ para árvore binária).

2.3.2 Busca em profundidade

A figura ilustra a busca em profundidade (DFS – *Depth-First Search*). Veja a diferença em relação à busca em largura. Nesse procedimento, busca-se o nó mais profundo a partir da raiz. Dessa forma, visita-se o nó A, depois o B para se chegar ao D, que é um dos nós mais profundos da árvore. Retorna-se ao B para visitar o E, outro dos nós mais profundos. Esse procedimento continua até que toda a árvore seja coberta.

Nessa busca, toda uma subárvore à esquerda é visitada primeiro, para depois considerar a subárvore à direita. Caso a solução do problema fosse o nó C, ele só seria encontrado depois da visita aos nós A, B, D e E.

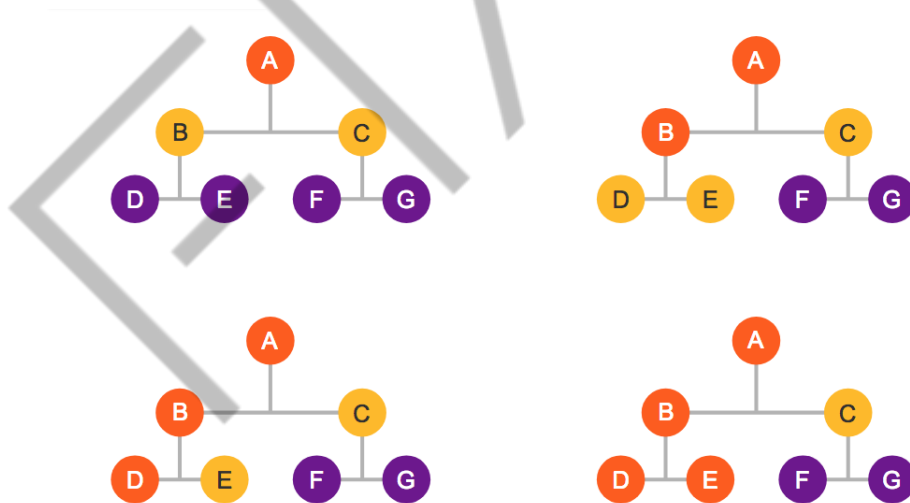


Figura 2.7. Ilustração de uma busca em profundidade
Fonte: Google (2017)

2.3.3 Comparação entre estratégias

Há diversas outras estratégias de busca não informadas, tais como:

- Busca com Custo Uniforme;
- Busca em Profundidade Limitada;
- Busca de Aprofundamento Iterativo e
- Busca Bidirecional.

Aqui vale citar quatro critérios utilizados para se escolher dentre as estratégias de busca:

1. **Completeza:** o algoritmo garante encontrar a melhor solução?
2. **Otimalidade:** encontra a melhor de todas as soluções possíveis ao problema?
3. **Complexidade de tempo:** quanto tempo para encontrar a solução.
4. **Complexidade de espaço:** quanto de memória é necessária para encontrar a solução.

Tendo esses quatro critérios em consideração, a tabela compara as estratégias de busca não informada. Na tabela, considere b como o fator de ramificação, m a profundidade máxima da árvore e d a profundidade da solução mais rasa (mais próxima da raiz). $O(n)$ é a notação assintótica, que indica a complexidade de tempo ou espaço para valores muito grandes de n .

Tabela 1. Comparação entre estratégias de busca

Critério	Largura	Profundidade
1	Sim	Não
2	Sim	Não
3	$O(b^d)$	$O(b^m)$
4	$O(b^d)$	$O(b^m)$

Fonte: Google (2017)

Olhando para a tabela, parece que a busca em largura é a melhor opção, mas em termos de complexidade. A busca em largura pode levar muito tempo e ocupar muito espaço em memória para problemas cuja solução esteja a uma profundidade

alta (d). Já esse custo é assintoticamente o mesmo para a busca em profundidade, que depende de m (profundidade máxima da árvore).

2.4 Busca informada (heurística)

Já a busca informada é assim chamada pois se tem informação, dado um conjunto de possíveis estados, de qual é mais promissor, dado o estado atual. Esse conhecimento adicional é obtido por heurísticas, ou aproximações, utilizadas durante a busca.

Nas estratégias de busca informada, a cada passo utiliza-se uma **busca de melhor escolha**. A escolha de um nó a ser percorrido é feita com base na **função de avaliação** $f(n)$. Assim, dado um estado atual, a função é aplicada a cada possível expansão da solução com a finalidade de avaliar o custo. A expansão com menor custo é escolhida.

A diferença entre as estratégias de busca informada consiste na escolha da função $f(n)$. Como parte de $f(n)$, a maior parte das estratégias utiliza uma função heurística $h(n)$, que estima o caminho de menor custo do estado n ao estado objetivo.

2.4.1 Busca gulosa de melhor escolha

A busca gulosa expande a busca para o nó que seja o mais próximo do estado objetivo considerando apenas o caminho do estado atual ao próximo estado. Considerando o exemplo dado no início deste capítulo, do agente de entregas, se o agente está no CD, objetivando ir ao LE, ele pode utilizar como $h(n)$ a distância em linha reta para a próxima rua. A figura ilustra um possível cenário para o agente de entregas. Ir do CD ao A custa 1, do CD ao B custa 2 e do CD ao C custa 5.

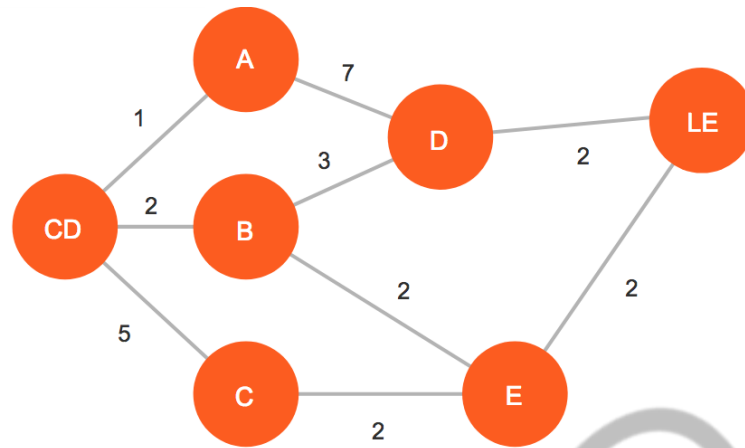


Figura 2.8. Possíveis caminhos entre o CD e o LE, para o problema do agente de entregas
Fonte: Google (2017)

A busca gulosa, estando no CD, considera o menor custo para a próxima expansão, isso levará ao nó A, que custa 1. A partir de A, a próxima expansão levará a D e, depois a LE. Esse caminho tem custo total de $1 + 7 + 2 = 10$. Obviamente, esse não é o melhor caminho, que é $CD - B - D - LE$, com custo total de $2 + 3 + 2 = 7$.

Nessa estratégia $f(n) = h(n)$, ou seja, a função de avaliação é igual à função heurística de escolher o menor custo a partir do estado atual.

2.4.2 Busca A* (minimizando custos)

A busca A* (“A estrela”) tem como função de avaliação $f(n) = g(n) + h(n)$, sendo $g(n)$ o custo para ir do nó inicial ao nó n e $h(n)$ o custo estimado para ir do nó n ao nó objetivo. Essa estratégia é uma combinação de aproximações heurísticas, como a busca em largura e o famoso algoritmo de *Dijkstra*. Ele é muito utilizado em aplicativos de rotas de deslocamento entre localidades e até na resolução de quebra-cabeças. Também é muito utilizado em jogos.

Considere:

Q = um conjunto de nós a serem percorridos;

P = conjunto de nós explorados;

S = o estado inicial da busca.

```
Inicialize Q com S
Repita
    Se Q está vazio
        Retorne "Falha"
    Obtenha nó n de Q, com melhor f(n)
    Se n é um estado objetivo
        Retorne "Solução(n)"
    Adicione n a P
    Para cada filho m de n, faça
        Adicione m a Q
```

Algoritmo 2.1. Algoritmo da busca A*
Fonte: Google (2017)

A complexidade espacial (de memória) dessa estratégia pode se tornar exponencial, pois todos os nós explorados devem ser mantidos em memória. Essa limitação do algoritmo pode ser reduzida com o uso do aprofundamento iterativo (IDA* - *Iterative Deepening A**).

Para finalizar a apresentação de diversas estratégias de busca, tanto cegas quanto informadas, que foram projetadas por cientistas da computação, temos a seguinte pergunta: seria possível um agente aprender para melhorar a busca? A resposta é sim, e baseia-se na ideia do **espaço de estados do nível meta**. Basicamente, a cada passo computacional da busca A*, armazenam-se erros cometidos para evitar que se cometa o mesmo erro futuramente, evitando gastos computacionais desnecessários, tornando a busca mais inteligente.

2.4.3 Funções heurísticas

Para finalizar este capítulo, apresenta-se, com o exemplo do quebra-cabeça de oito peças, a noção geral de heurística. Considere o problema em um tabuleiro de 3 por 3, com 8 peças, cujo estado objetivo é ter as peças arranjadas conforme ilustrado na figura.

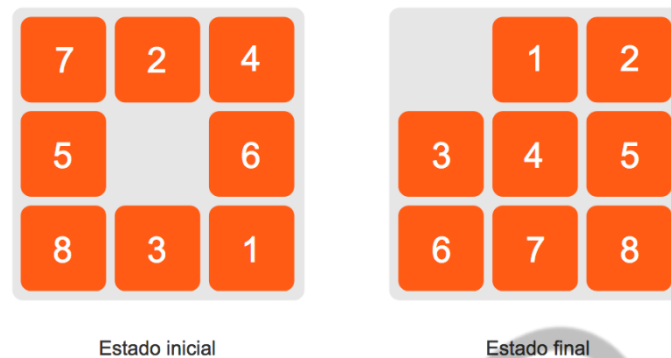


Figura 2.9. Exemplo do estado inicial e final de um quebra-cabeça de oito peças
 Fonte: Google (2017)

O custo médio para solucionar o quebra-cabeça de oito peças é de 22 passos. O fator de ramificação é cerca de 3 (para uma peça que estiver no meio do tabuleiro são possíveis 4 movimentos; quando estiver em um dos cantos, 2; e quando estiver ao longo da borda, 3 movimentos). Se fosse feita uma busca exaustiva em uma árvore de profundidade 22, seriam necessários 31.381.059.609 estados!

Em vez de explorar todas as possíveis movimentações de peças, de maneira exaustiva, duas heurísticas ($h(n)$) poderiam ser utilizadas:

$h1$: número de peças fora do lugar. Assim, na figura, $h1 = 8$ para o estado inicial, pois todas as peças estão fora do lugar.

$h2$: soma das distâncias das peças de suas posições-objetivo. Dado que as peças não podem se movimentar na diagonal, utiliza-se a **distância de Manhattan**, ou **distância de quarteirão**, que é a soma das distâncias horizontais e verticais. Para o estado inicial, temos:

$$h2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$$

Como exercício, quais seriam os valores para $h1$ e $h2$, dado o estado n, da figuraFigura 2.10?

Estado n

Figura 2.10. Configuração do quebra-cabeça, em um estado n
Fonte: Google (2017)

REFERÊNCIAS

RUSSEL, Stuart; NORVIG, Peter. **Inteligência Artificial**. Rio de Janeiro: Elsevier, 2013.

EVANGELISTA

GLOSSÁRIO

Análise assintótica	A análise assintótica se preocupa em verificar como um algoritmo/solução se comporta quando os dados do problema são de grande magnitude.
Autômato	Modelo matemático de uma máquina de estados finitos. Esse modelo é definido como tendo diversos estados e possíveis transições entre esses estados. Serve para modelar uma máquina ou um computador simples.
Dijkstra	Cientista da computação holandês <i>Edsger Dijkstra</i> que solucionou (em 1959) o problema do caminho mais curto em um grafo dirigido.
Grafo	Modelo formado por vértices e arestas, que ligam os vértices. As arestas podem ter direção, indo de um vértice a outro, passando a chamar dígrafo (Grafo dirigido).

INTELIGÊNCIA ARTIFICIAL E COMPUTACIONAL

REPRESENTAÇÃO DO CONHECIMENTO E RACIOCÍNIO

ERICK GALANI MAZIERO



LISTA DE FIGURAS

Figura 3.1. Exemplo de um recorte de uma ontologia de mundo.	12
Figura 3.2. Exemplo de rede semântica.	14



LISTA DE TABELAS

Tabela 3.1. Tabela verdade com resultados da aplicação de operadores entre P e Q.	7
--	---

EVANGELISTA

SUMÁRIO

3 REPRESENTAÇÃO DO CONHECIMENTO E RACIOCÍNIO	5
3.1 Agentes baseados em conhecimento.....	5
3.1.1 Representação do Conhecimento.....	5
3.1.2 Lógica proposicional.....	6
3.1.3 Lógica de primeira ordem.....	8
3.1.4 Engenharia de Conhecimento.....	10
3.1.5 Engenharia ontológica.....	11
3.1.6 Redes semânticas.....	13
REFERÊNCIAS	15
GLOSSÁRIO	16

3 REPRESENTAÇÃO DO CONHECIMENTO E RACIOCÍNIO

3.1 Agentes baseados em conhecimento

Os problemas apresentados no capítulo anterior têm um conhecimento do problema de uma forma bastante limitada: conhecem o estado atual, os estados percorridos e os possíveis próximos estados, além de uma função que verifica se um estado é um estado objetivo ou não. Essa perspectiva não tem muitas chances de sucesso em problemas que admitem um grande número de estados, dadas as limitações computacionais, tanto de tempo de processamento quanto de memória, para a resolução do problema.

Na Inteligência Artificial, há o que se chama de **agentes baseados em conhecimento**, que, como o nome diz, utilizam uma **base de conhecimento** (KB - *Knowledge Base*). Um KB armazena **sentenças**, que, desaliado de um sentido puramente linguístico, representa uma **asserção** sobre o mundo. As sentenças são expressas em uma **linguagem de representação do conhecimento**. Uma sentença pode ser chamada de **axioma**, quando tal sentença é dada originalmente, isto é, sem derivação de outra sentença.

O conhecimento em um KB é geralmente dinâmico, ou seja, novas sentenças são adicionadas e outras podem ser removidas ou modificadas. Vamos referenciar a inserção de novas sentenças pela operação TELL e a busca por uma sentença, como ASK. Essas operações podem utilizar-se de **inferências** de novas sentenças a partir das existentes no KB.

3.1.1 Representação do Conhecimento

Mas, afinal, o que é conhecimento? Antes, considere como **dados** quaisquer sequências numéricas ou alfanuméricas que não possuem um significado associado (3, 4, 5, ...). Quando esses dados são organizados e rotulados (tamanho 3, camada 4, 5 unidades, ...), passam a ser chamados de **informação**, pois significam algo. **Conhecimento** é a composição e organização de informações que permitem raciocínio por parte de um ser humano ou de uma máquina (objeto de tamanho 3, na camada 3, com 5 unidades, ...).

Tratando-se de representar o conhecimento de forma a permitir o raciocínio por parte de um agente inteligente, uma das formas mais antigas é o uso da **lógica**. Inclusive, diversos sistemas especialistas do início da IA tiveram a lógica como sua base de conhecimento e raciocínio.

Lógica é uma linguagem de propósito geral utilizada para descrever fatos (verdades) em um *mundo*. A lógica também possui mecanismos para operar sobre os fatos visando o raciocínio. Assim como outras linguagens, tem-se uma **sintaxe**, que define a formação das sentenças e uma **semântica**, que dá o significado às sentenças. Cada sentença é sempre avaliada em **verdade** ou **falso**.

Linguagens naturais, como o Português e o Inglês também são uma forma de representar conhecimento. Tanto é que o conhecimento deste capítulo está representado utilizando o Português. No entanto, como será percebido na continuação deste capítulo, a linguagem natural é muito complexa e tem muitos fenômenos de difícil tratamento computacional, como a ambiguidade. Portanto, vamos nos atentar a algumas formas simbólicas de representação do conhecimento cujo tratamento é mais plausível, computacionalmente.

3.1.2 Lógica proposicional

A **lógica proposicional** é um tipo de lógica bem simples. Sua sintaxe define **sentenças atômicas**, que consistem em um **símbolo proposicional**. Aqui definiremos que as palavras que iniciam com letra maiúscula (P, Q, Humano, Máquina, etc.) são símbolos proposicionais. Cada símbolo proposicional é sempre avaliado em **verdade** ou **falso**. **Sentenças complexas** são feitas a partir de sentenças mais simples com o uso de **conectivos lógicos**: $\neg$ (*não*), $\wedge$ (*e*), $\vee$ (*ou*), $\Rightarrow$ (*implica*) e $\Leftrightarrow$ (*se e somente se*).

Por exemplo, o uso do operador de negação para o símbolo P é $\neg P$, que significa “não P” ou “P é falso”, ou “não é o caso de P”. Os outros operadores dependem de uma **tabela verdade**. Considere a tabela verdade ilustrada na Tabela 3.1, a seguir, com os operadores $\neg$ (*não*), $\wedge$ (*e*), $\vee$ (*ou*), $\Rightarrow$ (*implica, condicional*) e $\Leftrightarrow$ (*se e somente se, bicondicional*).

Tabela 3.1. Tabela verdade com resultados da aplicação de operadores entre P e Q

P	Q	$\wedge$	$\vee$	$\Rightarrow$	$\Leftrightarrow$
V	V	V	V	V	V
V	F	F	V	F	F
F	V	F	V	V	F
F	F	F	F	V	V

Fonte: Google (2017)

Pela tabela verdade, a sentença $P \wedge Q$ (P e Q) assume valores verdadeiros (V) quando pelo menos um dos símbolos proposicionais forem falsos e só assume valor verdadeiro (V), quando os dois símbolos são verdadeiros. O operador $\vee$ (ou), no entanto, leva a um valor verdade quando pelo menos um dos símbolos é verdade e só assume falso quando os dois são falsos. Como exercício, interprete os outros dois operadores.

Embora o exposto acima pareça um tanto formal e teórico, suas aplicações são bem práticas na IA, principalmente em agentes baseados em conhecimento. Imagine um agente (um *Chatbot*, por exemplo) que deve responder “sim” (verdade) ou “não” (falso), para afirmações de um usuário. Ele deve avaliar perguntas como:

1. A terra é redonda?
2. A terra é quadrada **E** a lua é redonda?
3. A neve é branca **SE** a água é incolor?
4. A terra é quadrada **ou** a lua é redonda?
5. A terra **NÃO** é quadrada **E** a lua é redonda?

Baseado em nosso conhecimento de mundo, as sentenças 1, 3, 4 e 5 são verdade (V) e a sentença 2 é falso. Concorda?

Embora útil em alguns cenários, para representar um conjunto de sentenças (conhecimento), sua representatividade, em cenários mais complexos, é limitada. A lógica proposicional manipula bem conhecimentos que podem ser expressos pelos conectivos lógicos entre as sentenças atômicas. Mas e para expressar coisas como “Existe algum ... tal que...” ou “Para todo ...”. Como representar em lógica proposicional as sentenças: “Ser mais jovem que outrem”, ou “Existe um X que é maior que todos os outros Xs”? Veja no próximo subtópico uma linguagem com maior poder de expressão.

3.1.3 Lógica de primeira ordem

A lógica de primeira ordem (LPO) tem poder de expressão maior que a lógica proposicional. Sendo estudada por décadas, a LPO é utilizada como base para outras linguagens como o Prolog. A LPO estende lógica proposicional. Enquanto esta se ocupa da existência dos fatos, aquela se ocupa da existência de objetos e de relações entre eles.

Na LPO, as **sentenças atômicas** têm o seguinte formato: $P(t_1, \dots, t_n)$. P é chamado de **predicado** e tem um ou n **argumentos**. A LPO também contém **quantificadores**, a saber, $\exists x$ (para todo x) e $\forall x$ (para qualquer x). Tem-se também os **termos**, que são constantes, e, geralmente, se referem a objetos do mundo real, concretos ou abstratos. Assim, podemos expressar fatos por meio de predicados, como $\text{homem}(\text{joão})$ ou $\text{mulher}(\text{maria})$. Esses predicados podem significar que joão é um homem e maria é uma mulher.

Para expressar os quantificadores, temos que definir o conceito de variável, que será denotada com o uso de uma letra latina minúscula, geralmente mais do final do alfabeto, tais como x, y, z e w . Essas variáveis podem ser pensadas como posições de memória que podem ser ocupadas ou instanciadas por valores como joão e maria. Desta forma, podemos ter:

- $\text{homem}(x)$, em que x é um homem;
- $\text{mulher}(x)$, em que x é uma mulher;
- $\text{pai}(x, y)$, em que x é pai de y ;
- $\text{mãe}(x, y)$, em que x é mãe de y .

E, dados os predicados anteriores, $\forall y$ (para qualquer y), se $\text{pai}(x, y)$ então $\text{maisnovo}(y, x)$. Em outras palavras, se x é pai de y , então y é mais novo que x .

Para um conhecimento como: *todo filho é mais novo que seu pai*, definimos:

$$\forall y \left(\text{pai}(x, y) \rightarrow \exists x (\text{maisnovo}(y, x)) \right)$$

Veja que o predicado *homem* admite apenas uma variável, sendo considerado um predicado unário. Já o predicado *maisnovo* é binário, pois admite duas variáveis.

Dada uma base de conhecimento expressa em LPO, algumas inferências podem ser feitas para responder a algumas questões. Por exemplo, imagine uma base de conhecimento em que se definem algumas relações familiares. Algumas conclusões podem ser inferidas.

Sejam:

- $\text{pai}(x, y)$: x é pai de y;
- $\text{filho}(x, y)$: x é filho de y;
- $\text{irmão}(x, y)$: x é irmão de y.

Se quisermos descobrir quem é *irmão* de uma dada pessoa, podemos codificar:

$$\forall x \forall y \forall z (\text{pai}(x, y) \wedge \text{filho}(z, x) \rightarrow \text{irmão}(y, z))$$

Se instanciarmos $y = \text{pedro}$, teremos que qualquer instância de z será *irmão* de *pedro*, veja:

$$\forall x \forall y \forall z (\text{pai}(x, \text{pedro}) \wedge \text{filho}(z, x) \rightarrow \text{irmão}(\text{pedro}, z))$$

Se nossa base de conhecimento contiver:

- $\text{filho}(\text{carlos}, \text{amaral})$
- $\text{filho}(\text{pedro}, \text{joão})$
- $\text{filho}(\text{estância}, \text{amaral})$
- $\text{pai}(\text{amaral}, \text{lorena})$
- $\text{pai}(\text{amaral}, \text{pedro})$

Concluimos que *pedro* é irmão de *lorena*.

Para entendermos um pouco mais sobre os quantificadores, considere

$\forall x \exists y \text{ ama}(x, y)$: Para todo x, existe um y, tal que x, ama y, em outras palavras, todo mundo ama alguém. Porém, se quisermos dizer que existe um alguém que é amado por todo mundo, teremos que escrever $\exists y \forall x \text{ ama}(x, y)$: existe um y que, para todo x, x ama y.

Um agente baseado em conhecimento pode armazenar novas sentenças em BK, utilizando a operação TELL. Por exemplo, $\text{TELL}(\text{BK}, \text{pai}(\text{amaral}, \text{murilo}))$ adiciona

em BK o conhecimento de que *amaral* é pai de *murilo*, gerando um novo irmão para *pedro*.

3.1.4 Engenharia de Conhecimento

Na seção anterior, foi definida a LPO como uma forma de representar um conhecimento (BK), com a finalidade de se raciocinar e resolver problemas. Para se gerar a BK, tem-se um processo chamado **engenharia de conhecimento**. Um engenheiro de conhecimento investiga o domínio do problema, aprende conceitos importantes e define uma representação formal dos objetos e relações entre esses objetos no domínio.

Nesse processo podem-se gerar bases de conhecimento para um domínio bem circunscrito ou abordagem de uma variedade maior de conhecimentos humanos. Quanto maior a variedade, maior a complexidade da BK e mais desafiadora é a engenharia de conhecimento.

As seguintes etapas são necessárias:

1. *Identificar a tarefa*: deve-se delinear as questões que a base de conhecimento suportará e os tipos de fatos disponíveis.
2. *Agregar conhecimento relevante*: ou o engenheiro deve ser especialista do domínio ou fazer uma aquisição de conhecimento com um especialista do domínio. Por exemplo, para gerar uma base de conhecimento sobre diagnósticos médicos de uma dada classe de doenças, um especialista nessa classe de doenças deve ser amplamente consultado.
3. *Definir um vocabulário de predicados e constantes*: aqui se convertem os conhecimentos adquiridos em predicados e constantes em LPO, por exemplo. Definem-se predicados unário, binários e suas significações, por exemplo $\text{pai}(x, y)$ indica que x é pai de y , ou que y é pai de x ?
4. *Codificar o conhecimento geral do domínio*: o engenheiro de conhecimento escreve os axiomas (termos iniciais, não provindos de inferência) correspondente a todos os termos do vocabulário. Essa etapa permite ao especialista verificar se o conhecimento adquirido foi corretamente

convertido na representação lógica. Quando erros são identificados, deve-se retornar ao passo 3.

5. *Codificar uma descrição da instância específica do problema*: envolve a escrita de sentenças atômicas sobre o problema, instanciando predicados com conhecimentos adquiridos.
6. *Formular consultas de inferência para obter respostas*: aqui pode-se testar o funcionamento do raciocínio sobre os axiomas e fatos codificados nas etapas anteriores, inferindo novos conhecimentos, conforme nossos objetivos de utilizar a base de conhecimento. Utiliza-se todo o poder de inferência da LPO, não necessitando que se escreva um programa para cada inferência que se deseja obter.
7. *Depurar a base de conhecimento*: Se um axioma estiver ausente da base, algumas consultas poderão não ser respondidas. Axiomas ausentes ou axiomas muito fracos podem ser identificados quando a inferência é interrompida de forma inesperada. Faz-se necessária a revisão dos passos anteriores, garantindo que as consultas desejadas sejam respondidas satisfatoriamente.

3.1.5 Engenharia ontológica

Nas seções anteriores o enfoque foi na forma de representar conhecimento, agora vamos tratar um pouco a respeito do conhecimento que deve ser armazenado em uma base de conhecimento.

A **engenharia ontológica** desempenha papel importante na representação do conhecimento, pois organiza tudo no mundo em uma hierarquia de categorias. Dessa forma, são definidas categorias de objetos, substâncias e medidas, eventos e conhecimentos abstratos.

A figura ilustra a criação de uma ontologia. No nível mais superior, encontra-se “QualquerCoisa”, indicando que essa ontologia pode organizar todos os conceitos do mundo real. Representar *tudo*, no entanto, é uma tarefa muito complexa e quase inatingível. Embora comece com “QualquerCoisa”, o que se especifica de cima para baixo na ontologia pode ser específico a um determinado domínio.

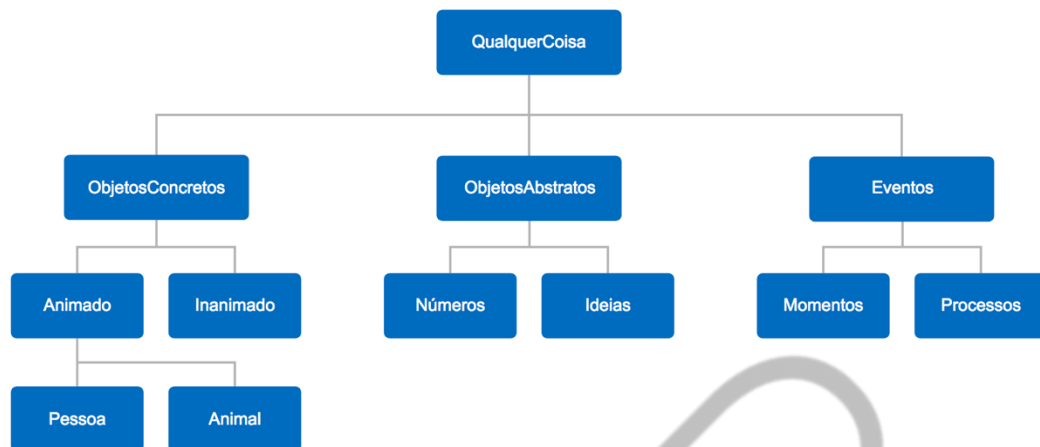


Figura 3.1. Exemplo de um recorte de uma ontologia de mundo
Fonte: Google (2017)

A Wikidata é uma iniciativa livre que busca organizar o conteúdo do mundo para leitura tanto humana quanto de máquinas. No momento da escrita desse material, mais de 87 milhões de fatos estão organizados hierarquicamente na Wikidata e abertos para uso público.

A ontologia ilustrada anteriormente é chamada de **ontologia superior**, pois define conceitos mais gerais na parte superior e vai especificando os conceitos à medida em que desce na ontologia.

A cada conceito que é especificado, deve-se levar em conta as similaridades dos objetos que pertencerão a cada ramo. Por exemplo, “Qualquer coisa” pode ser um “ObjetoConcreto” ou um “ObjetoAbstrato”. “ObjetoConcreto”, por sua vez, pode ser definido com o “Animado” (como pessoas e animais) ou “Inanimado” (objetos que não têm vida própria).

Embora, nesse material, tenha sido utilizada a LPO para representar os conhecimentos de mundo, as exceções (muito comuns no mundo real) não são facilmente tratadas. Por exemplo, dizer que *laranjas são da cor laranja* nem sempre é verdade, pois laranjas podem ser verdes ou até amarelas, como boa parte das frutas e legumes.

A definição de **categorias** é parte importante da engenharia ontológica, pois embora a interação no mundo real ocorra com objetos específicos, boa parte do raciocínio feito sobre conceitos ocorre no nível de categorias. Interage-se com um

carro específico, mas um consumidor, ao procurar um carro, busca na categoria de carros e não uma instância específica de carro.

Em LPO, representa-se uma categoria com predicados ou objetos. O predicado Carro(x), indica que x é um carro, ou um objeto Carro pode ser utilizado para representar todos os carros. Nesse tópico encontram-se muitas similaridades com o paradigma de Orientação a Objetos, muito comum na maior parte de linguagens de programação, com recursos como **herança de objetos**.

Outra definição importante a uma representação de conhecimento é a relação **parte de**. Objetos podem ser agrupados como **parte de** uma categoria de objetos. Por exemplo, temos ParteDe(roda, carro), ParteDe(motor, carro) ou ParteDe(freios, carro). Essa relação geralmente ocorre em **objetos compostos**, como o exemplo do Carro.

Na engenharia ontológica, outras definições devem ser levadas em consideração, como a definição de **medições, eventos, processos e intervalos de tempo**.

3.1.6 Redes semânticas

As **categorias** são os principais blocos de construção de bases de conhecimento em grande escala. As **redes semânticas** são um sistema que permite a visualização gráfica de uma base de conhecimento e algoritmos para a dedução de propriedades de um objeto.

Existem diversas variantes de redes semânticas, mas todas podem representar objetos individuais, categorias de objetos e relações entre os objetos. A notação gráfica típica representa os objetos e categorias nomeados em retângulos ou elipses e os conecta com arcos rotulados pelas relações entre os objetos ou categorias.

A figura ilustra uma rede semântica em que o “Carro A” é um objeto específico no mundo real, e é uma instância da categoria “Carro”, que é um tipo de “Veículo”. Por exemplo, “Motocicleta” e “Caminhão” também poderiam ser tipos de “Veículo”. “Carro A” tem “Marca”, “Modelo” e “Proprietário”, cujos respectivos valores são “BMW”, “i8” e “Pedro”. Além disso, na rede semântica é representado um tipo de “Roda” (parte de um “Carro”), a “Roda de liga leve”.

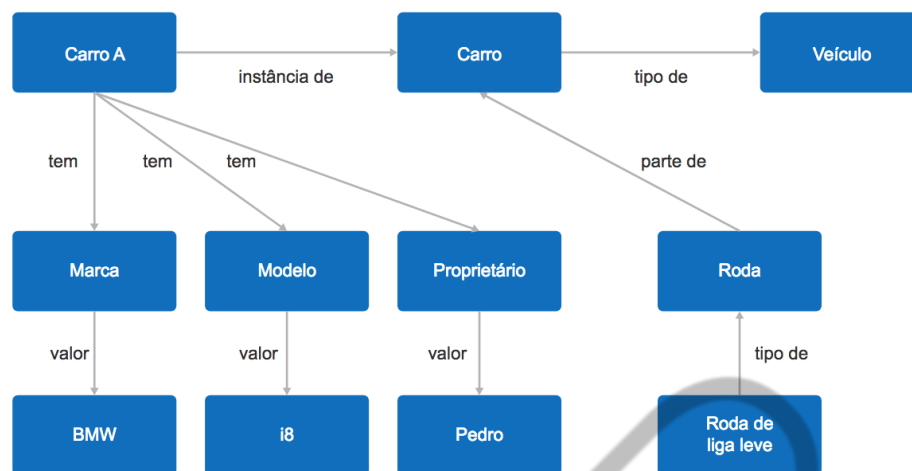


Figura 3.2. Exemplo de rede semântica
Fonte: Google (2017)

Como exercício, você poderia representar o conhecimento da rede semântica da figura em LPO.

REFERÊNCIAS

RUSSEL, Stuart; NORVIG, Peter. **Inteligência Artificial**. Rio de Janeiro: Elsevier, 2013.

PARDO, Thiago A. S.; REZENDE, Solange O. **Representação de Conhecimento**. 2015. 2018. Disponível em: <<http://wiki.icmc.usp.br/images/c/c3/Aula10-230t.pdf>>. Acesso em: 16 jan. 2018.

MOURA, Flávio L. C. **A Lógica de Primeira Ordem**. 2012. Disponível em: <<http://www.cic.unb.br/~flavio/disciplinas/graduacao/lc1/GramaticaLPredicados.pdf>>. Acesso em: 15 jan. 2018.

WIKIDATA. **Wikidata:Statistics**. [s.d.]. Disponível em: <<https://www.wikidata.org/wiki/Wikidata:Statistics>>. Acesso em: 25 jun. 2020.

GLOSSÁRIO

Chatbot	Programa que busca simular o comportamento humano em uma interação com um humano com o uso da linguagem natural.
Conectivos lógicos	Ou conectivos binários, que juntam duas sentenças, considerando os operandos de uma função.
Ontologia	Modelo de dados que representa um conjunto de conceitos em um domínio e os relacionamentos entre os conjuntos.
Orientação a Objetos	Modelo de análise, projeto e programação de software baseado na composição e interação entre as unidades do software, que representam objetos do mundo real.
Semântica	Estudo dos significados de objetos, palavras, etc., considerando o contexto em que ocorrem.

INTELIGÊNCIA ARTIFICIAL
E COMPUTACIONAL

INTRODUÇÃO AOS SISTEMAS ESPECIALISTAS

ERICK GALANI MAZIERO



4

LISTA DE FIGURAS

Figura 4.1. Arquitetura de um Sistema Especialista.....	7
Figura 4.2. Linhas de transmissão entre usinas e estações de distribuição de energia do estado do MS.	13
Figura 4.3. Exemplo de interface do REC_ESP com usuário do sistema especialista..	14



LISTA DE TABELAS

Tabela 4.1. Base de conhecimento de compatibilidade entre baterias e dispositivos eletrônicos.....	10
--	----

EVANGELISTA

SUMÁRIO

4 INTRODUÇÃO AOS SISTEMAS ESPECIALISTAS	5
4.1 Sistemas baseados em conhecimento	5
4.2 Etapas do desenvolvimento de um Sistema Especialista (SE)	7
4.3 O motor de inferência	9
4.4 Formas de interação com o SE	11
4.5 Tomada de decisão	11
4.6 Exemplos de Sistemas Especialistas	12
4.6.1 REC_ESP	12
4.7 Vantagens e limitações de um SE	14
REFERÊNCIAS	16
GLOSSÁRIO	17

4 INTRODUÇÃO AOS SISTEMAS ESPECIALISTAS

4.1 Sistemas baseados em conhecimento

Em 1969, na universidade de Stanford nasceu o DENDRAL, um sistema baseado em conhecimento que tinha a finalidade de inferir a estrutura molecular a partir das informações fornecidas por um espectrômetro de massa. Como o sistema tinha de explorar uma quantidade exponencial de possíveis estruturas para as medições de massa, seus autores consultaram especialistas de química analítica para, segundo algumas heurísticas, escolher as estruturas mais prováveis para as medições de massa.

O DENDRAL é digno de menção por ser o primeiro sistema de conhecimento intensivo bem-sucedido, abrindo caminho para sistemas baseados em conhecimento. Outros pesquisadores de Stanford investiram em um projeto chamado *Heuristic Programming Project* (HPP), que investigou até que ponto os sistemas baseados em conhecimento seriam úteis em outras áreas de conhecimento.

O principal esforço foi em um sistema de diagnóstico médico, o MYCIN, que detectava infecções sanguíneas. O MYCIN continha cerca de 450 regras obtidas com especialistas médicos e se saiu tão bem quanto médicos especialistas e até melhor que médico em início de carreira.

Em 1982 surgiu o primeiro sistema especialista de uso comercial bem-sucedido. O R1 ajudava a configurar novos pedidos de computadores feitos à DEC (Digital Equipment Corporation). A DEC, em 1988 já tinha entregue mais de 40 sistemas especialistas.

Esses sistemas evidenciam a importância da representação do conhecimento humano para uso de sistemas inteligentes. Isso ajudou a popularizar linguagens de programação lógica, como o Prolog e o PLANNER.

Um **sistema** é, portanto, dito **especialista** quando seu comportamento, ou decisões, imita o comportamento ou decisões de um especialista da área de atuação do sistema. Assim, um sistema especialista de diagnóstico em uma área médica baseia-se em conhecimento obtido de médicos da área. Esse sistema diagnosticará de acordo com o conhecimento que obteve dos médicos, imitando suas decisões.

Um sistema especialista é tipicamente escolhido como solução para problemas que um especialista humano poderia resolver. Geralmente, quanto mais complexo o problema a ser solucionado, mais técnicas de IA comporão o sistema especialista.

A diferença entre um **sistema algorítmico** e um sistema especialista é que o sistema algorítmico armazena limitado conhecimento em formato de código (por exemplo, pelo uso de comandos condicionais). Os sistemas especialistas, no entanto, utilizam uma base de conhecimento extraída de especialistas do domínio do problema a ser resolvido. Caso seja necessário aumentar o conhecimento do domínio para melhorar o desempenho do sistema, o sistema algorítmico necessitará de mudança em seu código. Já o sistema especialista precisará apenas aumentar sua base de conhecimento, raramente necessitando alterar seu motor de inferência na base de conhecimento.

Um SE pode ser utilizado em diversos cenários com o intuito de agilizar e uniformizar soluções em organizações. Podemos citar diversos cenários:

- Gerenciamento de decisões;
- Diagnóstico de problemas;
- Monitoramento e controle de processos;
- Seleção e classificação de opções.

Pode-se citar, também, diversas áreas funcionais em que um SE pode ser utilizado:

- Gestão financeira;
- Estratégias de marketing;
- Gerenciamento de materiais;
- Planejamento de recurso capital;
- Dentre outras.

4.2 Etapas do desenvolvimento de um Sistema Especialista (SE)

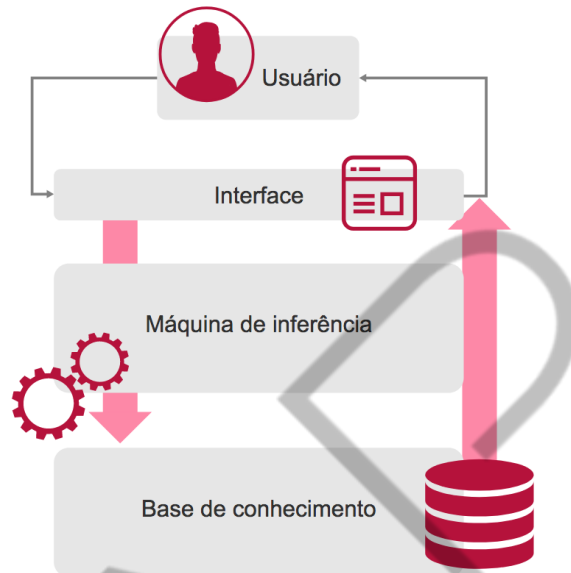


Figura 4.1. Arquitetura de um Sistema Especialista
Fonte: Google (2017)

A figura apresenta uma possível arquitetura para um SE. O usuário interage com o sistema através de uma interface, por meio da qual se possa obter respostas para problemas do domínio escolhido. A interface repassa as entradas fornecidas pelo usuário à máquina (ou motor) de inferência, que, por sua vez, busca na base de conhecimento a resposta mais provável para o questionamento do usuário.

A seguir, considere os passos para a construção de um SE:

Definição do problema: deve-se definir o “problema” a ser resolvido pelo SE. O escopo do problema deve ser formalizado, pois dificilmente um sistema especialista conseguirá tratar todo o domínio de um problema.

Imagine um sistema especialista da área médica, uma base de conhecimento que possibilite identificar todas as doenças conhecidas através de seus sintomas pode ser uma tarefa árdua, pois necessitaria de diversos especialistas para obtenção de conhecimento. Veja que há diversas especialidades médicas.

Aquisição do conhecimento: aqui, dada delimitação do problema (por exemplo, identificar doenças cardíacas, dados seus sintomas), um ou mais

especialistas do domínio deverão ser exaustivamente consultados para a montagem da base de conhecimento.

É importante que o construtor do SE tenha também algum conhecimento do problema, o que pode ser obtido em materiais de caráter introdutório ao assunto escolhido. Ter um conhecimento do domínio, possibilitará uma conversa mais informada com os peritos da área escolhida.

Representação do conhecimento: após a obtenção do conhecimento junto a especialistas, esse conhecimento deve ser adequado a uma representação do conhecimento. No capítulo anterior foram apresentados diversos formalismos para a representação do conhecimento, como a LPO (Lógica de Primeira Ordem) e Redes Semânticas. Esses formalismos possibilitam a criação de diversos motores de inferência, para tomada de decisão com base no conhecimento representado.

Um **engenheiro do conhecimento** é um profissional especialista em representar o conhecimento obtido dos especialistas em algum dos formalismos citados.

Codificação: nesse passo, basicamente, deve-se codificar o sistema que, utilizando a base de conhecimento em uma dada representação, atenderá a demandas do usuário do SE. Como outro sistema computacional, uma arquitetura modular e escalável deve ser adotada para que o sistema possa crescer sem modificações muito estruturais no código.

Avaliação do SE: após a construção do sistema, os especialistas do domínio, junto com os construtores do SE, devem verificar a consistência das respostas obtidas automaticamente. Geralmente, separa-se um conjunto do conhecimento obtido pelos especialistas para essa etapa. Esse conjunto de teste não deve estar presente na base de conhecimento do SE. Diversas medidas de avaliação podem ser utilizadas, como precisão (P), cobertura (C) e a medida-F (uma média harmônica entre P e C).

Detectando-se uma baixa performance do SE, a causa deve ser identificada. Pode ser devido ao conhecimento, que é insuficiente ou está armazenado de maneira inconsistente. Ou pode ser devido à máquina de inferência (codificação) que não é poderosa o suficiente para buscar o conhecimento necessário para a resposta esperada.

Manutenção: por fim, o sistema deve ser bem documentado e ter uma interface de fácil utilização para os usuários do sistema. Se a base de conhecimento possibilitar a dedução de novos conhecimentos, periodicamente, o sistema deve ser avaliado para verificar se o conhecimento aderido está correto.

4.3 O motor de inferência

É considerado o cérebro do SE, pois atua sobre a base de conhecimento (raciocina), em busca de uma resposta a uma questão do problema. Como visto nos capítulos anteriores, o motor de inferência pode utilizar uma abordagem de busca até encontrar uma resposta para o usuário, quando o conhecimento está armazenado em uma árvore de decisão, por exemplo.

O raciocínio pode ser feito de maneira progressiva ou regressiva. No raciocínio progressivo, à medida que o usuário vai fornecendo evidências do problema, o sistema vai desencadeando o processo de busca na base de conhecimento até encontrar uma resposta para o usuário.

Já no raciocínio regressivo, o SE escolhe, segundo alguma heurística, uma resposta/hipótese (que inclusive pode ser fornecida pelo usuário) ao problema e, numa busca reversa, verifica se a resposta é plausível, segundo o conhecimento armazenado na base de conhecimento.

Caso a representação do conhecimento tenha sido em Lógica de Primeira Ordem, o uso de uma linguagem como o Prolog traz recursos de inferência como **backtracking** (ou retrocesso) em cláusulas de LPO.

Para exemplificar o uso do **backtracking**, considere uma simples base de conhecimento em LPO que representa a compatibilidade entre modelos de baterias e determinados equipamentos eletrônicos:

Tabela 4.1. Base de conhecimento de compatibilidade entre baterias e dispositivos eletrônicos

1	Compatível(Bateria_58PX, Filmadora1)
2	Compatível(Bateria_58PX, Celular1)
3	Compatível(Bateria_98UX, Celular1)
4	Compatível(Bateria_765, Notebook1)
5	Compatível(Bateria_58PX, Celular2)
6	Compatível(Bateria_123, Celular3)

Fonte: Elaborado pelo autor (2017)

Agora imagine que se deseja saber se há algum dispositivo eletrônico compatível com dois tipos de bateria: “Bateria_58PX” e “Bateria_98UX”. Para isso realiza-se uma pergunta, no seguinte formato:

Compatível(Bateria_58PX, Y), Compatível(Bateria_98UX, Y)

A primeira cláusula da pergunta é instanciada com o conhecimento da linha 1 e $Y = \text{Filmadora1}$. Agora, deve-se procurar pela segunda cláusula com o valor de Y instanciado, ou seja, *Compatível (Bateria_98UX, Filmadora1)*. Essa busca não retorna valor. Nesse ponto ocorre o **backtracking**, em que se retorna ao momento em que Y foi instanciado como “Filmadora1” e desfaz essa operação.

Agora, a primeira cláusula é instanciada com o conhecimento da linha 2 e $Y = \text{Celular1}$. Agora, procura-se por *Compatível (Bateria_98UX, Celular1)* e obtém-se sucesso com o conhecimento da linha3. Assim, conclui-se que os dois modelos de bateria são compatíveis com o dispositivo “Celular1”.

Embora pareça trivial, o *backtracking* resolve problemas de busca em LPO sem que o programador tenha que codificar a busca. No entanto, em bases de conhecimento muito grandes, esse procedimento pode levar a uma explosão combinatória, pois é um procedimento inerentemente combinatório.

4.4 Formas de interação com o SE

Um sistema especialista pode ter diversas formas de interagir com o usuário, como uma interface gráfica que contém diversos elementos de escolha (formulários) para o usuário dar evidências ou sintomas, por exemplo, e obter uma resposta.

A forma mais intuitiva é o uso da linguagem natural, por meio de um diálogo. Nesse caso, o SE deve ter um bom módulo de PLN (Processamento da Linguagem Natural) para interpretar o texto do usuário e mapear o que é dito para o conhecimento armazenado na base de conhecimento.

Considerando o exemplo apresentado anteriormente (*backtracking*), um usuário poderia utilizar a seguinte entrada em linguagem natural:

“Existe algum dispositivo que seja compatível com as baterias ‘Bateria_58PX’ e ‘Bateria_98UX’?”

Cabe ao módulo de interpretação textual converter a entrada nas cláusulas *Compatível(Bateria_58PX, Y)*, *Compatível(Bateria_98UX, Y)* e retornar o valor de Y para o usuário.

4.5 Tomada de decisão

Um sistema especialista deve imitar um especialista da área em que vai atuar. Portanto, dada uma entrada ao sistema, seja por texto, voz ou imagens, ele deve **tomar a decisão** que melhor representa a decisão do especialista. Em um cenário real, um especialista ficará com certa incerteza com relação a qual decisão tomar e optará pela mais provável, isto é, optará pela decisão com maior **probabilidade** de acerto.

Levando em conta as possíveis decisões que um sistema pode tomar e suas probabilidades, tem-se um formalismo chamado de **redes de decisão** (ou diagramas de influência) muito útil à construção de sistemas especialistas.

As redes de decisão são um mecanismo geral para tomadas de decisão racional. Essas redes basicamente representam o estado atual do agente, suas ações

possíveis a partir do estado atual e o estado que resultará da escolha de uma ação e o ganho obtido dessa escolha.

4.6 Exemplos de Sistemas Especialistas

Além dos exemplos citados no início deste capítulo (DENDRAL, MYCIN e R1), veja mais um exemplo de SE, descrito a seguir.

4.6.1 REC_ESP

É fácil encontrar um problema que pode ser resolvido com um SE. Por exemplo, quando alguém liga para um *helpdesk*, para resolver um problema encontrado numa ferramenta ou serviço, como um serviço de conexão com a internet, o atendente, geralmente, segue um roteiro de perguntas. Após grande número de atendimentos, os atendentes poderiam ser consultados com o intuito de extrair esse “roteiro” do atendimento e gerar um SE.

Obviamente, no exemplo anterior, podem surgir problemas de clientes que não serão resolvidos diretamente pelo SE, mas boa parte dos problemas será! No caso de um problema que o SE não encontre solução, um atendente pode ser acionado e, nesse caso, o conhecimento do SE deve ser aumentado com novas regras para, numa próxima ligação ter maior poder de resolução.

Bronaut *et al.* (2007) apresentam o REC_ESP, um SE desenvolvido para a retomada do sistema de elétricos de potência do estado do Mato Grosso do Sul. Feito em conjunto entre a ENERSUL e a UFMS, o REC_ESP tem seu conhecimento extraído de um manual de operações da ENERSUL e do conhecimento dos operadores do sistema elétrico.

A base de conhecimento do REC_ESP consiste de um conjunto de regras que levam a sugestões sistêmicas, a partir de verificações de variáveis em pontos estratégicos do sistema elétrico (como tensão, patamar de carga, etc.). Um exemplo de regra, que apresenta uma sugestão, a partir da tensão é descrito a seguir:

Se Tensão na barra A maior que 145 kV

Então, tem-se Tensão elevada, o Disjuntor 01 não pode ser fechado

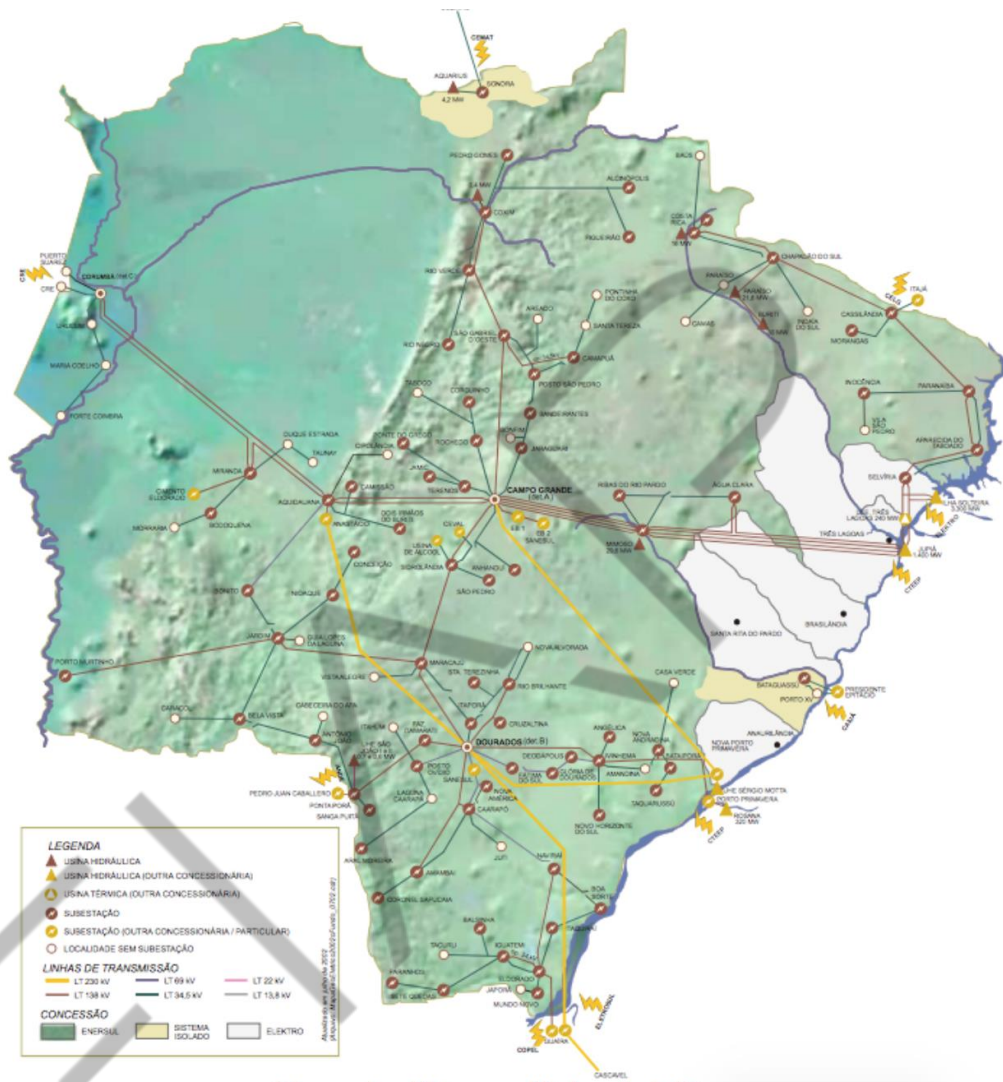


Figura 4.2. Linhas de transmissão entre usinas e estações de distribuição de energia de MS
Fonte: Bronaut et al. (2007)

Como o objetivo do SE é restabelecer as linhas de transmissão de energia mais prioritárias para o estado do MS, ele tem um sistema de busca (inferência) que analisa os caminhos entre as estações centrais e estações de borda, escolhendo as linhas mais prioritárias.

A interface do REC_ESP com os usuários é feita pela apresentação do diagrama do sistema elétrico, e os passos para a recomposição do sistema são feitos depois de priorizadas as linhas de transmissão. A figura apresenta um exemplo de

interface entre o sistema e os usuários do SE, em que se tem o diagrama da rede elétrica ao fundo e orientação para o restabelecimento da energia.



Figura 4.3. Exemplo de interface do REC_ESP com usuário do sistema especialista
Fonte: Figura extraída de Bronaut *et al.* (2007)

É interessante que o REC_ESP não serve apenas para guiar os operadores no restabelecimento do sistema de eletricidade, mas também é utilizado para treinar novos operadores antes de ocorrer problemas na rede elétrica.

4.7 Vantagens e limitações de um SE

A grande vantagem de um SE é que ele armazena conhecimento de um ou mais especialistas e o torna disponível para uma grande quantidade de usuários. Um motor de inferência atuando nesse conhecimento agiliza o procedimento de um usuário que não demandará a assimilação das regras armazenadas, levando a uma rápida atuação no problema.

As pessoas não são eternas e, em determinados segmentos profissionais, elas mudam constantemente de empresa, levando consigo seu conhecimento profissional e do negócio. As bases de conhecimento de SE permitem que boa parte do conhecimento dos profissionais seja “perpetuado” e fique disponível para futuros profissionais.

O SE, além de seu uso em situações reais, pode ser utilizado para treinamento de nossos operadores da área de conhecimento do SE.

Além das vantagens de um SE, vale a pena considerar cenários em que sua atuação nem sempre é possível. Um SE é útil em problemas de domínio bem delimitado com disponibilidade de especialistas, dos quais se possa extrair e construir uma base de conhecimento.

Em alguns problemas, no entanto, a extração do conhecimento de especialistas não é uma tarefa trivial, além do que se dispõe de uma grande quantidade de informações já tratadas pelos especialistas. Nesses casos, outras técnicas da IA podem ser aplicadas com maior agilidade e resultados satisfatórios.

Essas técnicas consistem na aplicação de algoritmos que identificam padrões nas informações e geram modelos que resolvem outros problemas baseados nos dados de entrada do sistema, “imitando” implicitamente o comportamento dos especialistas que tratam as informações. Esses métodos serão apresentados no próximo capítulo sob o tema de Aprendizagem de Máquina.

REFERÊNCIAS

BRONAUT , A.; PINTO, J. O. P.; COLLAZOS, L. K.; LEITE, L. C.; GALOTTO JR., L.; GONÇALVES, A. M. M.; SANTINI JR., L. C.; OLIVEIRA, L. L.; RENNÓ, M. A. G.; SANTOS, N. M. S. **Sistema Especialista para Auxílio na Tomada de Decisões para Recomposição em Tempo Real**. Anais do IV Congresso de Inovação Tecnológica em Energia Elétrica. Minas Gerais. 2007.

RUSSEL, Stuart; NORVIG, Peter. **Inteligência Artificial**. Rio de Janeiro: Elsevier, 2013.

WATERMAN, D. A. **A Guide to Expert Systems**. Addison-Wesley, 1986.

GLOSSÁRIO

Espectrômetro de massa	Ferramenta de análise física que identifica moléculas por medição da massa e da caracterização da estrutura química.
Programação lógica	Técnica de desenvolvimento de algoritmos baseada em lógica matemática.
Árvore de decisão	É uma representação gráfica de uma tabela de decisão. Permite avaliar problemas por meio de decisões sequenciais, até chegar a uma solução.
Explosão combinatória	Condição inerente a alguns problemas matemáticos, nos quais um pequeno aumento do tamanho do problema pode levar a um imenso aumento do tempo ou espaço necessário para obtenção da resposta.
Helpdesk	Termo em Inglês que significa literalmente “balcão de ajuda”. É um serviço de atendimento ao cliente, que busca solução de problemas e dúvidas.

INTELIGÊNCIA ARTIFICIAL
E COMPUTACIONAL

APRENDIZAGEM *DE MÁQUINA*

ERICK GALANI MAZIERO



5

LISTA DE FIGURAS

Figura 5.1. Principais etapas para um aprendizado automático	9
Figura 5.2. Taxonomia das tarefas de aprendizado de máquina.....	13
Figura 5.3. Representação dos principais elementos de uma árvore: raiz, nó interno e nó folha.....	15
Figura 5.4. Exemplo genérico de uma árvore de decisão que utiliza atributos A1 a A3 e pode chegar a 3 possíveis decisões	15
Figura 5.5. Árvore de decisão que atua sobre atributos com valores nominais	16
Figura 5.6. Árvore de decisão gerada sobre os dados da Tabela “Características observadas para a decisão de sair ou não para jogar tênis”	17
Figura 5.7. Conversão de valores nominais do atributo "tempo" para valores numéricos com a técnica "one-hot-encoding"	18
Figura 5.8. Hiperplano de separação entre duas classes em um espaço multidimensional.....	19
Figura 5.9. Formas de combinar classificadores binários em problemas multiclasse	20
Figura 5.10. Exemplo de regressão linear, ilustrada geometricamente	22
Figura 5.11. Aplicação do método K-means, com K = 3	23
Figura 5.12. Formalização matemática de um neurônio	25

LISTA DE TABELAS

Tabela 5.1. Exemplos de tarefas tratadas por aprendizado automático.....	7
Tabela 5.2. Características observadas para a decisão de sair ou não para jogar tênis.....	8
Tabela 5.3. Exemplo de atributos de entrada para um aprendizado automático.....	11
Tabela 5.4. Exemplo de Matriz de Confusão.....	30

EVANGELISTA

LISTA DE EQUAÇÕES

Equação 5.1. Cálculo da Precisão	28
Equação 5.2. Cálculo da Cobertura	28
Equação 5.3. Cálculo da Medida-F	29

EVANGELISTA

SUMÁRIO

5 APRENDIZAGEM DE MÁQUINA	6
5.1 A partir de exemplos, identificando padrões.....	7
5.2 Principais etapas do aprendizado automático	9
5.3 Taxonomia do aprendizado automático.....	13
5.4 Aprendizagem supervisionada	14
5.4.1 Árvores de decisão, um paradigma simbólico	14
5.4.2 Support Vector Machines (SVM), um paradigma matemático	17
5.4.3 Regressão linear, problema de classes contínuas	20
5.5 Aprendizagem não supervisionada	22
5.5.1 K-means, um método de agrupamento	23
5.6 Aprendizagem semissupervisionada.....	24
5.7 Redes neurais: matematizando o biológico.....	24
5.8 Medidas de avaliação da aprendizagem	28
REFERÊNCIAS	31
GLOSSÁRIO	32

5 APRENDIZAGEM DE MÁQUINA

Os capítulos anteriores apresentaram métodos de busca, formalismos para representação de conhecimento, raciocínio e sistemas especialistas (SE), que atuam sobre bases de conhecimento.

Mas nem sempre é possível implementar um SE para simular o comportamento humano, gerando um agente inteligente. Imagine um carro autônomo que deve navegar em uma cidade cheia de outros carros, pessoas e obstáculos. Não é possível prever todas as prováveis localizações de todos os objetos no ambiente. Na realidade, o carro autônomo deve aprender a se movimentar considerando qualquer configuração de objetos. Para isso, é necessário “ensinar” o sistema do carro autônomo a se movimentar dando exemplos de como se comportar quando algum objeto atravessa a rua, de como virar o carro, frear, etc. A montadora Tesla, por exemplo, em 2019, apresentou detalhes técnicos da tecnologia de seus carros autônomos com características semelhantes.

Em muitos cenários, há muita informação disponível para aplicar o que aprenderemos neste capítulo: **Aprendizado de máquina (AM)**. No exemplo do carro autônomo, um humano pode dirigir um carro numa cidade, tomando as decisões de frear, acelerar, virar, etc. e armazenar as configurações do ambiente a cada decisão. Essas informações serão exemplos para que o sistema autônomo aprenda a navegar em uma cidade dadas as configurações do ambiente, capturadas por sensores.

A respeito do AM, tem-se a definição a seguir:

Diz-se que um programa de computador aprende pela **experiência E**, com respeito a algum tipo de **tarefa T** e **performance P**, se sua performance P na tarefa T melhora com a experiência E. (MITCHELL, 1997)

Mitchell (1997) definiu o aprendizado de máquina em função de uma tarefa T que é ensinada a um programa de computador (modelo de aprendizado) por meio de exemplos, ou experiências, E. Se a performance P aumenta à medida que mais exemplos E são apresentados, o programa está aprendendo a tratar a tarefa T.

Para ficar mais claro, considere as instanciações de E, T e P para diversos problemas, na tabela.

Tabela 5.1. Exemplos de tarefas tratadas por aprendizado automático

Tarefa T	Previsão meteorológica
Experiência E	Valores de diversos sensores meteorológicos e estado meteorológico (por exemplo, chuva, sol, neblina, etc.)
Performance P	Quantidade de previsões corretamente identificadas
Tarefa T	Speech Recognition (reconhecimento de fala)
Experiência E	Sons gravados e respectivos textos transcritos
Performance P	Quantidade de transcrições corretas

Fonte: Elaborado pelo autor (2017)

Diversos algoritmos, ao longo do tempo, têm sido propostos para esse fim. Desde algoritmos puramente matemáticos até probabilísticos e simbólicos. Inclusive a aprendizagem de máquina pode-se dar por meio de exemplos com a solução indicada ou por exemplos cuja solução não tenha sido indicada por humanos.

5.1 A partir de exemplos, identificando padrões

Em geral, quando se fala em aprendizado de máquina e seus diversos algoritmos de aprendizado, o que se objetiva é a identificação de padrões. Na realidade, poderíamos chamar aprendizado de máquina por aprendizado de padrões.

Para exemplificar, considere um problema em que um jogador de tênis deseja saber se deve ou não sair para jogar, dadas algumas variáveis meteorológicas, como aparência do tempo, temperatura, umidade e vento.

A tabela “Características observadas para a decisão de sair ou não para jogar tênis” apresenta diversos exemplos das características observadas e a melhor decisão: sair ou não para jogar. Note que, para as características temperatura e umidade, são apresentados valores numéricos (temperatura em graus celsius e umidade em percentual) e valores nominais (temperatura entre quente, frio ou ameno e umidade entre normal e alta). Nas próximas seções deste capítulo serão

apresentados alguns algoritmos que podem tratar valores nominais ou apenas trabalham com valores numéricos.

Tabela 5.2. Características observadas para a decisão de sair ou não para jogar tênis

Aparência do tempo	Temperatura Numérica	Temperatura Nominal	Umidade Percentual	Umidade Nominal	Vento	Jogar?
Nublado	28	Quente	86	Alta	Não	Sim
Nublado	18	Frio	65	Normal	Sim	Sim
Nublado	22	Ameno	90	Alta	Sim	Sim
Nublado	27	Quente	75	Normal	Não	Sim
Chuvoso	21	Ameno	96	Alta	Não	Sim
Chuvoso	20	Frio	80	Normal	Não	Sim
Chuvoso	18	Frio	70	Normal	Sim	Não
Chuvoso	24	Ameno	80	Normal	Não	Sim
Chuvoso	17	Ameno	91	Alta	Sim	Não
Ensolarado	29	Quente	85	Alta	Não	Não
Ensolarado	27	Quente	90	Alta	Sim	Não
Ensolarado	22	Ameno	95	Alta	Não	Não
Ensolarado	21	Frio	70	Normal	Não	Sim
Ensolarado	24	Ameno	70	Normal	Sim	Sim

Fonte: Elaborado pelo autor (2017)

Um padrão que, olhando atentamente para a tabela, poderia ser observado é que não se deve sair para jogar quando a aparência do tempo é *ensolarado* e a temperatura nominal é *quente*. Outro padrão é que todas as vezes que a aparência do tempo for *nublado*, pode-se sair para jogar.

Os diversos algoritmos de aprendizado de máquina que serão apresentados utilizarão técnicas probabilísticas, matemáticas ou simbólicas (dentre outras) para identificar tais padrões e gerarão um modelo (segundo a técnica utilizada) para ser utilizado em dados cuja resposta esperada não é conhecida. Esse modelo, portanto, imitará o comportamento dos humanos que definiram as respostas esperadas dos exemplos utilizados no treinamento do aprendizado automático e terão, assim, um comportamento “inteligente”.

5.2 Principais etapas do aprendizado automático

No desenvolvimento de uma solução com o uso de aprendizado automático, há etapas que, na maioria das vezes, serão percorridas. A figura “Principais etapas para um aprendizado automático” apresenta essas etapas, iniciando pela **seleção dos dados**, passando pela etapa de **rotulação**, **balanceamento dos dados**, **extração de features**, **seleção de features**, **treinamento do modelo** e **avaliação** do modelo gerado. Geralmente, **refinamentos** podem ser feitos em alguma das etapas anteriores para alcançar melhores resultados na avaliação.

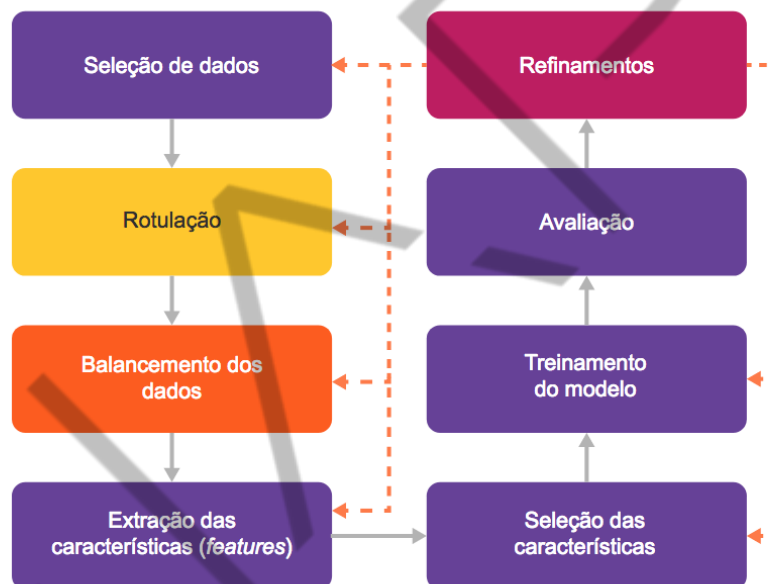


Figura 5.1. Principais etapas para um aprendizado automático
Fonte: Elaborado pelo autor (2017)

A **seleção dos dados** é uma tarefa primordial, pois, quanto mais representativos do problema forem os dados, mais chances de se obter um modelo de aprendizado que reagirá bem a novos dados do problema. Por exemplo, quando se deseja introduzir um agente inteligente proveniente do aprendizado automático em uma tarefa antes realizada manualmente, deve-se obter o máximo possível dos dados gerados manualmente. Desta forma, tais dados possibilitarão um comportamento automático similar ao humano.

Imagine a tarefa de criar um atendente de *helpdesk* com aprendizado de máquina, os dados que devem ser obtidos são os que foram gerados tanto pelo

atendente quanto pelo usuário do *helpdesk*. Se o objetivo é gerar um modelo de predição meteorológica, dados dos mais variados sensores (temperatura, pressão, velocidade do vento, etc.) devem ser reunidos do maior tempo de medição possível.

Feita a obtenção dos dados, deve-se proceder à **rotulação** desses dados. Essa rotulação consiste, basicamente, em indicar, para cada exemplo dos dados obtidos, qual a resposta esperada. O termo muito utilizado na área para a resposta esperada é **classe** (em tarefas de classificação, como será visto a seguir, na taxonomia). Assim, para cada exemplo, deve-se dizer qual a classe esperada. No exemplo dos dados meteorológicos, para cada conjunto de medições, qual é a condição do tempo (ensolarado, nublado, chuvoso, etc.), por exemplo.

Após a rotulação dos dados, o **balanceamento** dos dados pode se mostrar necessário. Considere o caso em que os dados obtidos foram rotulados majoritariamente com apenas duas de três classes do problema. Por exemplo, o problema meteorológico, num conjunto de 10 mil exemplos, 4,9 mil são de tempo ensolarado, 5 mil de nublado e apenas 0,1 mil de chuvoso. Tem-se poucos exemplos de dados para predizer tempo chuvoso. Deve-se, portanto, balancear os dados para evitar que o modelo aprenda e prediga mais as duas classes mais frequentes, errando muito a classe menos frequente.

O balanceamento dos dados pode ser feito por duas técnicas principais: **oversampling** e **undersampling**. Na primeira técnica, deve-se obter mais dados da(s) classe(s) minoritária(s), seja buscando mais dados, seja gerando dados sintéticos. Na segunda técnica, exemplos das classes majoritárias devem ser dispensados, para ter proporção similar à(s) classes(s) minoritária(s). O **undersampling** só deve ser aplicado caso o conjunto de dados tiver quantidade suficiente para dispensar dados sem perder performance no aprendizado.

Em algoritmos tradicionais de aprendizado de máquina, a etapa de **extração de atributos** (*features*) é essencial, pois são os atributos extraídos dos dados que permitem aos algoritmos de aprendizado gerar modelos para predizer as classes desejadas.

No exemplo meteorológico, cada medida obtida por cada sensor pode ser considerada um atributo do problema. Considerando outro exemplo, o problema de dar crédito ou não a clientes de um banco, tem-se na tabela “Exemplo de atributos de

entrada para um aprendizado automático” atributos e seus possíveis valores que podem ser extraídos das fichas cadastrais dos clientes.

Tabela 5.3. Exemplo de atributos de entrada para um aprendizado automático

Atributo	Valor
Gênero	Masculino ou feminino
Idade	Número inteiro
Nível educacional	Primeiro grau incomp., primeiro grau comp., segundo grau incomp., segundo grau comp., superior incomp., superior comp., pós-graduação
Estado Civil	Solteiro, casado, divorciado, viúvo
Profissão	Nome da profissão
Trabalha há quanto tempo	Quantidade de meses
Consta em órgãos de protesto	Sim ou não
Salário	Valor decimal
Altura	Valor decimal

Fonte: Elaborado pelo autor (2017)

Nem sempre, no entanto, a extração de atributos é tão imediata, como mapear cada medição de um sensor ou campo de cadastro para um atributo. Considere o problema de detectar e-mails *spam*. Nesse caso, os atributos consistirão em *pistas* espalhadas pelo título e corpo do e-mail. Por exemplo, a ocorrência de determinadas palavras que indicam um *spam* (*blacklist*) ou palavras que indicam um e-mail confiável (*whitelist*) podem ser atributos dessa tarefa.

Um outro passo, às vezes opcional, é a **seleção de atributos**. Nem sempre um atributo que escolhemos para resolver um problema é realmente importante para a solução. Isso quando tal atributo não atrapalha a identificação da solução pelo algoritmo de aprendizado. Alguns algoritmos, como a árvore de decisão, que veremos mais adiante, tem a robustez de selecionar os atributos que melhor se adequam à solução do problema. Já outros algoritmos podem não ser robustos assim. A seleção de atributos é feita com base em alguma métrica, como correlação, ganho de informação ou entropia.

Finalmente, chega a etapa de **treinamento do modelo**. Nessa etapa, de acordo com os atributos e seus tipos, quantidade de dados, tipo de classes (discretas ou contínuas), dentre outras características, deve-se escolher o melhor algoritmo de aprendizado automático. Na próxima seção será apresentada uma taxonomia das principais técnicas de aprendizado.

O treinamento do modelo consiste na aplicação do algoritmo de treinamento nos dados que foram tratados nas etapas anteriores. Esses algoritmos, basicamente, encontrarão, para cada classe, o padrão que a indica. Posteriormente, com dados não vistos durante a etapa de treinamento, o modelo gerado poderá ser aplicado e as classes “identificadas”.

Nem sempre treinar um modelo é o fim da solução. Uma etapa de **avaliação** se mostra essencial para definir uma *performance* para a solução. A avaliação é feita utilizando-se algumas métricas, como *precisão*, *cobertura*, *medida-F*, *acurácia*. Outro recurso muito útil em alguns casos é a *matriz de confusão*. É importante salientar que muitos algoritmos de aprendizado apresentam, ao final do treinamento, muitas dessas métricas, mas nem sempre baseados em dados específicos para testar a solução.

Antes do treinamento, os dados devem ser separados em conjunto de treinamento (para efetivamente treinar o modelo) e teste. O conjunto de teste, que não foi utilizado durante o treinamento, deve ser utilizado na etapa de avaliação para a geração das métricas.

A avaliação assim como o uso efetivo do modelo de aprendizado no problema ao qual foi proposto devem ser analisados para identificar possíveis melhorias no processo. Por exemplo, ao se identificar que uma classe do problema está sendo confundida com outra, deve-se retornar à etapa de extração de *features* e escolher atributos que permitam distinguir melhor as classes confundidas.

Tendo uma visão geral do processo de aprendizado automático, na próxima seção será apresentada uma taxonomia, ou organização, dos principais paradigmas e técnicas de aprendizado automático.

5.3 Taxonomia do aprendizado automático

As diversas formas de aprendizado automático podem ser categorizadas conforme a taxonomia da figura “Taxonomia das tarefas de aprendizado de máquina”. Os três principais paradigmas de aprendizado são: **supervisionado**, **semisupervisionado** e **não supervisionado**. A grande diferença está nos dados utilizados durante o treinamento do modelo que encapsulará a “inteligência”.

No supervisionado, os dados são rotulados, ou seja, têm a indicação da resposta esperada, dada as características de cada exemplo. No não supervisionado, os dados não têm essa rotulação. E no semisupervisionado, parte dos dados são rotulados e parte não são rotulados.

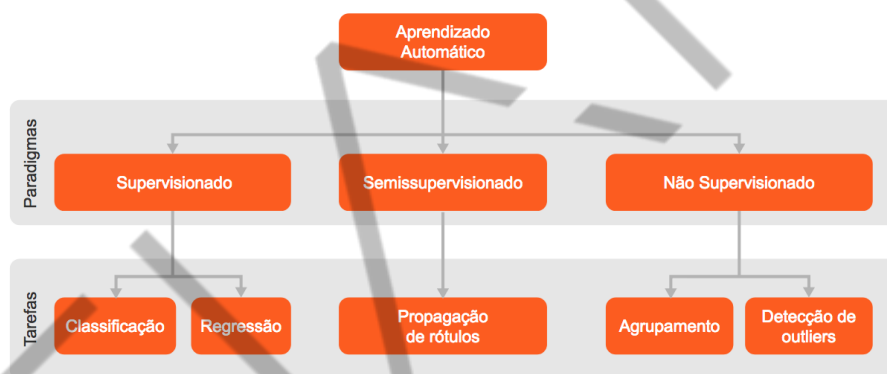


Figura 5.2. Taxonomia das tarefas de aprendizado de máquina
Fonte: Elaborado pelo autor (2017)

No paradigma supervisionado tem-se dois grandes grupos de tarefas: **classificação** e **regressão**. Na primeira tarefa objetiva-se, dadas as características de uma instância, definir uma classe (ou resposta esperada) dentre um número finito de possibilidades (conjunto discreto de valores). Na regressão, no entanto, esse conjunto de classes é infinito, como, por exemplo, um número real entre -1,0 e 1,0.

No paradigma não supervisionado, há também diversas tarefas dentre as quais podemos citar a principal, que é o **agrupamento** e também a **detecção de outliers**. No agrupamento, o objetivo é identificar grupos nos exemplos (dados) segundo alguma métrica de similaridade dadas as características extraídas dos exemplos.

No paradigma semissupervisionado, quaisquer das tarefas mencionadas podem ser abordadas, iniciando tanto pela supervisão quanto pela não supervisão. A semissupervisão é indicada para os casos em que se tem uma parte dos dados rotulados e uma grande quantidade de dados não rotulados.

Para ficar mais claros esses paradigmas e suas principais tarefas, considere a seguir alguns exemplos de algoritmos de aprendizado de máquina, seus fundamentos e aplicabilidades.

5.4 Aprendizagem supervisionada

No aprendizado supervisionado, os dados servem como um *instrutor* do algoritmo de treinamento, indicando para cada exemplo a resposta esperada. Nos dados apresentados na tabela “Características observadas para a decisão de sair ou não para jogar tênis”, a última coluna (Jogar?) é a resposta esperada, que foi definida por humanos, ao observar as características listadas nas outras colunas.

A seguir, serão apresentadas noções dos principais algoritmos de aprendizado supervisionado para a tarefa de classificação: Árvores de Decisão, *Naive Bayes* e SVM. Para a tarefa de regressão linear será apresentado um algoritmo de regressão linear.

5.4.1 Árvores de decisão, um paradigma simbólico

Uma árvore de decisão, a exemplo dos outros algoritmos de aprendizado em uma tarefa de classificação, é uma função que toma como entrada um conjunto de valores e retorna uma decisão. A árvore de decisão toma uma série de passos baseados nos valores de entrada até chegar a uma decisão. No Capítulo 2 foi definida a estrutura de dados em formato de árvore, como na figura, em que se tem basicamente dois tipos de nós: internos (incluindo o nó raiz) e raiz. Instanciando uma estrutura de árvore para tornar em uma árvore de decisão deve-se, a cada nó interno, definir uma decisão a ser tomada em relação aos valores de entrada (atributos ou *features*). Na figura tem-se a ilustração de uma árvore de decisão que toma decisões (D1 a D3) sobre os valores de três atributos (A1 a A3).

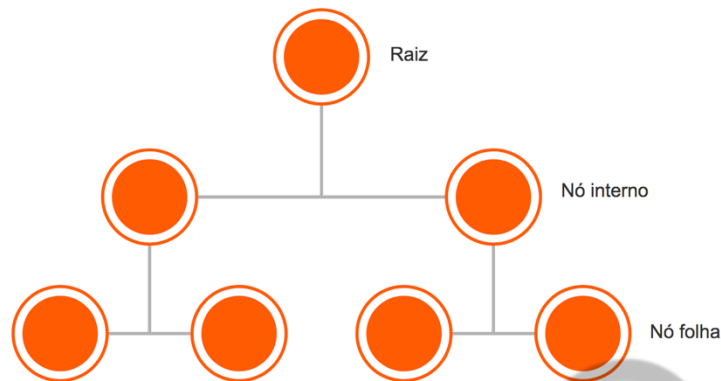


Figura 5.3. Representação dos principais elementos de uma árvore: raiz, nó interno e nó folha
Fonte: Elaborado pelo autor (2017)

Por exemplo, para se tomar a decisão 2 (D2), o atributo A1 deve ter valor menor ou igual a α e A2 deve ser maior que β . Já a decisão 1 (D1) pode ser escolhida para qualquer valor de A1, mas A2 deve ser menor ou igual a β ou A3 deve ser menor ou igual a γ .

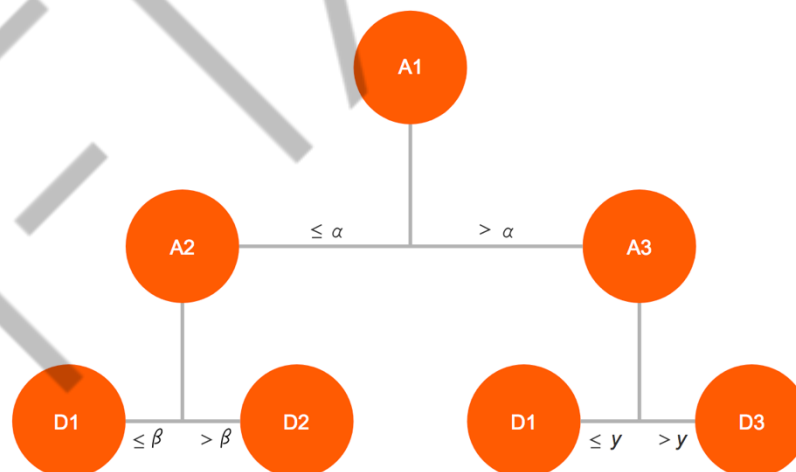


Figura 5.4. Exemplo genérico de uma árvore de decisão que utiliza atributos A1 a A3 e pode chegar a 3 possíveis decisões
Fonte: Elaborado pelo autor (2017)

Vale notar que uma árvore de decisão atua tanto sobre valores de atributos numéricos, como no exemplo anterior, como sobre valores nominais. Na figura, nos nós internos, as decisões foram alteradas para o caso de atributos nominais. Por exemplo, para a decisão 1 (D1), A1 deve ser igual a “alta” e A2 deve ser “p”.

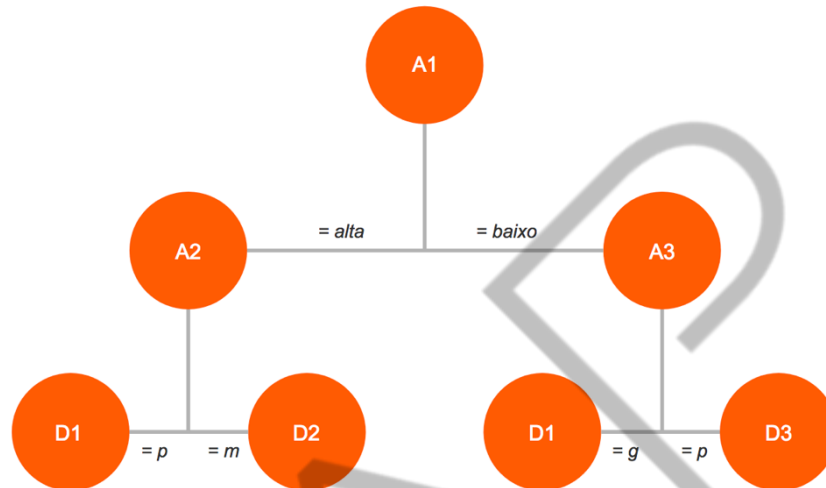


Figura 5.5. Árvore de decisão que atua sobre atributos com valores nominais
Fonte: Elaborado pelo autor (2017)

O processo de criação de uma árvore de decisão é também chamado de indução. Os algoritmos de indução objetivam a criação de árvores que tenham a menor profundidade possível sem perder seu poder e acurácia de classificação. Isso garante, quando o conjunto de atributos é muito grande, que a decisão seja tomada com maior eficiência.

Assim como encontrar a menor árvore de decisão para um problema, uma heurística simples pode encontrar uma solução aproximada: estratégia gulosa. Nessa estratégia, utilizam-se os atributos mais importantes primeiro para dividir o problema em problemas menores, até que todos os exemplos de treinamento da árvore sejam corretamente classificados pela árvore gerada.

Considerando os atributos da tabela (aparência do tempo, temperatura, umidade e vento) para decidir se devo ou não sair para jogar tênis (decisão SIM ou NÃO), podemos gerar a árvore de decisão ilustrada na figura “Árvore de decisão gerada sobre os dados da”. Há outras possíveis árvores de decisão? Mais ou menos eficientes?

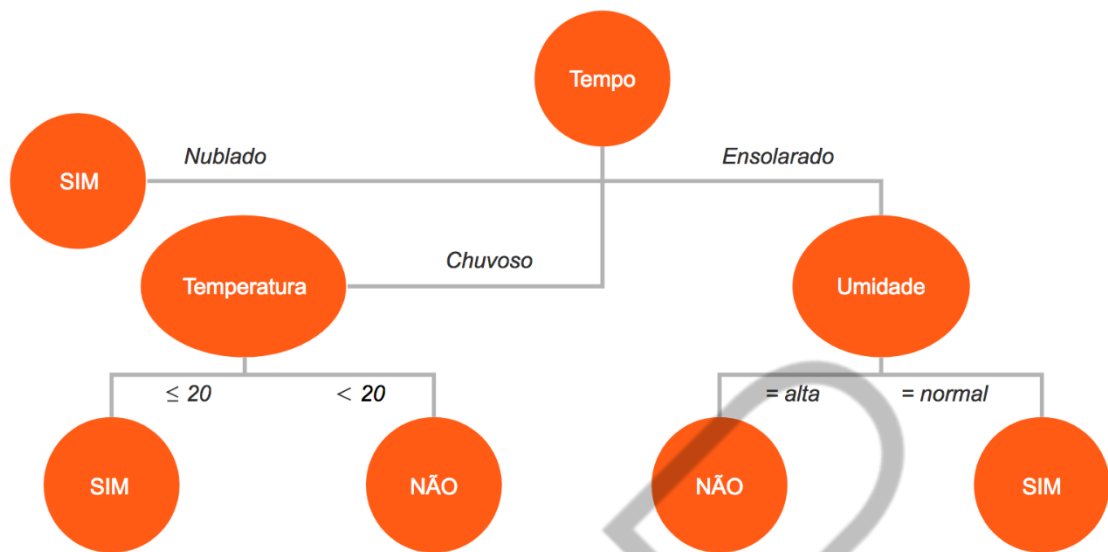


Figura 5.6. Árvore de decisão gerada sobre os dados da Tabela 5.2 “Características observadas para a decisão de sair ou não para jogar tênis”
Fonte: Elaborado pelo autor (2017)

5.4.2 Support Vector Machines (SVM), um paradigma matemático

O SVM (*Support Vector Machines*) é um paradigma matemático para a criação de classificadores. É uma das técnicas mais utilizadas em aprendizado supervisionado, por basicamente 3 razões (uma ideia básica, um truque hábil e não é paramétrico):

- Generaliza bem para dados não vistos, pois define um **separador de margem máxima** em espaço multidimensional (cada atributo do problema define uma dimensão do espaço).
- Embora crie uma separação linear no espaço multidimensional, pode utilizar **kernels** (truque hábil) que transformam o problema num espaço de dimensão superior. Assim, os problemas que não têm separação linear nos dados de entrada (atributos) podem ter separação linear num espaço de maior dimensão (gerado pelo kernel utilizado).

- Não é paramétrico e armazena os dados de treinamento que são os mais indicativos para o separador de margem máxima (são os *support vector machines*).

Uma das limitações do SVM é que ele não trata diretamente de valores nominais, pois tem de gerar um espaço multidimensional, com cada atributo em um dos eixos desse espaço. Como representar valores nominais em um espaço numérico? Assim, problemas com valores nominais devem ter esses valores convertidos devidamente para valores numéricos. Há diversas técnicas de conversão, sendo a **one-hot-encoding** uma das mais utilizadas.

Considere o atributo tempo, que pode assumir três possíveis valores: *chuvoso*, *nublado* e *ensolarado*. Utilizando o *one-hot-encoding*, geraremos tantos atributos quantos forem os possíveis valores do atributo. Nesse caso, três. Cada um dos atributos gerados será binário, ou seja, assumirá apenas os valores 1 ou 0. Veja o exemplo em que, para representar um valor do atributo tempo utilizam-se três atributos numéricos (binários).

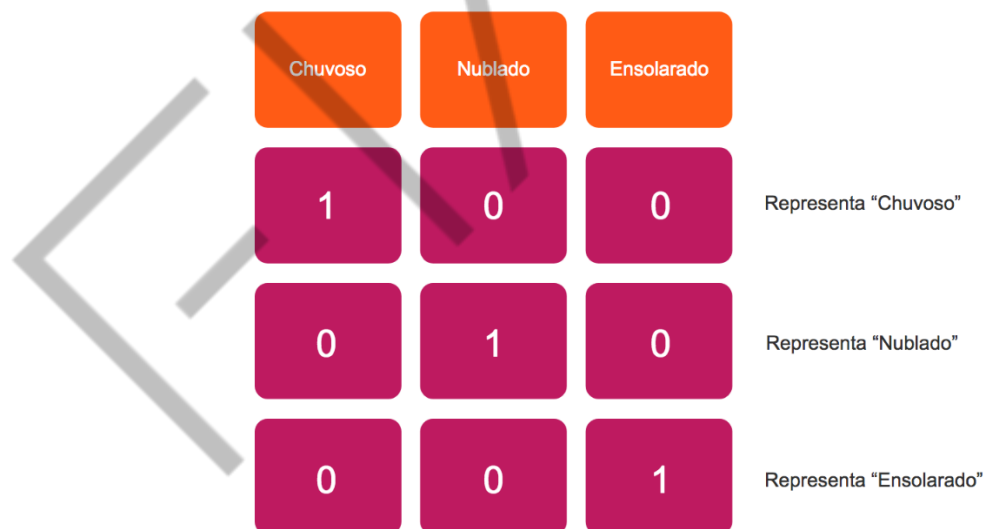


Figura 5.7. Conversão de valores nominais do atributo "tempo" para valores numéricos com a técnica "one-hot-encoding"

Fonte: Elaborado pelo autor (2017)

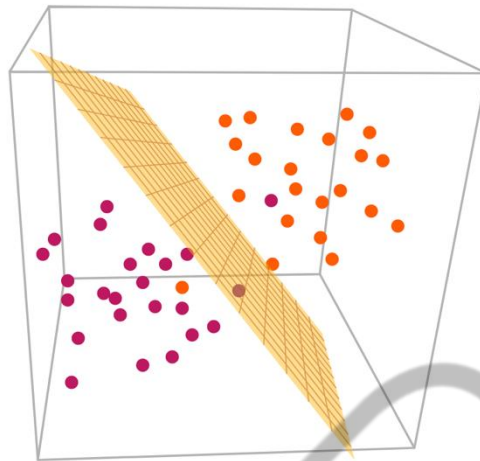


Figura 5.8. Hiperplano de separação entre duas classes em um espaço multidimensional
Fonte: Elaborado pelo autor (2017)

O SVM é essencialmente uma técnica de classificação binária, pois ele busca obter um hiperplano de separação entre duas classes, como ilustrado na Figura “Hiperplano de separação entre duas classes em um espaço multidimensional”. Mas o que fazer quando meu problema tem mais de duas classes? Nesses casos, treinam-se diversos classificadores binários e utiliza-se alguma métrica para escolher uma das classes.

Imagine um problema com três classes, representadas por cores: azul, verde e vermelho. Para decidir entre essas classes tem-se, por exemplo, duas formas de combinar classificadores binários SVM, a saber: **Um contra Todos** ou **Um Contra Um**. A figura “Formas de combinar classificadores binários em problemas multiclasse” ilustra essas duas formas de combinação.

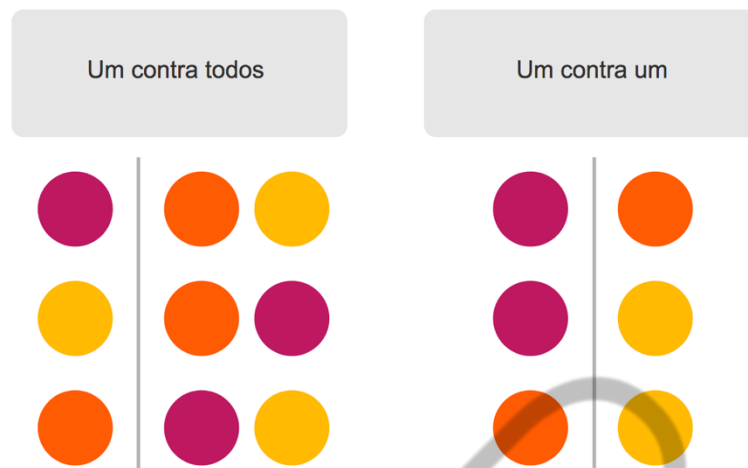


Figura 5.9. Formas de combinar classificadores binários em problemas multiclasse
Fonte: Elaborado pelo autor (2017)

Vale salientar que a maioria das implementações SVM, em diversos pacotes de algoritmos de aprendizado de máquina, tratam automaticamente os problemas multiclasse.

5.4.3 Regressão linear, problema de classes contínuas

Tanto as árvores de decisão quanto o SVM tratam de problemas cujas classes são discretas, ou seja, quando já se sabe a priori a quantidade de classes a serem tratadas. Mas e quando o problema não tem um número predefinido de classes? Isto é, a classe é definida como um valor contínuo?

Por exemplo, em grandes portais de vendas pela internet, em que se tem milhões de produtos à venda, como tratar o preço desses produtos de forma dinâmica? Seria interessante monitorar a concorrência e definir automaticamente o preço dos produtos com o intuito de se obter o maior volume de vendas.

Nesse problema, a classe é contínua, pois o preço pode variar continuamente em um intervalo de valores. Imagine o produto X, ele pode custar desde \$ 0,01 até \$ 100,00.

Uma técnica muito utilizada nesses tipos de problema é a **Regressão Linear**, que é uma técnica matemática que visa minimizar o erro de uma equação cuja representação geométrica se adapte a um conjunto de pontos. Esse conjunto de pontos são os exemplos do problema a ser solucionado.

Em outras palavras, a regressão linear é a definição de uma equação que gera uma linha que melhor se adequa ao relacionamento entre os atributos de entrada X (dados de treinamento) e as classes Y (contínuas) do problema. Isso é feito obtendo-se coeficientes para os atributos de entrada X . Por exemplo, o preço de um produto Y (classe) é dado de acordo com seus atributos de entrada X (como nome, marca, modelo, etc., que, por serem valores nominais, devem ser convertidos em valores numéricos).

Considere a equação simplificada a seguir. A classe Y_i é definida como o resultado do somatório da constante α (que indica a interceptação da linha com o eixo vertical, numa representação geométrica) com os atributos X_i multiplicados pelo coeficiente β .

$$Y_i = \alpha + \beta X_i$$

Desta forma, consegue-se um valor contínuo Y , de acordo com os valores de X . Essa equação define uma reta, como ilustrada na figura “Exemplo de regressão linear, ilustrada geometricamente”, em que os pontos vermelhos são as instâncias de treinamento da regressão. A reta azul (regressão) é utilizada para, dado um valor de X , definir o valor Y (classe).

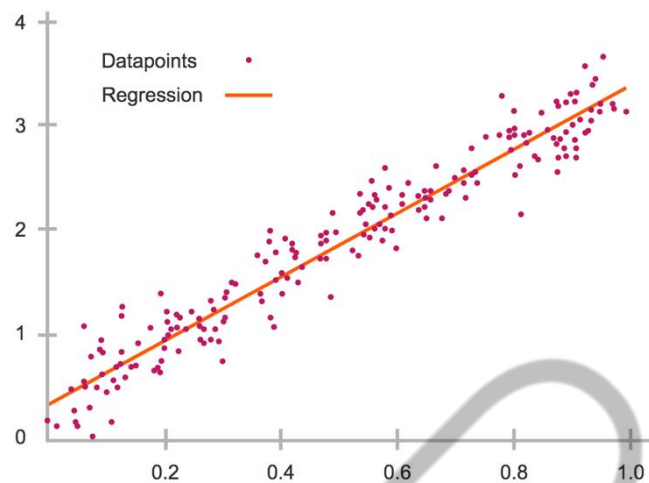


Figura 5.10. Exemplo de regressão linear, ilustrada geometricamente
Fonte: Elaborado pelo autor (2017)

5.5 Aprendizagem não supervisionada

Até agora, tratou-se de aprender a resolução de problemas quando se sabe quais são as possíveis respostas esperadas. Por exemplo, decidir se saio ou não para praticar esportes, qual será o preço do produto, etc. Não podemos nos esquecer, no entanto, de uma classe de problemas em que não se sabe quais classes esperar, ou seja, não temos exemplos de solução do problema.

Imagine um conjunto enorme de listas de compras realizadas em um supermercado. Será que podemos aprender alguma coisa com essa informação? Será que posso otimizar alguma coisa no supermercado, baseado no comportamento dos compradores, obtido de suas compras?

Imagine então a quantidade de logs gerado em um sistema bancário? E as informações coletadas por sensores meteorológicos em um país?

Esse tipo de aprendizado é dito não supervisionado, pois não se tem a definição ou anotação das classes do problema, mas ainda assim deseja-se aprender algo.

No aprendizado não supervisionado, podemos citar duas grandes tarefas: **associação** e **agrupamento**, dentre outras.

5.5.1 K-means, um método de agrupamento

O **K-means** é um dos métodos mais conhecidos do aprendizado não supervisionado. Esse é um método de agrupamento que, aplicando alguma técnica de similaridade vetorial, busca identificar K grupos nas instâncias de treinamento.

A figura “Aplicação do método K-means, com K = 3” ilustra a aplicação de um algoritmo de agrupamento K-means, com K = 3 (grupos azul, verde e vermelho, à direita). Para a definição de um grupo, o algoritmo escolhe um ponto que é chamado **centroide**. Definidos os centroides de cada grupo, um novo ponto (instância composta por atributos) é alocado no grupo a que estiver mais próximo, de acordo com a medida de similaridade escolhida.

Dentre as medidas de similaridade, podemos citar a distância **Euclidiana**, distância de **Manhattan**, e distância de **Mahalanobis**.

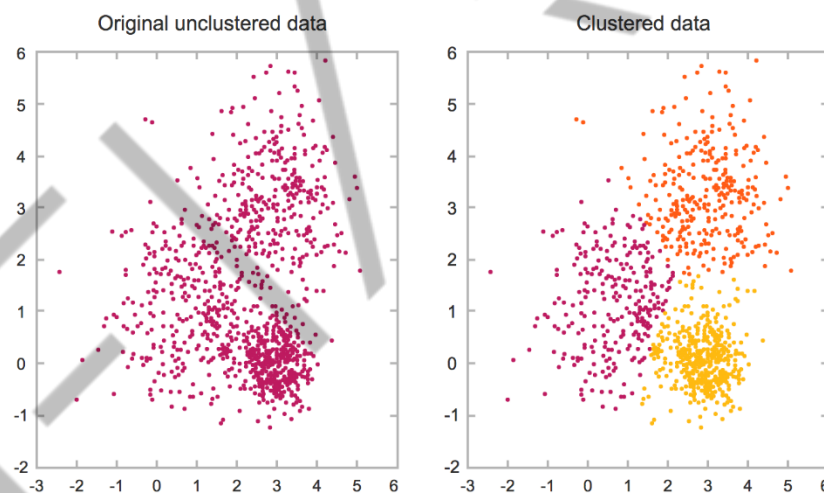


Figura 5.11. Aplicação do método K-means, com K = 3
Fonte: Elaborado pelo autor (2017)

Veja que aqui deve-se, a priori, definir a quantidade de grupos (K). Há outras técnicas que não necessitam que se defina a quantidade a priori, como os **métodos aglomerativos** ou **hierárquicos**.

5.6 Aprendizagem semissupervisionada

Entre o aprendizado supervisionado e o aprendizado não supervisionado há uma abordagem chamada de semissupervisionada. Nessa abordagem há diversos métodos que tomam vantagem de dados anotados, mas não deixam de aproveitar também os dados que não estão anotados e, geralmente, se tem em abundância.

Uma forma simples de aplicar o aprendizado semissupervisionado é utilizar os dados rotulados para treinar um modelo inicial. Esse modelo inicial será, então, aplicado aos dados rotulados com a finalidade de gerar novos dados rotulados. Esses novos dados rotulados podem conter erros, obviamente, pois provêm de uma anotação (classificação) automática. Dessa forma, a escolha de um método de treinamento que indique a confiabilidade de suas classificações é muito importante, pois permite escolher apenas as anotações que forem feitas com alta confiabilidade. Os exemplos com alta confiabilidade podem se juntar aos exemplos gerados manualmente e um novo modelo será treinado com mais dados anotados.

O algoritmo SVM, por exemplo, fornece, para cada classificação, a distância ao hiperplano de separação entre as classes do problema. Quanto maior for essa distância, maior a confiabilidade de classificação (consegue dizer o porquê?).

O processo descrito anteriormente é conhecido como **self-training**, um método de treinamento semissupervisionado. Existem outros algoritmos consagrados no paradigma semissupervisionado, dentre os quais podemos citar o **co-training** e suas variações (conhecidas como multivisão). Na realidade esses métodos são encapsuladores de outros algoritmos de aprendizado automático.

5.7 Redes neurais: matematizando o biológico

Redes neurais (RN) é um tópico estudado há muitas décadas (desde a década de 40, com a definição matemática de um neurônio: o Perceptron), mas seu início não foi favorecido dado o poder computacional da época. Já recentemente, com o aumento do poder computacional, principalmente com a popularização e barateamento do processamento paralelo em uma mesma máquina (GPUs), as redes

neurais emergiram e têm sido aplicadas com incrível sucesso em diversos problemas, com um novo rótulo: **Deep Learning**.

Dada a grande proeminência do assunto, é orientado que o aluno que lê este material busque maior aprofundamento no tema. Neste capítulo, as RNs estão abordadas muito superficialmente, por estar sendo abordadas com outras técnicas de aprendizado automático.

Basicamente, uma rede neural é composta por uma grande quantidade de neurônios artificiais, formalizados matematicamente. Uma formalização matemática é exemplificada na figura “Formalização matemática de um neurônio”. Um neurônio recebe (nos dendritos) e passa (ou não) impulsos elétricos (pelos axônios) no contato com outros neurônios. O que define se o sinal será repassado e com qual intensidade é o núcleo do neurônio.

Na formalização, cada neurônio tem n entradas (X_0 a X_n), e cada entrada é multiplicada (ponderada) por um peso w . Essas entradas ponderadas são somadas e uma função de ativação é aplicada, definindo a saída do neurônio, que será repassado para outros neurônios.

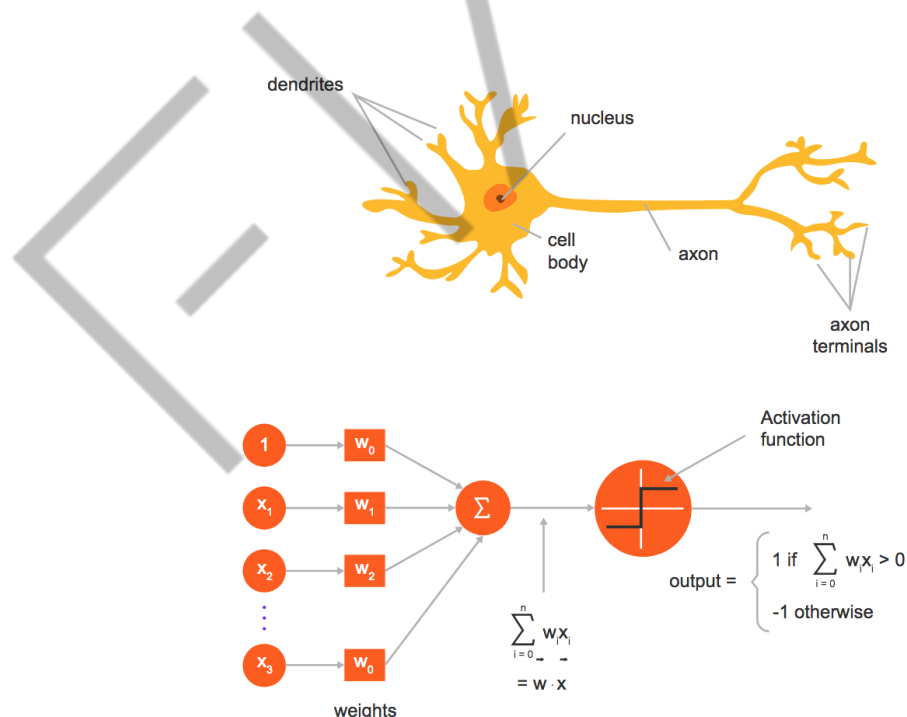


Figura 5.12. Formalização matemática de um neurônio
Fonte: Elaborado pelo autor (2017)

Há diversas funções de ativação. Na figura “Formalização matemática de um neurônio”, uma função de ativação *degrau* foi utilizada (há também as funções *sigmoide*, *tangente hiperbólica*, *softmax*, dentre outras). Nessa função, caso o somatório seja maior que zero, o sinal é repassado para o próximo neurônio. Caso o somatório seja menor ou igual a zero, o sinal é retido.

O aprendizado, na maioria das arquiteturas de redes neurais, é realizado no ajuste dos pesos aplicados às entradas X . O ajuste dos pesos W é realizado no processo chamado **backpropagation**. Nesse processo, os atributos X são inseridos na rede e a resposta obtida pela rede é comparada à resposta esperada (no caso de um aprendizado supervisionado). Caso a resposta esperada seja diferente da resposta predita, os pesos são ajustados com o intuito de que a rede prediga a resposta esperada. Um dos métodos mais conhecidos e utilizados no backpropagation é conhecido como **gradiente descent** (gradiente descendente).

Há diversas arquiteturas de redes neurais. Entenda arquitetura como a forma de combinação de milhares, milhões ou bilhões de neurônios com o intuito de realizar o aprendizado automático. Podemos citar as seguintes arquiteturas (as breves descrições apresentadas podem parecer um tanto complexas, mas à medida que o leitor tomar familiaridade com redes neurais, irá entender mais cada arquitetura):

- **Multilayer Perceptrons**: baseada no agrupamento de diversos Perceptrons em camadas, permite o tratamento de problemas não lineares. O Perceptron trata apenas de problemas lineares, isto é, cujas classes do problema sejam separáveis por um hiperplano no espaço de atributos.
- **Redes Convolucionais**: uma arquitetura amplamente utilizada no tratamento de imagens. Nessa arquitetura, os dados de entrada são tratados como matrizes numéricas tridimensionais cujas dimensões são alteradas a cada convolução na rede até gerar uma distribuição de probabilidades na camada de saída da rede, para tomada de uma decisão.
- **Redes Recorrentes**: muito utilizada no tratamento de dados sequenciais, como sequência de palavras, sons e dados temporais, por exemplo. Nessa arquitetura, a predição de um dado é utilizada na predição do próximo dado (série temporal, como os exemplos citados).

- Long Short-Term Memory (LSTM): nessa, tem-se a introdução de unidades LSTM que ajudam a rede a aprender correlação “mais espaçadas” entre dados em uma série temporal. Esse espaço pode ser até mais de 1000 passos de distância na sequência dos dados. As LSTMs são um avanço sobre as redes recorrentes para muitos problemas com longas séries temporais, como o Processamento da Linguagem Natural.
- Máquinas de Boltzmann: é um tipo de rede neural recorrente com uma característica mais estatística (estocástica). São úteis para resolver problemas combinatórios mais difíceis.
- Deep Belief Network: é uma opção para algumas limitações ao processo de aprendizado do *backpropagation*, tais como a necessidade de que os dados estejam todos rotulados (aprendizado supervisionado), tempo de aprendizado (muito demorado para redes muito profundas). O Deep Belief Network é uma opção de aprendizado não supervisionado em que os dados de entrada são agrupados. Os grupos aprendidos são as classes aprendidas. Um uso é o reconhecimento, agrupamento e geração de imagens, sequências de vídeos e dados de captura de movimento e o Processamento da Linguagem Natural.
- Deep Auto-Encoders: é uma arquitetura composta por duas redes simétricas, uma chamada de *encoder* e a outra, de *decoder*. São redes úteis para mapear uma representação em outra, como, por exemplo, mapear uma entrada textual em uma representação numérica.
- Generative Adversarial Network: é uma arquitetura bem profunda, composta por duas redes que são adversárias. Seu uso é no aprendizado da geração de alguma representação, seja imagem, texto ou música. Já imaginou uma rede que imite um compositor musical? Nessa arquitetura, em vez de prever uma classe Y com os dados de entrada X, ela tenta gerar o dados X para uma dada classe Y.

Essas arquiteturas podem ser combinadas, inclusive, para resolver os mais variados tipos de problemas, dependendo do tipo de dado a ser tratado, do tipo de predição (classes), etc.

Para um iniciante das redes neurais é importante a experimentação dessas diversas arquiteturas, desde as mais simples às mais complexas. Essa prática possibilitará a melhor escolha em face de um problema do mundo real.

5.8 Medidas de avaliação da aprendizagem

Por fim, como verificar o quanto um método automático de aprendizado aprendeu ou está aprendendo? Como comparar dois modelos gerados para uma tarefa específica? Há diversas medidas de avaliação que servem para essa finalidade. Veja, a seguir, algumas delas para um problema de classificação.

Considere Q a quantidade de instâncias disponíveis para realizar o aprendizado automático. Cada instância de Q tem uma classe y , que é a resposta esperada. Feito o aprendizado, cada instância q de Q , terá uma classe predita (y_p) pelo modelo gerado. A classe predita pode ser igual (acerto) ou diferente da esperada (erro).

Dadas as definições anteriores, podemos definir as seguintes medidas de avaliação:

Precisão: que é o número de acertos (a) sobre a quantidade de instâncias preditas pelo modelo (Q_p). Visto que nem sempre o modelo é capaz de predizer uma classe, nessa medida consideram-se apenas as respostas dadas pelo modelo gerado.

A precisão (P) é dada pela Equação 5.1.

$$P = \frac{a}{Q_p}$$

Equação 5.1. Cálculo da Precisão

Cobertura: número de acertos (a) sobre todas as predições esperadas pelo modelo (Q_e). Nessa medida, diferentemente da precisão, considera-se como denominador todas as instâncias fornecidas na avaliação do modelo, mesmo que o modelo não consiga dar alguma predição para alguma delas.

A cobertura (C) é dada pela Equação 5.2.

$$C = \frac{a}{Q_e}$$

Equação 5.2. Cálculo da Cobertura

Medida-F: é uma média harmônica que combina precisão e cobertura. Veja que algumas tarefas podem prezar mais pela precisão que pela cobertura. Outras, o inverso.

Para algumas tarefas, se o classificador der alguma resposta, é altamente desejável que essa resposta esteja correta. Então a precisão é mais importante que a cobertura. Para a maioria dos problemas, no entanto, é interessante um balanço entre a precisão e a cobertura, portanto, a medida-F é muito importante.

A medida-F (F) é dada pela Equação 5.3.

$$F = \frac{2 P C}{P + C}$$

Equação 5.3. Cálculo da Medida-F

Matriz de confusão: é uma forma matricial de análise das respostas de um modelo. Na matriz de confusão, verifica-se qualitativamente o comportamento do classificador.

Para isso, considere um modelo de classificação que distinga entre as classes a , b , c e d . O modelo pode prever todas as instâncias da classe a corretamente, mas pode prever algumas instâncias da classe a como sendo da classe b . Esses erros são contabilizados nas medidas de P , C e F , mas ficam implícitos. Pela matriz de confusão é possível analisar essas “confusões” feitas pelo classificador.

A tabela “Exemplo de Matriz de Confusão” ilustra uma matriz de confusão. Pela matriz de confusão, o modelo predisse corretamente todas as instâncias com classe esperada a . Já para a classe b , 6 instâncias estão corretas, 2 foram confundidas com a classe a , e 1 instância confundida com a classe d . As da classe c , 20 estão corretas e 1 errada para a classe a e 1 errada para a classe d . O que podemos concluir sobre as instâncias da classe d ?

Tabela 5.4. Exemplo de Matriz de Confusão

Esperada (ao lado) Predita (abaixo)	a	b	c	d
a	50	0	0	0
b	2	6	0	1
c	1	0	20	1
d	0	1	1	10

Fonte: Elaborado pelo autor (2017)

Outra informação importante é que os acertos estão na diagonal principal da matriz (células sombreadas) e os erros, nas outras células.

REFERÊNCIAS

ESTADÃO. **Tesla apresenta tecnologia de carros autônomos**. 22 abr. 2019. Disponível em: <<https://link.estadao.com.br/noticias/empresas,tesla-apresenta-tecnologia-de-carros-autonomos,70002799995>>. Acesso em: 25 jun. 2020.

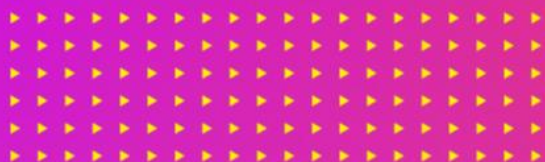
HAN, Jiawei.; KAMBER, Michele. **Data Mining: Concepts and Techniques**, 2nd edition, Morgan Kaufmann, 2006.

MITCHELL, Thomas M. **Machine Learning**. New York, NY, USA: McGraw-Hill, Inc., 1997.

RUSSEL, Stuart; NORVIG, Peter. **Inteligência Artificial**. Rio de Janeiro: Elsevier, 2013.

GLOSSÁRIO

Entropia	Grau de pureza de um conjunto. Conceito que vem da Teoria da Informação, que define a medida de “falta de informação”, mais precisamente, o número de bits necessários para representar uma informação em falta.
Estocástico	Da Teoria Probabilística, um processo estocástico é aquele cujos eventos são determinados aleatoriamente.
GPU	<i>Graphics Processing Unit</i> ou Unidade de Processamento Gráfico.
Heurística	Estratégia dependente de um problema definida na busca por uma solução viável, em casos de problemas altamente complexos (computacionalmente) em termos de tempo e espaço.
Outlier	Um ponto, ou instância, ou exemplo, muito diferente dos demais considerados em um problema.



INTELIGÊNCIA ARTIFICIAL
E COMPUTACIONAL

REPRESENTAÇÃO DA INCERTEZA

ERICK GALANI MAZIERO



6

LISTA DE FIGURAS

Figura 6.1. Exemplo de interpretador interativo do Python instalado em um sistema Linux (distribuição Ubuntu).....	9
Figura 6.2. Tela inicial do Jupyter. Por exemplo, é listado um notebook criado, com nome "Experiment.ipynb"	11
Figura 6.3. Iniciando um novo código Python.....	12
Figura 6.4. Exemplo de edição de código e execução bloco a bloco	12
Figura 6.5. Exemplo de execução de dois blocos de códigos e suas respectivas saídas.....	15
Figura 6.6. Gráfico gerado com os dados da avaliação do SVM com três <i>kernels</i> ...	17
Figura 6.7. Gráfico gerado com os dados da avaliação das árvores de decisão com variações da profundidade máxima.....	19
Figura 6.8. Exemplo de gráfico de avaliação do modelo de regressão linear	20

LISTA DE CÓDIGOS-FONTE

Código-Fonte 6.1. Importação dos módulos Python para os experimentos com aprendizado automático	14
Código-Fonte 6.2. Carregamento do <i>dataset</i> de câncer de mama e exibição de seus atributos e classes (discreta)	14
Código-Fonte 6.3. Carregamento do <i>dataset</i> de diabetes e exibição de seus atributos e classes (contínua)	14
Código-Fonte 6.4. Separação dos datasets nos conjuntos de treino e test.....	16
Código-Fonte 6.5. Treinamento e avaliação do SVM	16
Código-Fonte 6.6. Treinamento e avaliação de árvores de decisão	18
Código-Fonte 6.7. Treinamento e avaliação do algoritmo de regressão linear com o <i>dataset</i> de diabetes	20

SUMÁRIO

6 REPRESENTAÇÃO DA INCERTEZA	5
6.1 Incerteza.....	5
6.2 Lidando com a incerteza: Probabilidades.....	6
6.3 O teorema de Bayes.....	7
6.3 Hands On! Aplicando o aprendizado automático!	8
6.3.1 Ajustando o ambiente	8
6.3.2 Dados do experimento	10
6.3.3 Executando e interpretando o experimento	11
REFERÊNCIAS	22
GLOSSÁRIO	23

6 REPRESENTAÇÃO DA INCERTEZA

Neste capítulo veremos como lidar com a incerteza em aplicações de aprendizado de máquina. Mesmo as decisões humanas lidam, muitas das vezes implicitamente, com o incerto, pois não se pode prever todas as variáveis que levam a uma decisão.

Além da representação da incerteza, aproveitaremos o capítulo para uma abordagem prática ao aprendizado automático com o uso da linguagem *Python* e um módulo de *machine learning* amplamente utilizado, o *Scikit-Learn*. Os experimentos serão realizados, inclusive, no ambiente do *Jupyter Notebook*, que permite, dentre outras funcionalidades, visualização gráfica do *dataset* e execução do código, bloco a bloco.

6.1 Incerteza

Nem sempre temos acesso a exatamente todos os fatos de um determinado ambiente e evento. Se o tivéssemos, poderíamos prever tudo o que aconteceria com certa exatidão. Em ambientes reais, dificilmente conseguiremos prever com exatidão tudo que acontecerá. Isso se reflete nas interpretações e decisões de agentes inteligentes, inclusive os gerados com aprendizado de máquina.

Diversas variáveis não podem ser observadas, por estarem além do escopo do problema que estamos tentando resolver com inteligência artificial. Assim, temos de recorrer a métodos que levem em consideração as incertezas que permeiam uma predição automática.

Um agente inteligente poderia ter planos de contingência para cada situação incerta, devido aos dados parciais do ambiente a que tem acesso, mas essa solução pode não ser viável, pois a quantidade de situações não previstas pode crescer exponencialmente. Inclusive, pode-se não conseguir, a priori, um plano para cada imprevisto.

Imagine um sistema especialista que faça a triagem de paciente em um pronto-socorro. Sua base de conhecimento é composta de conhecimentos extraídos de especialistas em triagem e consiste basicamente de conjuntos de sintomas que

indicam possíveis “quadros” dos pacientes. Um paciente que chegue ao pronto-socorro com um conjunto de sintomas C1 pode encaixar-se nos “quadros” Q1, Q2 e Q3. Por se tratar de uma triagem automática, o sistema especialista, mesmo não tendo certeza do quadro em que o paciente está, deve atribuir o paciente a um dos quadros. Então, como lidar com a incerteza? Qual o mecanismo que o sistema especialista deve adotar para escolher o mais promissor?

6.2 Lidando com a incerteza: Probabilidades

Uma forma de lidar com a incerteza é com o uso das *probabilidades*. Considere, por exemplo, um sistema especialista que auxilie um dentista em identificar problemas dentários. Um sintoma *dor_de_dente* pode indicar *cáries*.

dor_de_dente => *cárie*

Mas nem todos os pacientes estão com *dor_de_dente* por causa da *cárie*, pois pode ser por outras causas:

dor_de_dente => *cárie* ou *periodontite* ou *abscesso*

Portanto, quanto menos características a serem observadas temos, menor a certeza que temos em uma previsão, ou classificação. No entanto, no exemplo anterior, suponhamos que 80% dos pacientes avaliados tinham o sintoma *dor_de_dente* devido a alguma *cárie*, então, lidamos com a incerteza de uma previsão baseada apenas nesse atributo com a afirmação: “O paciente com *dor_de_dente* tem *cárie*, com 80% de chances”.

É óbvio que em problemas reais não baseamos uma afirmação em apenas uma característica, dessa forma, lançamos mão da **teoria da probabilidade** para atribuir um grau de confiança a cada afirmação feita. Esse grau de confiança, geralmente, está entre os valores 0 e 1.

6.3 O teorema de Bayes

Em teoria da probabilidade, o teorema de Bayes (ou lei, ou regra de Bayes) descreve a probabilidade de um evento baseado na observação e quantização de conhecimentos anteriores relacionados ao evento. Em outras palavras, o teorema de Bayes computa a probabilidade (ou grau de confiança) de uma afirmação dado um conjunto de observações.

O teorema de Bayes define o seguinte:

$P(H|E)$ - a probabilidade de que a hipótese, ou proposição H , seja verdadeira dada a evidência E .

$P(E|H)$ - a probabilidade de que a evidência E será observada se a hipótese, ou proposição H , for verdadeira.

$P(H)$ - a probabilidade “a priori” de que a hipótese, ou proposição H , é verdadeira na ausência de qualquer evidência específica.

A probabilidade $P(A|B)$ é dada por (lei da probabilidade total):

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

Considere o seguinte exemplo: em um dado bairro, no caso do disparo de um alarme, deseja-se quantificar a chance de uma tentativa de roubo a uma casa. Para isso, tem-se as seguintes observações:

- quando há uma tentativa de roubo, em 95% dos casos, o alarme dispara;
- em apenas 1% dos casos, o alarme dispara por outros motivos;
- no bairro em questão, há uma chance de 1 em 10.000 (0.0001) de uma casa ser assaltada em determinado dia.

Podemos formular, então:

$P(\text{alarme}|\text{roubo}) = 0.95$, que é a probabilidade de um alarme disparar na tentativa de um roubo.

$P(\text{roubo}) = 0.0001$, probabilidade de um roubo.

$P(\text{alarme}|\text{não_roubo}) = 0.01$, probabilidade de um alarme por outros motivos.

$$\text{Então } P(\text{roubo}|\text{alarme}) = \frac{P(\text{alarme}|\text{roubo}) * P(\text{roubo})}{P(\text{alarme}|\text{não_roubo})} = \frac{0.95 * 0.0001}{0.01} = 0.0095$$

Dessa forma, podemos dizer que as chances de uma tentativa de roubo, quando um alarme dispara, são de 0,95%, ou seja, menos que 1%. Isto ocorreu, pois as chances de ocorrer uma tentativa de roubo no determinado bairro são muito pequenas (0.0001) em comparação com a probabilidade do alarme soar por qualquer outro motivo que não seja o roubo (0.01).

6.3 Hands On! Aplicando o aprendizado automático!

6.3.1 Ajustando o ambiente

Para preparar o ambiente dos experimentos, precisamos garantir que o Python esteja instalado. O restante dessa seção será conduzido considerando a versão 2.7 do Python. Também serão fornecidos comandos para os sistemas operacionais Windows e Linux (Ubuntu).

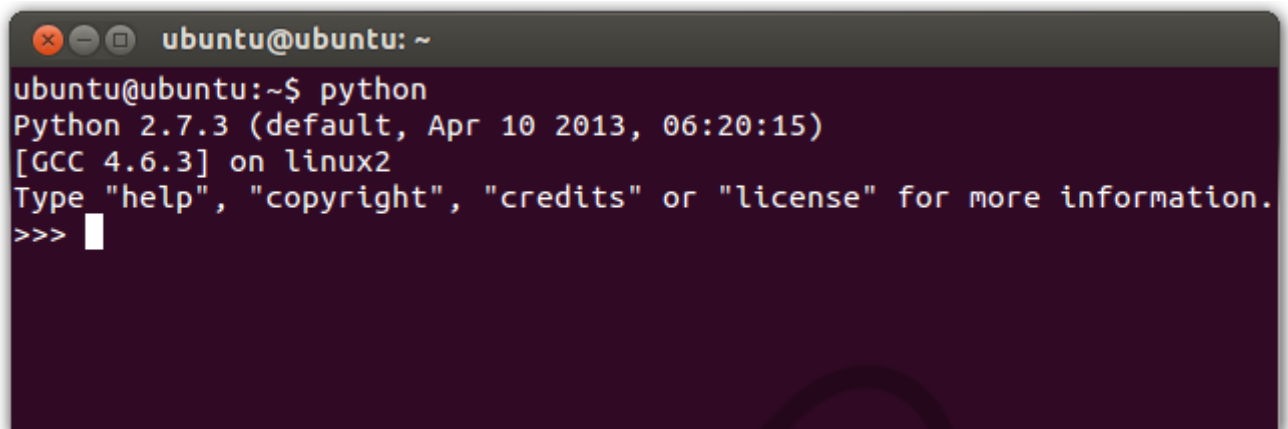
PYTHON

Primeiro, abra um terminal de comandos e verifique se o Python está instalado:

Windows: `Ctrl+R` e digite `cmd` (terminal). No terminal que abrirá, digite `python`.

Linux: Abra o Terminal e digite `python`.

Caso o Python já esteja instalado, será exibida uma nova linha com os símbolos `>>>`, como exemplificado a seguir.

A terminal window with a dark background and light text. The title bar shows 'ubuntu@ubuntu: ~'. The prompt is 'ubuntu@ubuntu:~\$'. The user has entered 'python'. The output shows 'Python 2.7.3 (default, Apr 10 2013, 06:20:15)', '[GCC 4.6.3] on linux2', and instructions to type 'help', 'copyright', 'credits' or 'license' for more information. The prompt '>>>' is followed by a cursor.

```
ubuntu@ubuntu: ~
ubuntu@ubuntu:~$ python
Python 2.7.3 (default, Apr 10 2013, 06:20:15)
[GCC 4.6.3] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> 
```

Figura 6.1. Exemplo de interpretador interativo do Python instalado em um sistema Linux (distribuição Ubuntu)

Fonte: Elaborado pelo autor (2018)

No caso de ausência do Python no sistema, proceda à instalação:

Windows: vá para <https://www.python.org/downloads/windows/> e faça o download de um instalador, por exemplo, “Windows x86 MSI installer”.

Linux: No terminal, digite: `sudo apt-get install python`

GERENCIADOR DE PACOTES PYTHON (PIP)

O pip é um utilitário indispensável para instalar alguns pacotes do Python, como o jupyter e scikit-learn. Então, vamos verificar se o pip já está no sistema:

Windows: Ctrl+R, digite `cmd` (terminal). Digite: `python -m ensurepip -upgrade`

Linux: Abra o Terminal e digite: `sudo apt-get install python-pip`

JUPYTER

Windows e Linux: abra o Terminal e digite: `python -m pip install jupyter`

SCIKIT-LEARN

O Scikit-learn é um conjunto de ferramentas para as mais diversas abordagens de aprendizado automático, tais como classificação, agrupamento, seleção de

modelos, redução de dimensionalidade e pré-processamento dos dados. Sua instalação é simples, basta:

Windows e Linux: abra o Terminal e digite: `python -m pip install -U scikit-learn`

OUTROS MÓDULOS

Como pré-requisitos para os experimentos, serão necessários os seguintes módulos Python: “pandas”, “numpy”, “scipy”, “matplotlib”. Para instalar, basta executar o comando abaixo:

Windows e Linux: abra o Terminal e digite: `python -m pip install pandas numpy scipy matplotlib`

Terminada a instalação das ferramentas anteriores, estamos prontos para iniciar nosso experimento de *machine learning* com dados da área médica. Mas ainda antes, vamos falar um pouco a respeito dos dados que serão utilizados.

6.3.2 Dados do experimento

Há diversos dados (*datasets*) disponíveis para experimentos de aprendizado automático disponíveis para as mais diversas áreas. No caso desse primeiro experimento, vamos considerar dois *datasets*, um de câncer de mama (*Breast Cancer Wisconsin (Diagnostic)* *Database:* [https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+\(Diagnostic\)](https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic))) e outro de diabetes (*Diabetes* *Data:* <https://www4.stat.ncsu.edu/~boos/var.select/diabetes.html>)

O *dataset* de câncer de mama foi gerado a partir de imagens digitalizadas de amostras retiradas de mamas. Essas imagens descrevem características dos núcleos celulares presentes nas imagens. Foram extraídos 30 atributos numéricos, tais como o raio e perímetro dos núcleos celulares. São 569 exemplos de amostras. Desses, 212 indicam tumores malignos e 357, benignos. Esse é um típico problema de classificação binária.

O *dataset* de diabetes consiste em dados recolhidos de 442 pacientes com diabetes. Portanto, não serve para detecção de diabetes, mas para verificar a

progressão da doença um ano após sua observação. Foram analisadas 10 características, como idade, sexo, índice de massa corporal, etc. A classe desse problema é contínua, isto é, indica a progressão da doença. Esse tipo de classe é indicado para a aplicação de regressão linear.

6.3.3 Executando e interpretando o experimento

Podemos executar os experimentos a seguir, utilizando qualquer editor de texto ou IDE específica para o Python, como PyCharm. No entanto, vamos aprender a utilizar a ferramenta Jupyter Notebook. Essa ferramenta é executada em algum browser web, como o Google Chrome ou Mozilla Firefox.

O Jupyter Notebook permite executar partes do código Python isoladamente, considerando todo o contexto do código que já foi executado, reexecutar alguma parte sem ter de executar o resto do código já executado. Além disso, esse ambiente de edição tem muitas *features* gráficas, com o uso do Matplotlib, por exemplo, permitindo visualizar gráficos e imagens logo abaixo do código Python que gerou a informação. No caso de experimentos com IA, esse ambiente é muito útil e produtivo.

Para iniciar o Jupyter, basta abrir o terminal e digitar: `jupyter notebook`. Logo após, o browser padrão abrirá a seguinte tela da imagem, que será exibida abaixo.



Figura 6.2. Tela inicial do Jupyter. Por exemplo, é listado um notebook criado, com nome "Experiment.ipynb"

Fonte: Desenvolvido pelo autor (2018)

Para iniciar um código Python, basta clicar em "New->Python 2", ", como ilustrado a seguir.



Figura 6.3. Iniciando um novo código Python
Fonte: Elaborado pelo autor (2018)

Como ilustrado na Figura 6.4, você pode iniciar a criação de seu código Python e ir executando à medida que quiser ver o resultado até então. No exemplo, no primeiro bloco, um texto foi atribuído a uma variável. No segundo bloco, o comando `print` foi utilizado para exibir o valor da variável. Veja que cada bloco foi executado independentemente. No caso de códigos mais complexos e longos, executar bloco a bloco pode ser uma “mão na roda” para verificar os resultados intermediários ou *debugar* o código.

Como não é o objetivo neste capítulo apresentar esse ambiente de edição e execução de código Python, vamos ao que interessa: experimentos de *machine learning*!

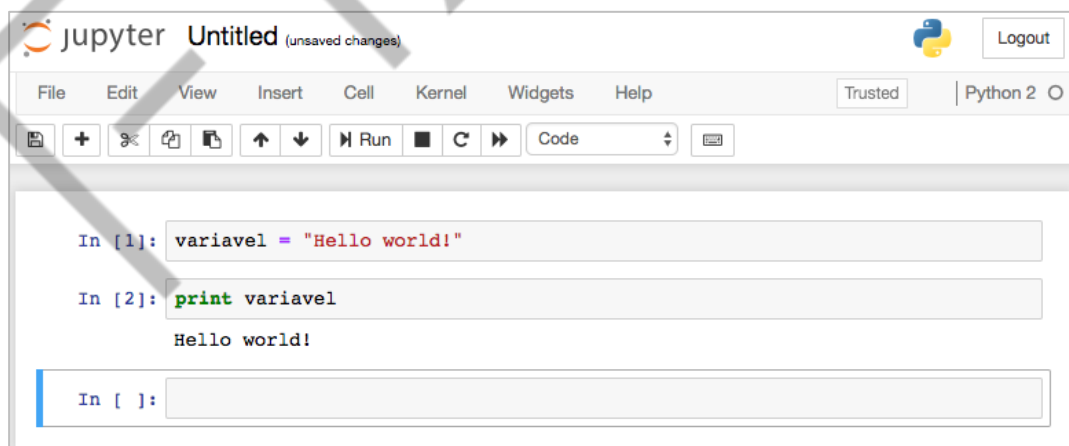


Figura 6.4. Exemplo de edição de código e execução bloco a bloco
Fonte: Elaborado pelo autor (2018)

O roteiro do experimento é o seguinte:

1. Carregar os módulos Python a serem utilizados.
2. Carregar os *datasets* a serem utilizados para o aprendizado automático.
3. Treinar modelos de aprendizado automático, variando algum parâmetro e fazer sua avaliação.

Que fique claro uma coisa, você terá que explorar mais modelos de aprendizado automático, assim como seus parâmetros! O Scikit-Learn, dentre tantos outros módulos Python, contém inúmeros modelos de classificação, regressão, agrupamento, etc. que você deve conhecer e explorar. Tal conhecimento fará com que você escolha o melhor modelo, com os melhores parâmetros para o problema que enfrentar.

Carregamento dos módulos

O Código-Fonte 6.1 “Importação dos módulos Python para os experimentos com aprendizado automático” mostra os módulos necessários para os experimentos a seguir. O módulo **sklearn** contém os *datasets* de câncer de mama (carregados pela função *load_breast_cancer*) e diabetes (*load_diabetes*). Contém também os algoritmos de aprendizado de máquina, dentre eles o *svm* (support-vector machines), *DecisionTreeClassifier* (árvores de decisão) e *LinearRegression* (regressão linear). Além dos dados e algoritmos de treinamento, contém um importante módulo, o *train_test_split*, que faz a separação do *dataset* em dados de treinamento e teste. Importante lembrar que o conjunto de teste é apenas utilizado para avaliar o modelo gerado por um algoritmo de treinamento e não deve ser utilizado durante o treinamento.

```
from sklearn.datasets import load_breast_cancer
from sklearn.datasets import load_diabetes
from sklearn import svm
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
%matplotlib inline
```

Código-Fonte 6.1. Importação dos módulos Python para os experimentos com aprendizado automático

Fonte: Elaborado pelo autor (2018)

O módulo **matplotlib** faz a exibição de gráficos diretamente no jupyter notebook. Um gráfico agiliza a análise dos dados. Como será visto à frente, cada algoritmo de aprendizado tem parâmetros que devem ser ajustados para que o modelo de aprendizado gerado obtenha a melhor avaliação nos dados de teste.

O matplotlib traz diversas possibilidades de gráficos, o que ajudará na escolha do melhor parâmetro para os algoritmos. O trecho de código `%matplotlib inline` é necessário para que os gráficos gerados sejam exibidos diretamente no notebook e não salvos em um arquivo de imagem.

Carregamento dos datasets

O Código-Fonte 6.2 e Código-Fonte 6.3 exibem, respectivamente, o carregamento dos *datasets* de câncer de mama e diabetes. Para cada *dataset*, são exibidos os nomes dos atributos (método `.features_names`). Para o dataset de câncer de mama, as classes são discretas (maligno ou benigno). Já para o de diabetes, a classe é um valor contínuo (que varia de 25.0 a 346.0).

```
dataset_cancer = load_breast_cancer()
print dataset_cancer.feature_names
print dataset_cancer.target_names
```

Código-Fonte 6.2. Carregamento do *dataset* de câncer de mama e exibição de seus atributos e classes (discreta)

Fonte: Elaborado pelo autor (2018)

```
dataset_diabetes = load_diabetes()
print dataset_diabetes.feature_names
print dataset_diabetes.target
```

Código-Fonte 6.3. Carregamento do *dataset* de diabetes e exibição de seus atributos e classes (contínua)

Fonte: Elaborado pelo autor (2018)

Para obter a visualização dos *prints* dos códigos fonte anteriores, no *jupyter* notebook, basta clicar no botão “Run”, quando o foco estiver no respectivo bloco de

código. A Figura 6.4 “Exemplo de edição de código e execução bloco a bloco” mostra a saída produzida para o Código-Fonte 6.2 e Código-Fonte 6.3. O *dataset* de câncer de mama tem uma lista de 30 atributos (*features_names*) e duas classes (*target_names*). Já o *dataset* de diabetes tem 10 atributos (*features_names*) e, para cada instância do *dataset*, um valor para a classe (*target*).

```
In [42]: dataset_cancer = load_breast_cancer()
print dataset_cancer.feature_names
print dataset_cancer.target_names

['mean radius' 'mean texture' 'mean perimeter' 'mean area'
 'mean smoothness' 'mean compactness' 'mean concavity'
 'mean concave points' 'mean symmetry' 'mean fractal dimension'
 'radius error' 'texture error' 'perimeter error' 'area error'
 'smoothness error' 'compactness error' 'concavity error'
 'concave points error' 'symmetry error' 'fractal dimension error'
 'worst radius' 'worst texture' 'worst perimeter' 'worst area'
 'worst smoothness' 'worst compactness' 'worst concavity'
 'worst concave points' 'worst symmetry' 'worst fractal dimension']
['malignant' 'benign']

In [41]: dataset_diabetes = load_diabetes()
print dataset_diabetes.feature_names
print dataset_diabetes.target

['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
[151. 75. 141. 206. 135. 97. 138. 63. 110. 310. 101. 69. 179. 185.
 118. 171. 166. 144. 97. 168. 68. 49. 68. 245. 184. 202. 137. 85.
 131. 283. 129. 59. 341. 87. 65. 102. 265. 276. 252. 90. 100. 55.
 61. 92. 259. 53. 190. 142. 75. 142. 155. 225. 59. 104. 182. 128.
 52. 37. 170. 170. 61. 144. 52. 128. 71. 163. 150. 97. 160. 178.
 48. 270. 202. 111. 85. 42. 170. 200. 252. 113. 143. 51. 52. 210.
 65. 141. 55. 134. 42. 111. 98. 164. 48. 96. 90. 162. 150. 279.
 92. 83. 128. 102. 302. 198. 95. 53. 134. 144. 232. 81. 104. 59.
 246. 297. 258. 229. 275. 281. 179. 200. 200. 173. 180. 84. 121. 161.
 99. 109. 115. 268. 274. 158. 107. 83. 103. 272. 85. 280. 336. 281.
 118. 317. 235. 60. 174. 259. 178. 128. 96. 126. 288. 88. 292. 71.
 197. 186. 25. 84. 96. 195. 53. 217. 172. 131. 214. 59. 70. 220.
 268. 152. 47. 74. 295. 101. 151. 127. 237. 225. 81. 151. 107. 64.
 138. 185. 265. 101. 137. 143. 141. 79. 292. 178. 91. 116. 86. 122.
 72. 129. 142. 90. 158. 39. 196. 222. 277. 99. 196. 202. 155. 77.
 191. 70. 73. 49. 65. 263. 248. 296. 214. 185. 78. 93. 252. 150.
 77. 208. 77. 108. 160. 53. 220. 154. 259. 90. 246. 124. 67. 72.
 257. 262. 275. 177. 71. 42. 187. 175. 78. 51. 258. 215. 203. 243.]
```

Figura 6.5. Exemplo de execução de dois blocos de códigos e suas respectivas saídas
Fonte: Desenvolvido pelo autor (2018)

Treinamento e avaliação dos modelos

Antes de realizar o treinamento, os *datasets* devem ser separados em treino e teste. Considere *X_train_can* como os atributos de treinamento e *X_test_can*, os de teste do *dataset* de câncer de mama. *Y_train_can* e *y_test_can*, respectivamente, as classes do *dataset* de câncer de mama. Analogamente, é feita a divisão para o *dataset* de diabetes.

```
X_train_can, X_test_can, y_train_can, y_test_can = train_test_split(
dataset_cancer.data, dataset_cancer.target, stratify=dataset_cancer.target,
random_state=42)
```

```
X_train_dia, X_test_dia, y_train_dia, y_test_dia = train_test_split(
dataset_cancer.data, dataset_cancer.target, stratify=dataset_cancer.target,
random_state=42)
```

Código-Fonte 6.4. Separação dos datasets nos conjuntos de treino e test
Fonte: Elaborado pelo autor (2018)

A seguir, serão exibidos o treinamento e avaliação de dois algoritmos de treinamento, a saber, o SVM e Árvores de Decisão. Vale salientar novamente que diversos outros algoritmos estão disponíveis e fica como exercício sua exploração nos *datasets* abordados e em outros *datasets*. O treinamento de modelos com SVM é apresentado no Código-Fonte 6.5. São criadas duas listas (*training_accuracy* e *test_accuracy*) que armazenarão as avaliações do modelo gerado tanto no conjunto de treinamento quanto no de teste.

Veja que os resultados obtidos no conjunto de treinamento devem sempre ser altos, próximos de 1.0 (100%), pois foi nesses dados que o algoritmo se baseou para gerar o modelo. Já os resultados obtidos no conjunto de teste são os valores que indicam o quão acurado está o modelo gerado. São utilizadas essas listas, pois a cada iteração do comando *for* é testado um parâmetro diferente.

O SVM trabalha com o uso de um *kernel* que faz a transformação dimensional dos atributos (pesquise um pouco mais sobre o SVM). Há diversos possíveis *kernels*, então são testados três: *linear*, *rbf* (*radial basis function*) e *sigmoid*.

```
training_accuracy = []
test_accuracy = []

kernels = ['linear', 'rbf', 'sigmoid']
for kernel in kernels:
    svm_model = svm.SVC(kernel=kernel)

    svm_model.fit(X_train_can, y_train_can)
    training_accuracy.append(svm_model.score(X_train_can, y_train_can))
    test_accuracy.append(svm_model.score(X_test_can, y_test_can))

plt.plot(kernels, training_accuracy, label='Acuracia no conj. treino')
plt.plot(kernels, test_accuracy, label='Acuracia no conj. teste')
plt.ylabel('Accuracy')
plt.xlabel('Kernels')
plt.legend()
```

Código-Fonte 6.5. Treinamento e avaliação do SVM
Fonte: Elaborado pelo autor (2018)

O módulo *matplotlib* (*plt*) é utilizado para criar um gráfico de acurácia por *kernel* utilizado. O respectivo gráfico é visto na Figura 6.6. Essa é uma vantagem do uso do jupyter notebook: é possível executar um bloco de código e já visualizar seu resultado. A curva azul indica a acurácia do modelo com os três *kernels*, já a linha laranja, indica a acurácia no conjunto de teste. Para escolher o melhor *kernel*, utilizamos a linha laranja e, como pode ser concluído, o *kernel linear* foi o que melhor

se comportou nesse problema, com uma acurácia de aproximadamente 0.95 (95%). Já os outros *kernels* ficaram com acurácias abaixo de 0.65.

```
In [48]: training_accuracy = []
test_accuracy = []

kernels = ['linear', 'rbf', 'sigmoid']
for kernel in kernels:
    svm_classifier = svm.SVC(kernel=kernel)

    svm_classifier.fit(X_train_cancer, y_train_cancer)
    training_accuracy.append(svm_classifier.score(X_train_cancer, y_train_cancer))
    test_accuracy.append(svm_classifier.score(X_test_cancer, y_test_cancer))

plt.plot(kernels, training_accuracy, label='Acuracia no conj. treino')
plt.plot(kernels, test_accuracy, label='Acuracia no conj. teste')
plt.ylabel('Accuracy')
plt.xlabel('Kernels')
plt.legend()
```

Out[48]: <matplotlib.legend.Legend at 0x110ad0950>

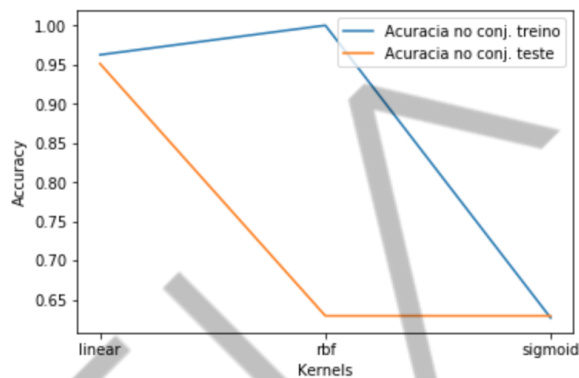


Figura 6.6. Gráfico gerado com os dados da avaliação do SVM com três *kernels*

Fonte: Elaborado pelo autor (2018)

O uso de árvores de decisão é feito no código abaixo. Nesse caso também foram utilizadas duas listas para armazenar os resultados das avaliações. O parâmetro que foi escolhido para exemplificar o melhor ajuste foi a profundidade máxima (*max_depth*) da árvore de decisão.

Fica, claro, pela “Figura 6.7”, que as árvores geradas com profundidade máxima 4 e 5 obtiveram os melhores resultados na avaliação.

```
training_accuracy = []
test_accuracy = []

prof_max = range(1,10)

for md in prof_max:
    tree = DecisionTreeClassifier(max_depth=md, random_state=0)
    tree.fit(X_train_can, y_train_can)
    training_accuracy.append(tree.score(X_train_can, y_train_can))
    test_accuracy.append(tree.score(X_test_can, y_test_can))
```

```
plt.plot(prof_max, training_accuracy, label='Acuracia no conj. treino')
plt.plot(prof_max, test_accuracy, label='Acuracia no conj. teste')
plt.ylabel('Acuracia')
plt.xlabel('Profundidade Maxima')
plt.legend()
```

Código-Fonte 6.6. Treinamento e avaliação de árvores de decisão
Fonte: Elaborado pelo autor (2018)

```
In [50]: training_accuracy = []
test_accuracy = []

profundidade_maxima = range(1,10)

for md in profundidade_maxima:
    tree = DecisionTreeClassifier(max_depth=md, random_state=0)
    tree.fit(X_train_cancer, y_train_cancer)
    training_accuracy.append(tree.score(X_train_cancer, y_train_cancer))
    test_accuracy.append(tree.score(X_test_cancer, y_test_cancer))

plt.plot(profundidade_maxima, training_accuracy, label='Acuracia no conj. treino')
plt.plot(profundidade_maxima, test_accuracy, label='Acuracia no conj. teste')
plt.ylabel('Acuracia')
plt.xlabel('Profundidade Maxima')
plt.legend()
```

Out[50]: <matplotlib.legend.Legend at 0x111671f90>

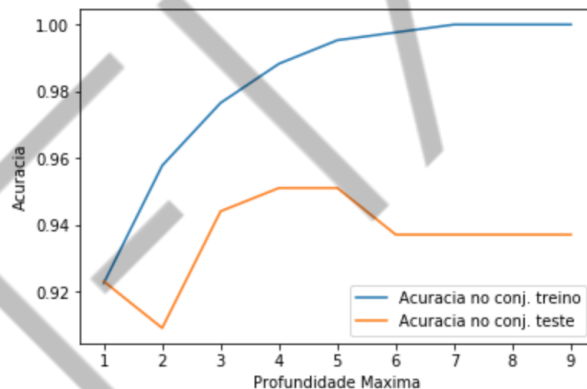


Figura 6.7. Gráfico gerado com os dados da avaliação das árvores de decisão com variações da profundidade máxima

Fonte: Elaborado pelo autor (2018)

Por fim, o Código-Fonte 6.7” apresenta o treinamento (método *.fit*) e avaliação (método *.score*) de um modelo de regressão linear, dado que o *dataset* de diabetes tem classes contínuas, indicando a progressão da doença em diabéticos, um ano após sua avaliação.

```
training_accuracy = []
test_accuracy = []
```

```

for interception in [True, False]:
    regr = LinearRegression(fit_intercept=interception)
    regr.fit(X_train_dia, y_train_dia)
    training_accuracy.append(regr.score(X_train_dia, y_train_dia))
    test_accuracy.append(regr.score(X_test_dia, y_test_dia))

plt.plot(["Interc", "No Interc"], training_accuracy, label='Acuracia no conj. treino')
plt.plot(["Interc", "No Interc"], test_accuracy, label='Acuracia no conj. teste')
plt.ylabel('Acuracia')
plt.xlabel('Fit Intercept')
plt.legend()

```

Código-Fonte 6.7. Treinamento e avaliação do algoritmo de regressão linear com o *dataset* de diabetes

Fonte: Elaborado pelo autor (2018)

Foi variado apenas um parâmetro, o *fit_intercept*, que assume valores booleanos (*True* ou *False*). Um gráfico foi gerado para escolher o melhor parâmetro (Figura 6.8 Exemplo de gráfico de avaliação do modelo de regressão linear”). Pelo gráfico, o melhor parâmetro é *fit_intercept* igual a *True*.

```

In [61]: training_accuracy = []
         test_accuracy = []

         for interception in [True, False]:
             regr = LinearRegression(fit_intercept=interception)
             regr.fit(X_train_dia, y_train_dia)
             training_accuracy.append(regr.score(X_train_dia, y_train_dia))
             test_accuracy.append(regr.score(X_test_dia, y_test_dia))

         plt.plot(["Interc", "No Interc"], training_accuracy, label='Acuracia no conj. treino')
         plt.plot(["Interc", "No Interc"], test_accuracy, label='Acuracia no conj. teste')
         plt.ylabel('Acuracia')
         plt.xlabel('Fit Intercept')
         plt.legend()

Out[61]: <matplotlib.legend.Legend at 0x111bb3550>

```

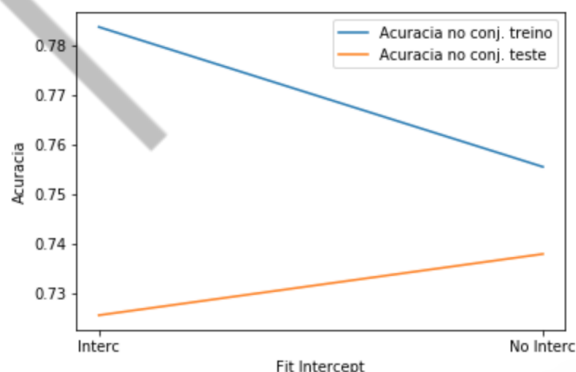


Figura 6.8. Exemplo de gráfico de avaliação do modelo de regressão linear

Fonte: Elaborado pelo autor (2018)

Após o treinamento dos modelos exemplificados acima, eles podem ser utilizados para classificar dados ainda não vistos (ou fazer a regressão linear deles), ou seja, classificando dados num cenário real.

Os dados que serão classificados devem ter os mesmos atributos encontrados no treinamento. Por exemplo, o *dataset* de diabetes tem 10 atributos: ('age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6'), então, ao utilizar o modelo gerado para prever a progressão da diabetes, os dados de entrada devem ser valores para esses 10 atributos.

Não fique restrito aos *datasets*, algoritmos e parâmetros abordados neste capítulo, tem muito, muito mais a ser explorado! Mãos à obra!

REFERÊNCIAS

RUSSEL, Stuart; NORVIG, Peter. **Inteligência Artificial**. Rio de Janeiro: Elsevier, 2013.

EVANGELISTA

GLOSSÁRIO

Acurácia	Em uma avaliação de um modelo gerado por algoritmos de aprendizado de máquina, indica a porcentagem de acertos no conjunto de teste fornecido ao modelo.
Bayes (Thomas Bayes)	Pastor presbiteriano e matemático inglês que formulou o teorema de Bayes.

EVANGELISTA

INTELIGÊNCIA ARTIFICIAL
E COMPUTACIONAL

AGENTES DE SISTEMAS

ERICK GALANI MAZIERO

LISTA DE FIGURAS

Figura 7.1. Ilustração da segmentação de imagens	8
Figura 7.2. Exemplos de serviços disponíveis no Watson da IBM	11
Figura 7.3. Habilitação do serviço de interpretação textual do Watson	12
Figura 7.4. Informações sobre o serviço habilitado	13
Figura 7.5. Menu de opções do serviço habilitado	14
Figura 7.6. Tela inicial do Jupyter. No exemplo, é listado um notebook criado com nome "Experiment.ipynb"	15
Figura 7.7. Iniciando um novo código Python.....	15
Figura 7.8. Verificando mais módulos a serem importados (<i>auto-complete</i>).....	16

LISTA DE CÓDIGOS-FONTE

Código-Fonte 7.1. Importação dos módulos necessários para acesso a uma das APIs do Watson.....	16
Código-Fonte 7.2. Instanciando o objeto de acesso à API.....	17
Código-Fonte 7.3. Exemplo de consulta à API de interpretação textual (NLU) do Watson	17
Código-Fonte 7.4. Exibição identada do retorno da chamada à API	18
Código-Fonte 7.5. Outra chamada à API de NLU	19
Código-Fonte 7.6. Chamada à API com texto em Português.....	22

SUMÁRIO

7 AGENTES DE SISTEMAS	5
7.1 Processamento da Linguagem Natural	6
7.2 Processamento de Imagens	7
7.3 Hands On! Usando APIs de Inteligência Artificial!	9
7.3.1 Ajustando o ambiente	9
7.3.2 Acessando as Apis	14
REFERÊNCIAS	24
GLOSSÁRIO	25

7 AGENTES DE SISTEMAS

Sistemas computacionais clássicos atuam em suas tarefas por meio de algoritmos desenvolvidos para lidar com situações e dados já previstos. Por exemplo, um sistema de cadastro de pacientes de um consultório médico pode estar programado para apenas receber os dados digitados por um usuário e designá-los para suas respectivas tabelas em um banco de dados. Nesse sistema hipotético, algumas funcionalidades, como busca por pacientes e criação de relatórios, podem, também, estar implementadas.

Mas qual a diferença de um sistema desses para um sistema inteligente? Como tornar esse suposto sistema em um sistema inteligente? Imagine que, dentre as informações inseridas pelo usuário no sistema, estejam dados do paciente, como: data de nascimento, peso, altura, doenças anteriores e atuais, medicamentos que o paciente toma, se já fez alguma cirurgia, etc. etc. Um sistema inteligente pode analisar essas informações e correlacioná-las com as informações de outros pacientes com o intuito de diagnosticar alguma doença ou criar um alerta para o médico que atenderá o paciente, indicando algo que deve ser verificado.

Um sistema que seja dotado dessa “inteligência” foge dos moldes padrões de desenvolvimento de um sistema de consultório médico e contém um agente inteligente que o auxilia no tratamento da informação, assim como nas tomadas de decisão.

Neste curso, até o momento, vimos diversas abordagens e técnicas que permitem a criação de agentes inteligentes, tais como a representação do conhecimento, sistemas especialistas e o uso de aprendizado de máquina, considerando, inclusive, a incerteza nas tomadas de decisão.

Neste capítulo, veremos que tais abordagens e técnicas podem ser aplicadas em diversos escopos e tipos de dados. Cada qual com suas peculiaridades e técnicas mais indicadas. Consideraremos o Processamento da Linguagem Natural (tanto escrita quanto falada), o Processamento de Imagens e Robótica.

7.1 Processamento da Linguagem Natural

O Processamento da Linguagem Natural (PLN) visa capacitar os computadores a processar a linguagem natural. Uma linguagem natural, como o Português (usado para a escrita deste documento), tem peculiaridades que linguagens de programação não têm. Um compilador não fica na dúvida quando vê um comando, ele sabe exatamente o que fazer. Já uma frase em linguagem natural pode levar a uma ambiguidade de interpretação. Considere a simples frase a seguir:

“Eu vi o homem de binóculos.”

Quem estava de binóculos, “Eu” ou o “homem” que foi visto? Talvez, se disséssemos:

“Eu vi o leão de binóculos.”

Teríamos mais chances de interpretar que “Eu” estava de binóculos, pois leões não usam binóculos. Certo?

O PLN é composto de diversas técnicas que possibilitam a interpretação textual de forma automática, desde o entendimento de cada palavra até porções maiores como sentenças, parágrafos e documentos. Tal interpretação também ocorre em diversos níveis linguísticos: léxico, morfossintático, sintático, semântico e discursivo, visto que a complexidade e subjetividade de interpretação aumenta do léxico ao discursivo.

Para se familiarizar um pouco mais com o PLN, nessa área são desenvolvidos os tradutores automáticos, revisores ortográficos e gramaticais, e quaisquer outras ferramentas que necessitem lidar com textos de forma inteligente.

Um fato que tem tornado o interesse em PLN mais forte é que boa parte das informações disponíveis na Web estão em formato textual. Inclusive, nas redes sociais (como Facebook e Twitter) e blogs, as pessoas expressam suas opiniões, fazem suas críticas e sugestões, falam sobre os mais diversos assuntos, tornando tais conteúdos preciosíssimos para empresas, por exemplo. Empresas podem verificar a aceitação de seus produtos e serviços por meio dessas informações, assim como obter sugestões de melhoria em tais produtos e serviços.

Podemos citar diversas tarefas realizadas pelo PLN, tanto de análise, como de transformação e geração textual:

- Análise:
 - Etiquetação morfofssintática (identificação desambiguada das classes de palavras: substantivo, verbo, adjetivo, etc.);
 - Parsing Sintático (identificação da estrutura das sentenças em sujeito, verbo, objetos, adjuntos, etc.);
 - Análise de Sentimentos (se um trecho de texto tem um sentimento positivo, negativo ou neutro);
 - Identificação de entidades (identificação de locais, organizações, pessoas, etc.);
 - Parsing Discursivo (identificação dos motivos que levaram o produtor do texto a dizer o que disse e da forma que disse).
- Transformação:
 - Tradução automática (transformação automática da língua de um texto para outra);
 - Sumarização automática (redução do tamanho do texto mantendo as informações mais importantes).
- Geração textual (geralmente para colocar em linguagem natural algum conhecimento em alguma base de conhecimento, tal como é feito pelos *chatbots* mais avançados).

Ainda neste capítulo teremos mais um *hands on* e nele utilizaremos APIs que realizam diversos níveis de interpretação textual, possibilitando o desenvolvimento de sistemas que tenham certa capacidade linguística para tomada de decisão inteligente.

7.2 Processamento de Imagens

Diz um ditado: “uma imagem vale mais que mil palavras”. Então, se eu tiver a capacidade de interpretar textos, quero ter também a capacidade de interpretar imagens, que podem sumarizar, em si mesmas, muita informação. Um agente

inteligente, dependendo de seu propósito, deve ser capaz de perceber o ambiente à sua volta e isso inclui o Processamento de Imagens.

Imagine que você vá a uma loja para comprar uma camiseta igual a de uma pessoa que você viu na internet. Você pode levar para o vendedor uma foto da pessoa que você viu vestindo a camiseta que você quer comprar. O vendedor identificará os detalhes, como cor, modelo, estampa e dirá se tem ou não tal camiseta. Ou então, ele poderá sugerir camisetas semelhantes. Será que um site de vendas online poderia fazer o mesmo? Se ele for dotado de um sistema de Processamento de Imagens, sim!

Um outro exemplo são os carros autônomos, que são munidos de diversas câmeras para perceber o ambiente (além dos mais diversos tipos de sensores, como sonares e infravermelhos).

O Processamento de Imagens processa um lote de informações (imagem) e pode detectar os limites de cada objeto presente, assim como classificar esses objetos.

A figura “Ilustração da segmentação de imagens” ilustra a tarefa de segmentação de imagens. À esquerda da figura está a imagem original e à direita, a imagem com sombreamentos coloridos de acordo com os objetos identificados automaticamente nas imagens. Isso habilita um computador a identificar objetos nas imagens e fazer sua classificação, sejam pedestres, vias, carros, árvores, placas de sinalização, etc.

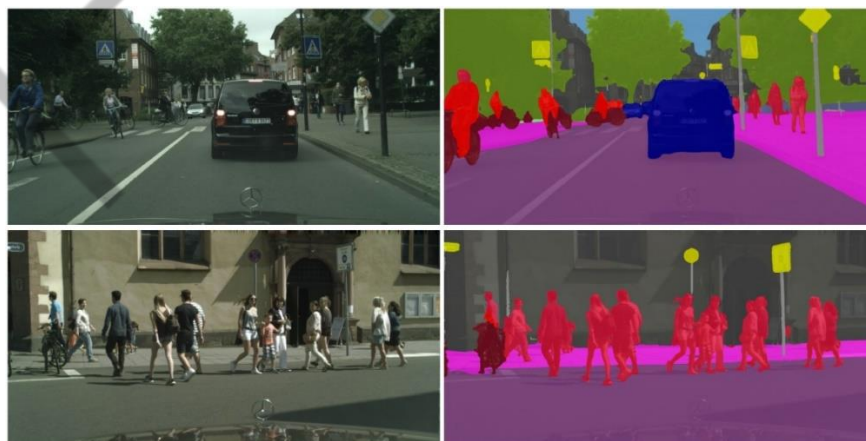


Figura 7.1. Ilustração da segmentação de imagens

Fonte: <http://vladlen.info/publications/feature-space-optimization-for-semantic-video-segmentation/> (2016)

Dentre as diversas tarefas do Processamento de Imagens, podemos citar:

- Segmentação de imagens (identificando os objetos presentes na imagem);
- Reconhecimento dos objetos segmentados (classificação dos objetos reconhecidos na imagem em 'coisas' do mundo real);
- Reconstrução de cenas (reconstrução de partes danificadas de uma ou mais imagens de um local, por exemplo);
- Restauração de imagens (removendo ruídos, por exemplo).

As APIs de IA também têm serviços de Processamento de Imagens, facilitando a integração desse tipo de inteligência computacional em aplicações.

7.3 Hands On! Usando APIs de Inteligência Artificial!

7.3.1 Ajustando o ambiente

Agora vamos utilizar um ambiente bem parecido com o da prática anterior. Vamos precisar do Python e seu gerenciador de pacotes, o PIP, e do Jupyter.

Antes de mais nada, vamos aprender a utilizar algumas APIs de um dos gigantes da computação: a IBM. Obviamente outras APIs, de outras companhias, poderiam ser utilizadas, tais como o Google Cloud, da Google, ou o Azure, da Microsoft.

A IBM disponibiliza uma série de serviços, para uma série de áreas, tais como Big Data, Internet das Coisas e Aprendizado de Máquina. Para essa última área, tem-se serviços para Processamento de Texto (escrito e falado) e Processamento de Imagens.

Conta IBM

O primeiro passo é a criação de uma conta IBM. Vá para o link a seguir e crie sua conta.

<https://console.bluemix.net/registration/>

Caso já tenha uma conta IBM, faça seu login.

<https://idaas.iam.ibm.com/>

O Watson é um conjunto de serviços disponíveis por meio de APIs, tais como a que utilizaremos para processar textos e imagens. Você pode visualizar esses serviços em:

<https://console.bluemix.net/developer/watson/services>

Na figura “Exemplos de serviços disponíveis no Watson da IBM”, são apresentados exemplos de serviços disponíveis na IBM Cloud. Há diversos relacionados ao Processamento da Linguagem Natural, tais como *Speech to Text* (que converte fala para texto escrito), *Language Translator* (que faz a tradução automática de uma língua destino para um alvo) e *Natural Language Understanding* (que realiza a interpretação textual, em seus diversos níveis). É exibido também o serviço de reconhecimento de imagens (*Visual Recognition*).

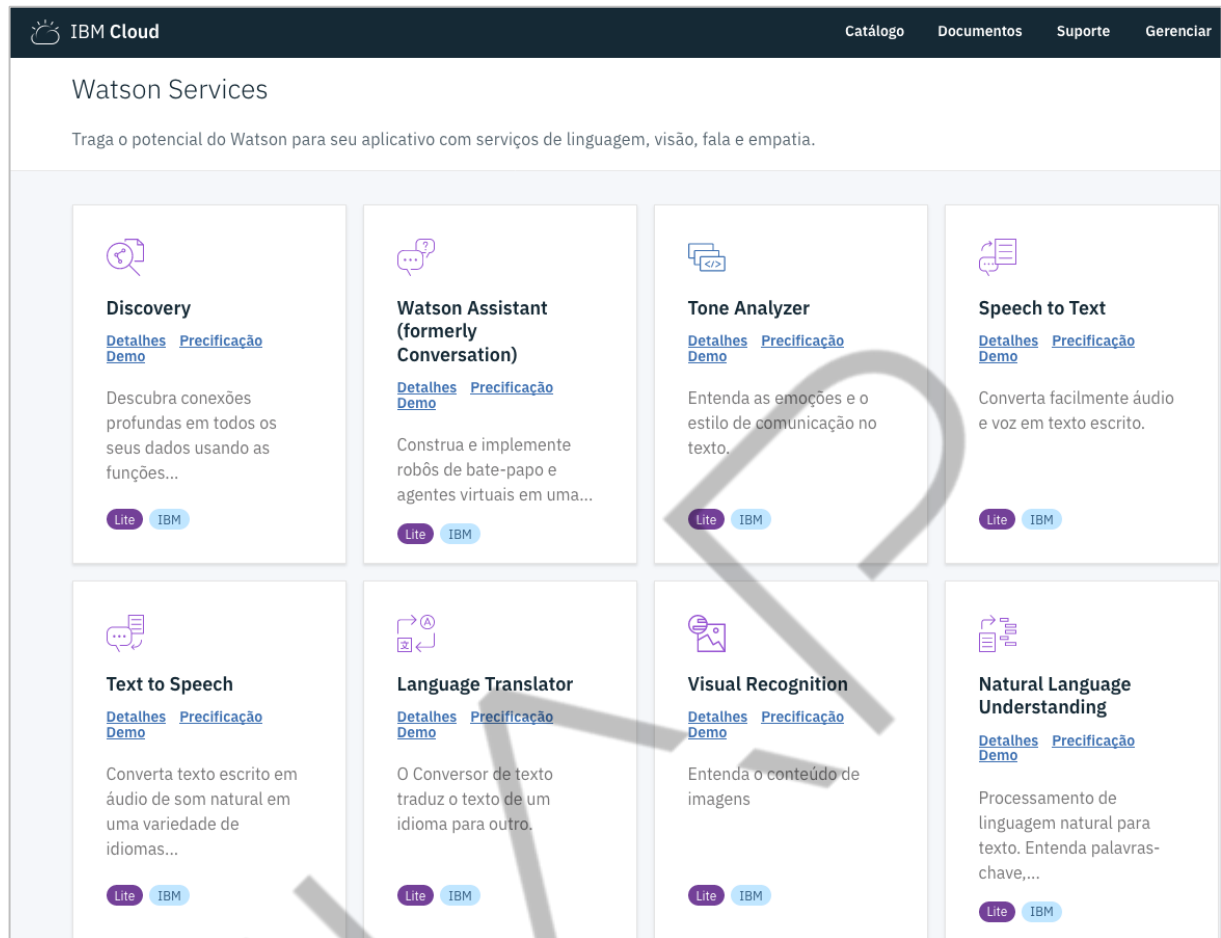


Figura 7.2. Exemplos de serviços disponíveis no Watson da IBM
Fonte: <https://console.bluemix.net/developer/watson/services> (2018)

Habilitando o serviço de interpretação textual

Acesse o serviço de interpretação textual (*Natural Language Understanding*):

`https://console.bluemix.net/catalog/services/natural-language-understanding`

Defina um nome para o serviço, por exemplo, “Natural Language Understanding-fiap”. Pode deixar as outras opções com seus valores padrões. Por exemplo, a região/local em que vai implementar o serviço diz respeito à infraestrutura física em que o serviço será executado.

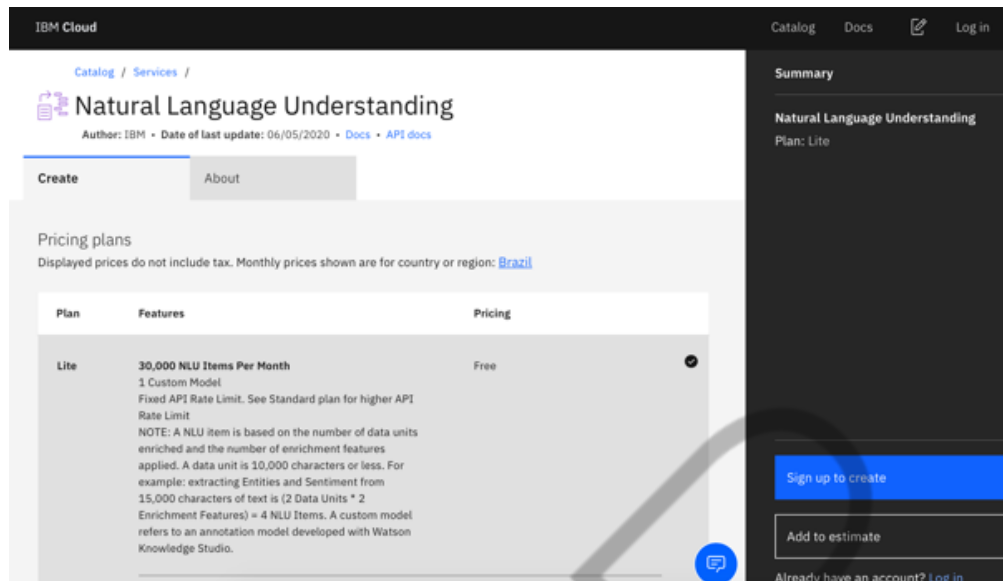


Figura 7.3. Habilitação do serviço de interpretação textual do Watson

Fonte: <https://console.bluemix.net/catalog/services/natural-language-understanding> (2020)

Esse tipo de serviço não é totalmente gratuito, então há um esquema de precificação. A IBM libera um plano (*Lite*) para testes, com precificação grátis. Essa é a primeira opção da tabela de Planos de precificação. Deixe essa opção selecionada e clique em “Criar”.

Você será direcionado a uma página com detalhes do serviço que acabou de habilitar, ilustrado na figura “Informações sobre o serviço habilitado”. Uma informação para uso posterior são as credenciais (*credentials*) com nome de usuário e senha. Anote essas duas informações, que serão necessárias para acessar a API habilitada.

The screenshot displays the Watson Natural Language Understanding console interface. At the top, it shows the service name 'Natural Language Understanding : Natural Langua...' and metadata including 'Localização: Sul dos EUA', 'Organização: [organization name]', and 'Espaço: dev'. Below this, a 'Get started with the service.' section offers a 'Getting started tutorial' button and an 'API reference' link, with a 'Plan: free Upgrade' indicator. The 'Credentials' section features a 'Show' button and a 'Configure credentials' link, displaying a JSON object for API credentials. The 'Try an API call' section provides instructions on how to use the service and includes a sample curl command for a POST request to the analyze endpoint.

```
{
  "url": "https://gateway.watsonplatform.net/natural-language-understanding/api",
  "username": ".....",
  "password": "....."
}
```

```
curl -X POST \
-u "{username}":"{password}" \
-H "Content-Type: application/json" \
```

Figura 7.4. Informações sobre o serviço habilitado

Fonte: <https://console.bluemix.net/> (2018)

Caso seja necessário acessar novamente as credenciais, apagar, gerar uma nova ou realizar outra operação, no menu lateral esquerdo (figura “Menu de opções do serviço habilitado”), clique em “Credenciais de serviço”. Para cada serviço que for habilitado, deve-se gerenciar uma respectiva credencial de acesso.

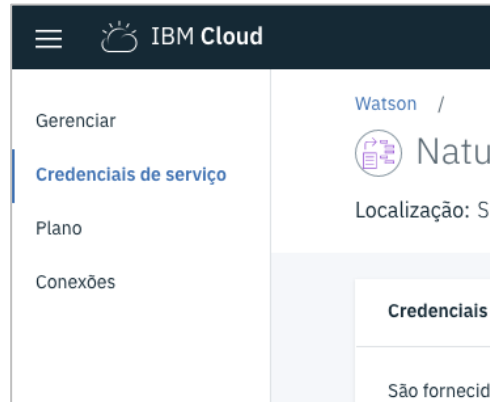


Figura 7.5. Menu de opções do serviço habilitado
Fonte: <https://console.bluemix.net/> (2018)

Instalação do módulo de acesso às APIs

Antes de acessar a API utilizando o Python, devemos instalar o módulo necessário. Podemos fazer isso pelo comando a seguir:

```
pip install --upgrade watson-developer-cloud
```

Caso deseje ver o código desse módulo de acesso às APIs, o projeto está disponível em:

```
https://github.com/watson-developer-cloud/python-sdk
```

7.3.2 Acessando as Apis

Pronto, se tudo correu certo até agora, vamos fazer nosso primeiro acesso à API. Inicie o Jupyter (basta abrir o terminal e digitar: `jupyter notebook`). Logo após, o browser padrão irá abrir e a seguinte tela da figura “Tela inicial do Jupyter” será exibida.



Figura 7.6. Tela inicial do Jupyter. No exemplo, é listado um notebook criado com nome "Experiment.ipynb"

Fonte: Imagem gerada pelo autor (2018)

Para iniciar um código Python, basta clicar em "New->Python 2", como ilustrado na figura "Iniciando um novo código Python".



Figura 7.7. Iniciando um novo código Python
Fonte: Imagem gerada pelo autor (2018)

Carregamentos dos módulos

Utilizaremos basicamente dois módulos: `json` e `watson_developer_cloud`.

O primeiro (`json`) é utilizado para manipular as informações retornadas pela API. O formato `json` (*Javascript Object Notation*) é amplamente utilizado em acessos a serviços. Ele é equivalente ao tipo dicionário do Python, isto é, um conjunto de um ou mais chave-valor. Já o módulo `watson_developer_cloud` contém os métodos e tipos necessários para acesso às diversas APIs do Watson.

O código-fonte a seguir apresenta a importação dos módulos. No primeiro bloco, são importados os módulos `json` e, a partir de `watson_developer_cloud`,

o procedimento `NaturalLanguageUnderstandingV1`. No bloco 2, são importadas quais as características do texto serão extraídas. A saber: Conceitos (`ConceptsOptions`), Relações (`RelationsOptions`), Emoções (`EmotionOptions`), Entidades (`EntitiesOptions`).

Além das características (`Features`) importadas, estão disponíveis outras, como Palavras-Chave (`KeywordsOptions`) e Metadados (`MetadataOptions`).

```
import json
from watson_developer_cloud import NaturalLanguageUnderstandingV1
from watson_developer_cloud.natural_language_understanding_v1 import
Features, ConceptsOptions, RelationsOptions, EmotionOptions, EntitiesOption
s, SemanticRolesOptions, SentimentOptions
```

Código-Fonte 7.1. Importação dos módulos necessários para acesso a uma das APIs do Watson
Fonte: Desenvolvido pelo autor (2018)

Essas características são utilizadas em cada requisição feita à API, dependendo do que se deseja obter como retorno.

Caso deseje ver todas as opções de importação a partir de `watson_developer_cloud`, faça como na Figura 7.8 “Verificando mais módulos a serem importados (*auto-complete*)”, aperte a tecla *tab* após a palavra reservada *import*, será exibida uma lista dos módulos que se pode importar.

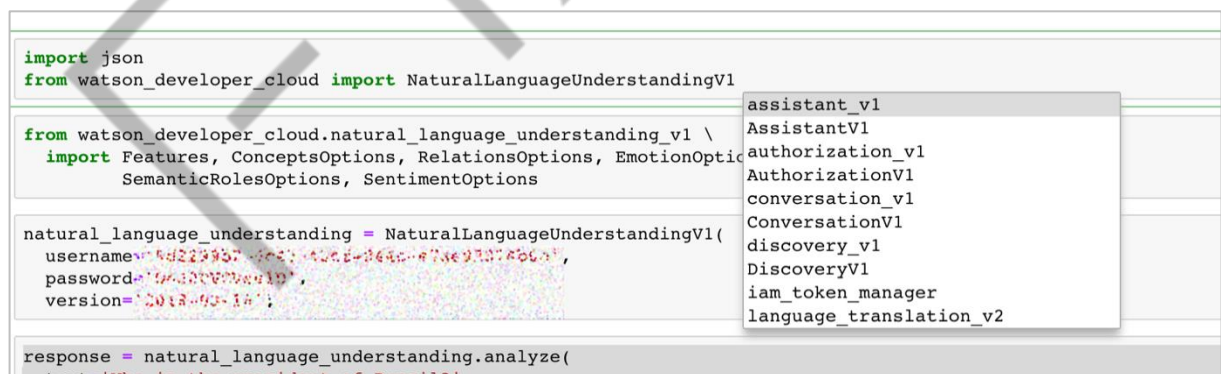


Figura 7.8. Verificando mais módulos a serem importados (*auto-complete*)
Fonte: Imagem gerada pelo autor (2018)

Outra funcionalidade interessante e útil do Jupyter é verificar a assinatura de um método, para isso, com o cursor dentro dos parênteses de um método, aperte *<shift>* + *<tab>*. Experimente!

Instanciando um objeto de acesso à API

Antes de chamarmos a API, temos de instanciar um objeto. Conforme é apresentado no código a seguir, o objeto `natural_language_understanding` é uma instância da classe `NaturalLanguageUnderstandingV1`. Na instanciação é necessário indicar o usuário (`username`) e senha (`password`), assim como a data da versão dos modelos de IA que serão utilizados (`version`).

```
natural_language_understanding = NaturalLanguageUnderstandingV1(  
    username='coloque_aqui_seu_usuario',  
    password='aqui_sua_senha',  
    version='2018-03-16')
```

Código-Fonte 7.2. Instanciando o objeto de acesso à API
Fonte: Desenvolvido pelo autor (2018)

As informações de usuário e senha são obtidas logo na habilitação do serviço. Reveja a Figura 7.4, na seção de credenciais (*credentials*).

Consultando a API

Como um objeto, instanciamos e fazemos seu uso para chamar a API de interpretação textual, conforme o exemplo do código abaixo. O método `analyze()` é chamado e seu retorno é posto na variável `response`.

```
response = natural_language_understanding.analyze(  
    text='Who is the president of Brazil?',  
    features=Features(  
        concepts=ConceptsOptions(),  
        emotion=EmotionOptions(),  
        entities=EntitiesOptions(),  
        sentiment=SentimentOptions(),  
    ))
```

Código-Fonte 7.3. Exemplo de consulta à API de interpretação textual (NLU) do Watson
Fonte: Desenvolvido pelo autor (2018)

Interessante notar que, no exemplo, o texto apresentado (argumento `text`) está em Inglês. Isso será feito para mostrar melhor as características de análise textual da API, com seus modelos padrões de IA. Para algumas das características (*features*) não há modelos padrões para o Português. Mais adiante, testaremos alguns textos em Português para ver o resultado.

O retorno da API, colocado em `response`, está estruturado como um dicionário. Então, para ver melhor o retorno, utilizamos o código a seguir.

```
print(json.dumps(response, indent=2))
```

Código-Fonte 7.4. Exibição identada do retorno da chamada à API

Fonte: Desenvolvido pelo autor (2018).

O retorno é apresentado a seguir. Nessa resposta, cada chave na raiz do dicionário diz respeito a uma das características (*features*) referenciadas na chamada à API.

```
{
  "emotion": {
    "document": {
      "emotion": {
        "anger": 0.149442,
        "joy": 0.151672,
        "sadness": 0.117418,
        "fear": 0.047087,
        "disgust": 0.198362
      }
    }
  },
  "sentiment": {
    "document": {
      "score": 0.0,
      "label": "neutral"
    }
  },
  "language": "en",
  "entities": [
    {
      "relevance": 0.33,
      "text": "president",
      "type": "JobTitle",
      "count": 1
    },
    {
      "relevance": 0.33,
      "text": "Brazil",
      "disambiguation": {
        "subtype": [
          "GovernmentalJurisdiction",
          "CompanyShareholder",
          "Country"
        ],
        "name": "Brazil",
        "dbpedia_resource": "http://dbpedia.org/resource/Brazil"
      },
      "type": "Location",
      "count": 1
    }
  ],
  "concepts": [
    {
      "relevance": 0.886784,
      "text": "President",
      "dbpedia_resource": "http://dbpedia.org/resource/President"
    },
    {
      "relevance": 0.862208,
      "text": "President of the United States",
      "dbpedia_resource": "http://dbpedia.org/resource/President_of_the_United_States"
    }
  ]
}
```

```

    },
    {
      "relevance": 0.845824,
      "text": "Politics of Brazil",
      "dbpedia_resource": "http://dbpedia.org/resource/Politics_of_Braz
il"
    }
  ],
  "usage": {
    "text_characters": 31,
    "features": 5,
    "text_units": 1
  }
}

```

No retorno, na característica “*sentiment*”, por exemplo, vemos que o texto é neutro, ou seja, não apresenta algum sentimento. Trata-se apenas de uma pergunta.

Já como entidades no texto (*entities*), foram identificadas duas: “Brazil” e “president”. Para “Brazil” apresentou-se que é um local e um país, além de um link para uma página da DBPedia, que contém mais informações sobre o país “Brazil”. Para a entidade “president”, a API também informou que se trata de um título de uma função, ou trabalho (“JobTitle”).

Veja e analise as outras características que a API identificou no texto.

```

response = natural_language_understanding.analyze(
    text='Steve Jobs is the founder of Apple',
    features=Features(
        entities=EntitiesOptions(),
        semantic_roles=SemanticRolesOptions(),
    ))
print(json.dumps(response, indent=2))

```

Código-Fonte 7.5. Outra chamada à API de NLU
Fonte: Desenvolvido pelo autor (2018)

Outra chamada, bem semelhante, para ilustrar a característica de papéis semânticos (*semantic roles*) é feita no código “Outra chamada à API de NLU”. A saída é exibida a seguir:

```

{
  "usage": {
    "text_characters": 34,
    "features": 2,
    "text_units": 1
  },
  "semantic_roles": [
    {
      "action": {
        "text": "is",

```

```

        "verb": {
            "text": "be",
            "tense": "present"
        },
        "normalized": "be"
    },
    "sentence": "Steve Jobs is the founder of Apple",
    "object": {
        "text": "the founder of Apple"
    },
    "subject": {
        "text": "Steve Jobs"
    }
}
],
"language": "en",
"entities": [
    {
        "relevance": 0.33,
        "text": "Steve Jobs",
        "disambiguation": {
            "subtype": [
                "BoardMember",
                "CompanyFounder",
                "ComputerDesigner",
                "FilmProducer"
            ],
            "name": "Steve Jobs",
            "dbpedia_resource": "http://dbpedia.org/resource/Steve_Jobs"
        },
        "type": "Person",
        "count": 1
    },
    {
        "relevance": 0.33,
        "text": "founder",
        "type": "JobTitle",
        "count": 1
    },
    {
        "relevance": 0.33,
        "text": "Apple",
        "type": "Company",
        "count": 1
    }
]
}

```

Por exemplo, na análise de papéis semânticos (*semantic_roles*), o sujeito é “*Steve Jobs*”, o verbo é “*is*” e o objeto é “*the founder of Apple*”. Como entidades, são apresentadas “*Steve Jobs*”, como uma pessoa, e “*Apple*”, como uma companhia.

Já imaginou o trabalho de desenvolver um modelo com aprendizado de máquina para identificar tais tipos de informações em textos dos mais variados domínios? Essa é uma das grandes vantagens do uso de APIs já prontas, para as quais foram gastos muitos recursos para atingir tal grau de acurácia e maturidade.

Como o retorno já vem como um objeto json, quaisquer processos que deseje realizar com as informações retornadas é imediato no Python. Não é necessário ficar convertendo de um tipo para outro.

Teste com seus próprios textos. Pense em como utilizar as informações retornadas para tomada de decisão e programas que lidam com informações textuais.

Identificação da Língua do texto

A API de interpretação textual (NLU) faz a identificação automática da língua do texto passado como argumento. Se nós alterarmos o código para utilizar o texto “Steve Jobs é o fundador da Apple”, teremos o seguinte retorno:

```
{
  "usage": {
    "text_characters": 32,
    "features": 2,
    "text_units": 1
  },
  "semantic_roles": [],
  "language": "en",
  "entities": [
    {
      "relevance": 0.33,
      "text": "Steve Jobs",
      "disambiguation": {
        "subtype": [
          "BoardMember",
          "CompanyFounder",
          "ComputerDesigner",
          "FilmProducer"
        ],
        "name": "Steve Jobs",
        "dbpedia_resource": "http://dbpedia.org/resource/Steve_Jobs"
      },
      "type": "Person",
      "count": 1
    },
    {
      "relevance": 0.33,
      "text": "Apple",
      "type": "Company",
      "count": 1
    }
  ]
}
```

```
]
}
```

Observe que a API continua identificando o texto como sendo em Inglês (*language: “en”*), mas não consegue fazer a identificação dos papéis semânticos como no retorno do código “Outra chamada à API de NLU”. Mesmo assim, identifica as mesmas duas entidades: “Steve Jobs” e “Apple”.

Se fizermos outra requisição, com várias características (*features*), com um texto em Português, que seja identificado em Português, diversas características não irão funcionar. Veja no código a seguir:

```
response = natural_language_understanding.analyze(
    text='Na FIAP, os alunos são muito dedicados.',
    features=Features(
        relations=RelationsOptions(),
        concepts=ConceptsOptions(),
        emotion=EmotionOptions(),
        entities=EntitiesOptions(),
        semantic_roles=SemanticRolesOptions(),
        sentiment=SentimentOptions(),
    )
)

print(json.dumps(response, indent=2))
```

Código-Fonte 7.6. Chamada à API com texto em Português
Fonte: Desenvolvido pelo autor (2018)

O retorno da última chamada é:

```
{
  "sentiment": {
    "document": {
      "score": 0.0,
      "label": "neutral"
    }
  },
  "language": "pt",
  "warnings": [
    "concepts: unsupported text language: pt",
    "emotion: unsupported text language: pt",
    "semantic_roles: unsupported text language: pt"
  ],
  "relations": [],
  "entities": [],
  "usage": {
    "text_characters": 39,
    "features": 3,
    "text_units": 1
  }
}
```

É apresentada uma chave “warnings” em que se diz que algumas características não são suportadas para o Português. Nesses casos, é possível o

desenvolvimento de modelos específicos para Línguas e para as características desejadas. Acesse o link a seguir para ter uma ideia de como fazer isso.

<https://console.bluemix.net/catalog/services/knowledge-studio>

Explore mais

O procedimento anterior é introdutório e, portanto, um tanto superficial. Se deseja ir mais a fundo, consulte o guia das APIs. Por exemplo, para a API explorada, de interpretação textual (*Natural Language Understanding*), você pode acessar o link a seguir e ver mais opções:

<https://www.ibm.com/watson/developercloud/natural-language-understanding/api/v1/>

Veja também as opções para o Processamento de Imagens em:

<https://www.ibm.com/watson/developercloud/visual-recognition/api/v3/python.html>

REFERÊNCIAS

IBM DEVELOPER. **API Explorer**. [s.d.]. Disponível em: <<https://developer.ibm.com/api/list>>. Acesso em: 24 abr. 2018.

RUSSEL, Stuart; NORVIG, Peter. **Inteligência Artificial**. Rio de Janeiro: Elsevier, 2013.

EVANGELISTA

GLOSSÁRIO

API	Sigla para <i>Application Programming Interface</i> . Um conjunto de métodos para integração de software.
------------	---

EVANGELISTA

INTELIGÊNCIA ARTIFICIAL
E COMPUTACIONAL

Serviços COGNITIVOS

ERICK GALANI MAZIERO



8

LISTA DE FIGURAS

Figura 8.1. Exemplo de notebooks para implementação do projeto.....	15
---	----

EVANGELISTA

LISTA DE TABELAS

Tabela 8.1. Definição do problema.....	7
Tabela 8.2. Dados disponíveis para o projeto	8
Tabela 8.3. Ferramentas indicadas para o desenvolvimento.	9
Tabela 8.4. APIs disponíveis para uso no projeto.	9
Tabela 8.5. Forma de avaliação	11
Tabela 8.6. Disponibilização da solução	12
Tabela 8.7. Definição do projeto.....	14

EVANGELISTA

LISTA DE CÓDIGOS-FONTE

Código-Fonte 8.1. Algoritmo da solução	10
--	----

EVANGELISTA

SUMÁRIO

8 SERVIÇOS COGNITIVOS	6
8.1 Definição do projeto.....	6
8.1.1 Definição do problema	7
8.1.2 Dados disponíveis	8
8.1.3 Ferramentas e APIs	8
8.1.4 Desenvolvimento	10
8.1.5 Forma de avaliação	11
8.1.6 Disponibilização da solução	11
8.2 Hands On!	14
REFERÊNCIAS.....	16
GLOSSÁRIO	17

8 SERVIÇOS COGNITIVOS

Agora que você tem conhecimentos da área de Inteligência Artificial e, inclusive, já desenvolveu algumas aplicações pequenas e ilustrativas usando técnicas de Aprendizado de Máquina, que tal desenvolver um projeto na área? Vamos fazer um passo a passo para você atacar um problema, desde a sua definição até o desenvolvimento de uma solução, sua avaliação e disponibilização.

É muito importante a definição inicial do escopo de um projeto, pois atacar (sair desenvolvendo) um problema sem algumas definições iniciais é como atirar sem um alvo, querendo atingir muita coisa, sem sucesso. Além disso, por se tratar de um projeto em que visaremos o uso de técnicas de Aprendizado de Máquina, a disponibilidade de dados é essencial, principalmente em uma abordagem supervisionada. Mesmo em uma abordagem não supervisionada, os dados são necessários, ainda que não anotados de alguma forma. Por fim, como você avaliará o sucesso de sua solução? Definir uma forma de avaliação é importante, seja ela intrínseca ou extrínseca.

A seguir, são apresentadas diversas etapas e em algumas delas podem aparecer conceitos e técnicas com as quais talvez você não esteja familiarizado. Portanto, gaste um tempo pesquisando a respeito e tente implementar!

8.1 Definição do projeto

Ao mesmo tempo que você será guiado na definição do seu projeto, um exemplo será apresentado para auxiliá-lo a entender cada passo. Depois dessa seção, você poderá iniciar o desenvolvimento da solução com maiores chances de sucesso. Vale salientar que, no momento do desenvolvimento, alguns ajustes na definição serão necessários, aumentando ou diminuindo o escopo, modificando os dados, ferramentas, etc.

8.1.1 Definição do problema

Bom, vamos partir da definição do problema. Como exemplo, considere a tabela “Definição do problema”.

Tabela 8.1. Definição do problema

Problema	Diversos aplicativos para telefones celulares são disponibilizados com problemas até então desconhecidos por seus desenvolvedores.
Lacunas	Os testes unitários e funcionais elaborados pelos desenvolvedores podem não cobrir todo o código criado ou todas as funcionalidades do aplicativo. Portanto, outras formas de identificar problemas devem ser elaboradas.
Hipóteses	A análise dos comentários e revisões dos usuários em uma loja virtual de aplicativos, o Google Play, pode indicar problemas não identificados pelos desenvolvedores.
Objetivos	Analisar os comentários e revisões de aplicativos com técnicas de PLN (Processamento da Linguagem Natural), mais especificamente análise de sentimentos, com o intuito de levantar possíveis problemas, para auxiliar os desenvolvedores na melhoria da qualidade do software desenvolvido.

Fonte: Desenvolvido pelo autor (2018)

A tabela “Definição do problema” apresenta a definição de um problema clássico em desenvolvimento de softwares. Nem sempre os desenvolvedores conseguem fazer o teste completo de todo o código e funcionalidades de um aplicativo para smartphones, por exemplo. Essa é uma lacuna que pode ser melhor elaborada com técnicas de Inteligência Artificial em larga escala. Então, define-se uma hipótese em que se acredita que a análise de comentários a respeito de um aplicativo em uma loja virtual possa conter relatos de problemas encontrados pelos usuários e que não sejam de conhecimento dos desenvolvedores.

Baseado nessa hipótese, traça-se como objetivo específico o uso de uma técnica de PLN chamada Análise de Sentimentos, para identificar possíveis problemas

em aplicativos, por meio da análise dos comentários em uma loja virtual, o Google Play.

8.1.2 Dados disponíveis

Para atingir o objetivo definido, temos que verificar se temos dados disponíveis, assim como certificar que tais dados têm licença que possibilita o uso. No caso dos comentários do Google Play (Tabela 8.2), a Google disponibiliza o acesso de forma gratuita por meio de uma API (<https://cloud.google.com/apis/>).

Tabela 8.2. Dados disponíveis para o projeto

Dados disponíveis	Comentários (<i>Reviews</i>) dos aplicativos na Google Play.
Licença	Acesso livre e gratuito

Fonte: Desenvolvido pelo autor (2018)

Cada comentário será classificado em positivo ou negativo pela análise de sentimentos. Os comentários negativos serão selecionados para identificar as palavras-chave (substantivos). Esses substantivos podem indicar o problema encontrado pelo usuário, ou a funcionalidade que não tem o comportamento esperado.

Uma versão mais rebuscada desse projeto poderia ser com o uso da análise de sentimentos voltada a aspectos, que em um mesmo comentário identifica partes do texto que são positivas e partes negativas. Para cada parte, identifica a entidade avaliada, assim como o aspecto da entidade. Por exemplo, a interface de um aplicativo pode ser uma entidade e o tamanho da fonte, um aspecto, que pode ser positivo ou negativo.

8.1.3 Ferramentas e APIs

Como ferramentas (Tabela 8.3), vamos definir a linguagem de programação, seus pacotes e APIs que serão utilizadas para atingir o objetivo do projeto. Como de costume, vamos utilizar a linguagem Python, interpretada no Jupyter. Para disponibilizar a solução na forma de uma API Rest, utilizaremos o Flask.

Tabela 8.3. Ferramentas indicadas para o desenvolvimento

Ferramentas	Python, Jupyter, Flask
--------------------	------------------------

Fonte: Desenvolvido pelo autor (2018)

Como APIs (Tabela 8.4) utilizaremos a API do Google para acesso aos dados. Para realizar a análise de sentimentos, utilizaremos a API do Watson para o processamento da linguagem natural (NLU – *Natural Language Understanding*), que, dentre as várias análises, faz a análise de sentimentos.

Para cada API é importante saber a forma de acesso, se exige algum tipo de autenticação. No caso das duas APIs citadas, ambas exigem acesso com autenticação, uma com conta Google (<https://developers.google.com/android-publisher/authorization>), outra, IBM.

Outra informação que se deve saber é a precificação. No caso da API da Google, para acesso aos comentários, é necessário um registro como desenvolvedor no Google Play com o pagamento de uma tarifa (<https://play.google.com/apps/publish/signup/#ApiAccessPlace>). Já no caso da IBM Watson, são possíveis apenas 30 mil itens NLU por mês no plano gratuito. Para entender um item NLU, segundo o console da API (<https://cloud.ibm.com/catalog/services/natural-language-understanding>), um NLU consiste em um texto de 10 mil caracteres e uma extração, por exemplo, a identificação de entidades.

Tabela 8.4. APIs disponíveis para uso no projeto

APIs	Google, IBM Watson
Precificação	<i>Google</i> : exige registro de desenvolvedor <i>IBM Watson</i> : 30.000 itens NLU por mês
Acesso	<i>Google</i> : com autenticação <i>IBM Watson</i> : com autenticação

Fonte: Desenvolvido pelo autor (2018)

8.1.4 Desenvolvimento

Agora que já sei o que vou utilizar, tenho que definir meus algoritmos. Como busco os dados, em que momento chamo a API. Como lido com os retornos e como gero as minhas respostas.

No exemplo de análise de sentimentos (AS) de comentários, o seguinte algoritmo de alto nível poderia ser definido no Código-Fonte 8.1.

```
Obtenha o package_name (id do app) da requisição
Busque os comentários sobre o app
Para cada comentário faça
    Pegue o texto do comentário e chame a API de AS
    Se for positivo, guarde na lista_positivos
    Senão, guarde na lista_negativos
Para cada comentário em lista_positivos faça
    Pegue os substantivos e armazene em subst_positivos
    Faça a contagem da frequência dos substantivos
    Ordene a contagem decrescente
Para cada comentário em lista_negativos faça
    Pegue os substantivos e armazene em subst_negativos
    Faça a contagem da frequência dos substantivos
    Ordene a contagem decrescente
Gere a resposta do serviço
Retorne a resposta da requisição
```

Código-Fonte 8.1. Algoritmo da solução
Fonte: Desenvolvido pelo autor (2018)

O uso de Inteligência Artificial está encapsulado na chamada à API do Watson, que faz todo o trabalho de PLN. Caso essa solução não atenda aos requisitos, principalmente após a avaliação da solução (próxima subseção), o ideal é fazer o treinamento de um novo modelo de PLN que faça a AS com bons resultados para o problema.

Cada um dos passos do Código-Fonte 8.1 contém diversos outros passos, omitidos aqui por se tratar de um exemplo para você desenvolver a sua solução, para o seu problema.

A solução proposta será disponibilizada por meio de um serviço REST, conforme será explicado a seguir.

8.1.5 Forma de avaliação

Para fazer a avaliação da solução, serão selecionados, manualmente, 20 comentários obtidos manualmente de um mesmo aplicativo, sendo 10 positivos e 10 negativos. Para definir um comentário positivo ou negativo, serão utilizadas as estrelas (score) dos comentários, validados manualmente. A validação manual é necessária, pois um comentário pode ter 5 estrelas, mas ser negativo, e vice-versa.

O ideal é que os comentários positivos e negativos sejam validados por mais de uma pessoa, de forma que haja concordância entre os validadores. Por questão de simplicidade, você mesmo pode fazer isso.

Tabela 8.5. Forma de avaliação

Dados de teste	Criação de um conjunto de teste com 20 comentários. Esse conjunto deve ser balanceado, com 10 comentários positivos e 10 negativos. Seleção dos substantivos dos comentários.
Avaliação intrínseca	Executar a solução sobre os dados de teste e apresentar as métricas de acurácia, precisão e cobertura.

Fonte: Desenvolvido pelo autor (2018)

Na Tabela 8.5 é apresentada uma avaliação intrínseca da solução projetada. Uma avaliação intrínseca considera a solução como um fim, ou seja, por meio de algumas métricas, verifica o quão correto está seu comportamento. No exemplo, quantos dos exemplos positivos e negativos são corretamente identificados com a solução proposta. Caso tenha alguma dúvida com relação às métricas de avaliação, retorne ao capítulo de aprendizagem de máquina e veja algumas métricas que foram apresentadas.

8.1.6 Disponibilização da solução

Feito o desenvolvimento da solução, como tornar essa implementação facilmente acessível aos usuários? O exemplo apresentado neste capítulo pode ser útil para os desenvolvedores de aplicativos, que podem consultar as análises dos comentários de seus aplicativos em busca de melhorias.

Uma forma de tornar simples e fácil o acesso a uma solução é disponibilizá-la na forma de um serviço REST (*REpresentational State Transfer*). Essa forma de

acessar aplicações tem sido amplamente utilizada e possibilita a interoperabilidade de sistemas diversos.

Tabela 8.6. Disponibilização da solução

Disponibilização	A solução será disponibilizada através de um serviço REST
Endpoints	<code>http://meuservico:8080/ api/analyzereviews/package_name</code>

Fonte: Desenvolvido pelo autor (2018)

Para a linguagem utilizada em nossa solução exemplo, o Python, temos um *framework* chamado Flask (<https://palletsprojects.com/p/flask/>) que permite a criação de serviços REST a partir de seu código Python. Essa é a forma de disponibilização do serviço indicada no projeto exemplo, conforme a Tabela 8.6Disponibilização da solução”.

Quando se define um serviço REST, faz-se necessário disponibilizar cada funcionalidade da solução por meio de um *endpoint*. No exemplo, temos apenas um *endpoint*, definido como `http://meuservico:8080/api/analyzereviews/package_name`, visto que definimos uma funcionalidade em que simplesmente é feita a análise de comentários de um dado aplicativo.

Quando o serviço é executado, ele fica disponível para acesso na rede e geralmente é necessário definir uma porta (8080). O final do *endpoint* (`package_name`) deve ser substituído pelo código do aplicativo que se deseja analisar, por exemplo, `http://meuservico:8080/api/analyzereviews/com.spiffygame`.

Para acessar esse *endpoint*, o usuário deve fazer uma requisição HTTP em qualquer linguagem de programação, ou através de um browser qualquer. O retorno da requisição, geralmente, é feito no formato JSON (*Javascript Object Notation*) que é equivalente ao dicionário, do Python. Você deve, então, definir a estrutura do retorno, com suas chaves e valores. Por exemplo:

```
{ "analyzereviews":  
  
  {  
  
    "package_name": "com.spiffygame",
```

```
    "number_reviews": 10,  
    "positive_reviews":  
    {  
        "number": 3,  
        "most_common_nouns": ["jogabilidade", "gráficos", ...]  
        "reviews": [  
            "Texto do comentário 1...",  
            "Texto do comentário 2 ...",  
            ...  
        ]  
    }  
    "negative_reviews":  
    {  
        "number": 7,  
        "most_common_nouns": ["botões", "travamento", ...]  
        "reviews": [  
            "Texto do comentário 1...",  
            "Texto do comentário 2 ...",  
            ...  
        ]  
    }  
}  
}
```

8.2 Hands On!

Agora que já foram apresentados e ilustrados os passos para a definição do projeto, é sua vez de projetar! Comece preenchendo a tabela a seguir (Tabela 8.7). Lembre-se de que, durante a implementação, algumas informações poderão mudar.

Tabela 8.7. Definição do projeto

Problema	
Problema	
Lacunas	
Hipóteses	
Objetivos	
Dados	
Dados disponíveis	
Licença	
Ferramentas e APIs	
Ferramentas	
APIs	
Precificação	
Acesso	
Avaliação	
Dados de teste	
Avaliação intrínseca	
Disponibilização	
Disponibilização	
Endpoints	

Fonte: Desenvolvido pelo autor (2018)

Agora, utilizando a linguagem Python no Jupyter, você já pode começar a criar seus notebooks, conforme ilustrado na imagem a seguir. Lembre-se que o Jupyter notebook dá a facilidade de verificar a execução de cada trecho de código logo após o código, numa mesma tela, inclusive com a utilização de gráficos.

Já a disponibilização de sua solução será feita em um ou mais arquivos python (.py) que será chamado pelo Flask para a criação de um serviço REST, tornando seus *endpoints* disponíveis na rede.

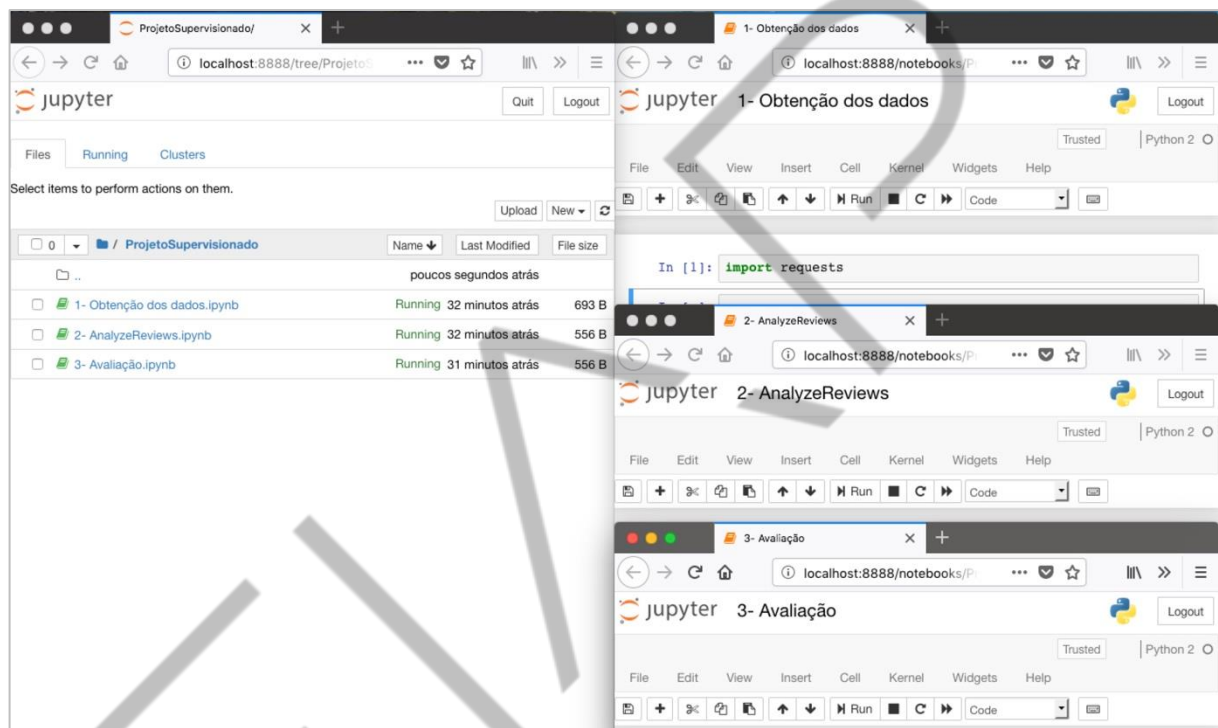


Figura 8.1. Exemplo de notebooks para implementação do projeto
Fonte: Desenvolvido pelo autor (2018)

Há muito a falar sobre cada um dos passos abordados neste capítulo, mas com o que foi apresentado você já consegue ter uma boa noção de como desenvolver um projeto com Inteligência Artificial e pode vislumbrar o caminho a seguir para criar suas próprias soluções com IA.

REFERÊNCIAS

GOOGLE PLAY DEVELOPER API. **Reviews**. [s.d.]. Disponível em: <<https://developers.google.com/android-publisher/api-ref/reviews>>. Acesso em: 26 jul. 2018.

IBM. **Api explorer**. [s.d.]. Disponível em: <<https://developer.ibm.com/api/list>>. Acesso em: 26 jul. 2018.

EVANGELISTA

GLOSSÁRIO

Endpoint	Endereço (URL) de um serviço disponível para acesso por uma aplicação.
-----------------	--

EVANGELISTA